

Site Didactic

Unități de învățare

Limbaje de programare pentru server

Dezvoltarea limbajului PHP

Server web local

Editoare de cod PHP

Inițiere în PHP

Variabile

Constante

Operatori

Structuri condiționale

Structuri repetitive

Tipul tablou

Tipul șir de caractere, în PHP

Funcții în PHP

Funcții predefinite în PHP

Prelucrarea datei și orei în PHP

Clase și obiecte în PHP

Construiește un site cu PHP

Securitatea paginilor web

Preluarea datelor din formulare

Validarea datelor trimise serverului

Baze de date în aplicații web

- Stil obiect orientat de programare

- Stil procedural de programare

Citirea conținutului unui director

Lucrul cu fișiere

Creare imagini cu PHP

PHP - GD Library

Ce este World Wide Web?

Greșeli de web design

Limbaje de programare pentru server

Tehnologiile Server Side Scripting permit crearea de aplicații Web complexe prin procesarea datelor pe server și generarea unor pagini în mod dinamic. În felul acesta se pot interfața aplicațiile Web cu servere de baze de date sau servere de poștă electronică. Limbajele Server Side Script au posibilitatea de a accesa datele citite în formulare HTML și implementează biblioteci de acces la resurse externe cum ar fi o bază de date. Spre deosebire de tehnologii mai vechi care realizau același deziderat (Ex. CGI) server side scripting asigură o viteză și o securitate sporită. În general interpretoarele limbajelor SSS sunt integrate sub formă de module chiar în serverul de Web. Exemple de limbaje SSS: ASP, ColdFusion Markup Language, JSP, PHP, Perl, Python, Ruby etc.

Dezvoltarea limbajului PHP. Caracteristicile de bază.

PHP, înseamnă "PHP: Hypertext Preprocessor" este un limbaj de scripting de uz general, cu cod-sursă deschis (Open Source), utilizat pe scară largă, și care este potrivit în special pentru dezvoltarea aplicațiilor Web și poate fi integrat în HTML. Sintaxa sa provine din C, Java și Perl și este ușor de învățat. Scopul principal al limbajului este acela de a permite programatorilor web să creeze rapid pagini web generate dinamic, însă cu PHP puteți realiza mult mai multe.

PHP/FI

PHP succede un produs mai vechi, numit PHP/FI. PHP/FI a fost creat de către **Rasmus Lerdorf** în 1995, inițial doar ca un simplu set de scripturi Perl pentru a urmări accesele la rezumatul său online. El a denumit acest set de scripturi 'Personal Home Page Tools – Instrumente pentru paginile personale'. Deoarece era necesară mai multă funcționalitate, Rasmus a scris o implementare mult mai largă în C, care era în stare să comunice cu baze de date și dădea posibilitate utilizatorilor să dezvolte simple aplicații Web dinamice. Rasmus a hotărât să lanseze codurile-sursă ale PHP/FI pentru ca oricine să le poată vedea, utiliza, să poată corecta erorile din ele și să îmbunătățească codul-sursă.

PHP/FI, care se descifra ca Personal Home Page / Forms Interpreter — Interpretor al paginilor personale / al formularelor, includea unele funcționalități de bază ale PHP pe care îl cunoaștem astăzi. El posedă variabile asemănătoare cu cele din Perl, interpretare automată a variabilelor din formulare și sintaxă inclusă în HTML. Însăși sintaxa era similară cu cea din Perl, cu toate că era mult mai limitată, simplă și într-un anumit fel inconsistentă.

În 1997, PHP/FI 2.0, a doua re-scriere a implementării în C, avea un cult din câțiva mii de utilizatori din lume (estimat), cu aproximativ 50,000 domenii, care raportau că îl au instalat, ceea ce reprezenta aproximativ 1% din toate domeniile din Internet. Cu toate că erau câteva persoane, care contribuiau porțiuni de coduri-sursă în acest proiect, el era totuși în mare măsură un proiect al unui singur om.

PHP/FI 2.0 a fost oficial lansat doar în noiembrie 1997, după ce și-a petrecut o bună parte a vieții în versiuni beta. În scurt timp el a fost înlocuit de versiunile alfa ale PHP 3.0.

PHP 3

PHP 3.0 a fost prima versiune care se aseamănă mult cu PHP, așa cum îl cunoaștem astăzi. El a fost creat de către Andi Gutmans și Zeev Suraski în 1997 ca o re-scriere completă, după ce ei au descoperit că PHP/FI 2.0 avea o capacitate foarte redusă pentru a dezvolta o aplicație de comerț electronic, asupra căreia ei lucrau într-un proiect al Universității. Într-un efort de a coopera și a continua dezvoltarea PHP/FI cu baza existentă de utilizatori, Andi,

Rasmus și Zeev au decis să coopereze și au anunțat PHP 3.0 în calitate de succesor oficial al PHP/FI 2.0, iar dezvoltarea PHP/FI 2.0 a fost în mare măsură stopată.

Unul din cele mai mari avantaje ale PHP 3.0 erau facilitățile sale largi de extindere. În afară de faptul că oferea utilizatorilor finali o infrastructură solidă pentru o multitudine de baze de date, protocoale și interfețe API, facilitățile de extindere ale PHP 3.0 au impulsionat zeci de programatori să se alăture și să înainteze noi module de extindere. Discutabil, aceasta a fost cheia succesului enorm al PHP 3.0. Alte facilități de bază introduse în PHP 3.0 au fost susținerea sintaxei orientate pe obiecte și o sintaxă a limbajului mult mai puternică și consistentă.

Noul limbaj a fost lansat sub un nou nume, care excludea implicarea unei utilizări personale limitate, pe care o avea numele PHP/FI 2.0. El a fost numit pur și simplu 'PHP', sensul fiind un acronim recursiv - PHP: Hypertext Preprocessor — Preprocesor al Hipertextului.

Spre sfârșitul anului 1998, PHP a atins un număr de zeci de mii de utilizatori (estimat) și sute de mii de site-uri Web raportau că îl au instalat. În punctul său de vîrf, PHP 3.0 a fost instalat aproximativ pe 10% din toate serverele Web din Internet.

PHP 3.0 a fost lansat oficial în iunie 1998, după ce a petrecut aproximativ 9 luni în faza de testare publică.

PHP 4

Către iarna anului 1998, la scurt timp după ce PHP 3.0 a fost lansat oficial, Andi Gutmans și Zeev Suraski au început lucrul asupra rescrierii nucleului PHP. Scopurile reproiectării erau de a îmbunătăți performanța aplicațiilor complexe și de a îmbunătăți modularitatea bazei de coduri-sursă ale PHP. Astfel de aplicații puteau fi realizate datorită noilor facilități din PHP 3.0, precum și a susținerii unei game largi de baze de date și interfețe API ale terților producători, însă PHP 3.0 nu fusese proiectat să lucreze cu astfel de aplicații complexe în mod eficient.

Noul nucleu, denumit 'Zend Engine' (compus din numele autorilor, Zeev și Andi), a făcut față cerințelor de proiect cu succes, și a fost prezentat pentru prima dată la mijlocul anului 1999. PHP 4.0, bazat pe acest nucleu și în cuplu cu o gamă largă de facilități noi a fost lansat oficial în mai 2000, aproape doi ani după predecesorul său, PHP 3.0. Suplimentar la performanța îmbunătățită substanțial în această versiune, PHP 4.0 a inclus alte facilități de bază, cum ar fi susținerea mult mai multor servere Web, sesiunilor HTTP, buferizarea ieșirii, metode mai securizate de gestionare a datelor introduse de utilizatori și câteva construcții noi ale limbajului.

Astăzi PHP este utilizat de sute de mii de programatori (estimat) și câteva milioane de site-uri raportează că îl au instalat, ceea ce alcătuiește peste 20% din toate domeniile din Internet.

Echipa de dezvoltare a PHP include zeci de programatori, precum și alte zeci de persoane ce lucrează asupra proiectelor asociate cu PHP, cum ar fi PEAR și proiectul de documentare.

PHP 5

Pe 13 iulie 2004 a fost lansat PHP 5, cu Zend Engine II, ce a adus și o orientare obiect mai pronunțată și suportînd mai multe caracteristici ale acestui tip de programare.

PHP 5 aduce mai multe noutăți față de versiunea 4:

Suport îmbunătățit pentru OOP

Introduce extensia PDO - PHP Data Objects, care definește o modalitate facilă și consistentă de accesare a diferitelor baze de date

Îmbunătățiri de performanță

Suport îmbunătățit pentru MySQL și MSSQL

Suport nativ pentru SQLite

- Suport SOAP integrat
- Iteratori pentru date
- Controlul erorilor prin tratarea de excepții

La sfîrșitul lui 2007 doar versiunea 5.x mai era întreținută, deoarece în data de 13 iulie 2007 (exact la 3 ani după lansarea PHP5), PHP Group a anunțat că PHP4 va fi scos din uz pe 31 decembrie 2007 , deși prognozează că anumite upgrade-uri de securitate se vor oferi pînă pe 8 august.

PHP 6

La versiunea PHP 6 s-a lucrat mai bine de 5 ani cu rezultate modeste și după tot acest efort s-a renunțat la această versiune în 2010. Acest efort considerabil nu a rămas fără urmări iar diferite caracteristici se regăsesc în versiunile PHP 5.3 și PHP 5.4. Versiunile PHP 5.3 și PHP 5.4 au venit cu diferite caracteristici foarte impresionante și din acest motiv mulți programatori considera că versiunea PHP 5.4 ar fi trebuit să devină versiunea PHP 6.0 a limbajului însă nu a fost cazul.

Din cauza că diferite caracteristici ce trebuiau implementate în versiunea PHP 6 au fost implementate în PHP 5.3 și PHP 5.4 și întreaga documentație cu tot ce urma să vină în PHP 6 a fost publică timp de 5 ani atunci s-a luat decizia să se renunțe la versiunea PHP 6 și să fie omisă pentru a nu exista nici-o confuzie în legătură cu noile caracteristici ale limbajului PHP.

PHP 7

PHP 7 reprezintă singura lansare a acestui limbaj din ultimii 11 ani și înainte de a fi lansat a stat o perioada de 2 ani în care a fost dezvoltat. PHP 7 vine cu o nouă versiune a motorului ZEND, ceea ce îl face să fie de doua ori mai rapid folosind mai puțină memorie. PHP 7 vine cu un suport considerabil pentru platformele ce rulează pe 64 de biți.

PHP 7 a fost lansat în data de 3 Decembrie 2015.

PHP 7 este de doua ori mai rapid decât orice versiune anterioară. PHP 7 convertește mult mai rapid liniile noastre de cod în format binar și folosește mult mai puțină memorie pentru acest proces. Pentru a înțelege mai bine uitați-vă la următoarele date ce reprezintă cererile pe care PHP 5.6 și PHP 7.0 le poate procesa într-o secundă pe diferite platforme.

	PHP 5.6	PHP 7
WordPress	191 cereri/secundă	374 cereri/secundă
Drupal	182 cereri/secundă	316 cereri/secundă
Magento	41 cereri/secundă	69 cereri/secundă
Laravel	256 cereri/secundă	485 cereri/secundă
Zend Framework	250 cereri/secundă	489 cereri/secundă

PHP 7 este proiectat și refactored având în vedere volumul de lucru de astăzi.

Popularitate:

PHP-ul este unul din cele mai folosite limbaje de programare server-side, conform unui studiu efectuat de Netcraf în aprilie 2002, apărînd pe 9 din cele 37 milioane de domenii cercetate în studiu. De asemenea, există un grafic al

creșterii folosirii PHP-ului pe site-ul oficial. Popularitatea de care se bucură acest limbaj de programare se datorează următoarelor caracteristici:

Familiaritatea: sintaxa limbajului este foarte ușoară combinând sintaxele unora din cele mai populare limbaje Perl sau C

Simplitate: sintaxa limbajului este destul de liberă. Nu este nevoie de includere de biblioteci sau de directive de compilare, codul PHP inclus într-un document executându-se între marcajele speciale;

Eficiența: PHP-ul se folosește de mecanisme de alocare a resurselor, foarte necesare unui mediu multiutilizator, așa cum este web-ul;

Securitate: PHP-ul pune la dispoziția programatorului un set flexibil și eficient de măsuri de siguranță;

Flexibilitate: fiind apărut din necesitatea dezvoltării web-ului, PHP a fost modularizat pentru a bine pasul cu dezvoltarea diferitelor tehnologii. Nefiind legat de un anumit server web, PHP-ul a fost integrat pentru numeroasele servere web existente: Apache, IIS, Zeus, server, etc.

Gratuitate : este probabil cea mai importantă caracteristică a PHP-ului. Dezvoltarea PHP-ului sub licența open-source a determinat adaptarea rapidă a PHP-ului la nevoile web-ului, eficientizarea și securizarea codului.

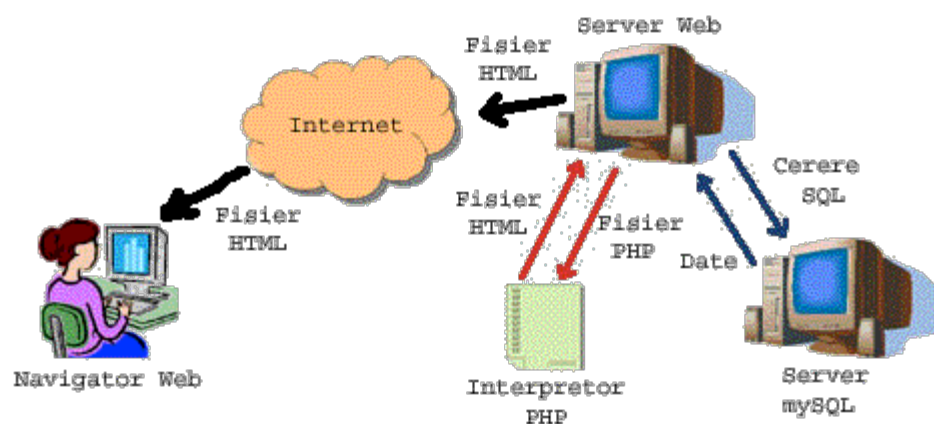
Utilizare:

PHP este simplu de utilizat, fiind un limbaj de programare structurat, ca și C-ul ,Perl-ul sau începând de la versiunea 5 chiar Java , sintaxa limbajului fiind o combinație a celor trei. Datorită modularității sale poate fi folosit și pentru a dezvolta aplicații de sine stătătoare, de exemplu în combinație cu PHP-GTK sau poate fi folosit ca Perl sau Python în linia de comandă. Probabil una din cele mai importante facilități ale limbajului este conlucrarea cu majoritatea bazelor de date relaționale, de la MySQL și pînă la Oracle, trecînd prin MS Sql Server, PostgreSQL, sau DB2.

PHP poate rula pe majoritatea sistemelor de operare, de la UNIX ,Windows, sau Mac OS X și poate interacționa cu majoritatea serverelor web. Codul dumneavoastră PHP este interpretat de serverul WEB și generează un cod HTML care va fi văzut de utilizator (clientului -browserului- fiindu-i transmis numai cod HTML).

Arhitectura tip LAMP a devenit populară în industria web ca modalitate rapidă, gratuită și integrată de dezvoltare a aplicațiilor. Alături de Linux, Apache și Mysql, PHP reprezintă litera P, deși uneori aceasta se referă la Python sau Perl. Linux ocupă rolul de sistem de operare pentru toate celelalte aplicații, MySQL gestionează bazele de date, Apache are rol de server web, iar PHP are rol de interpretator și comunicator între acestea.

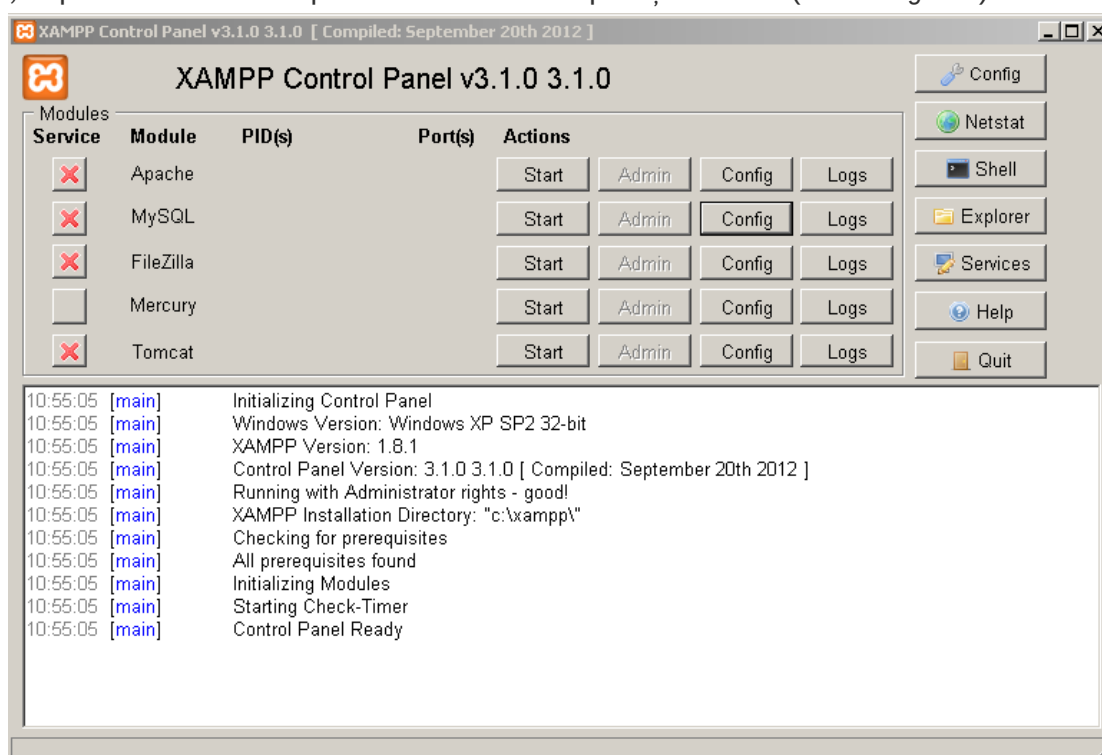
PHP folosește extensii specifice pentru fișierele sale: .php, .php3, .ph3, .php4, .inc, .phtml. Aceste fișiere sunt interpretate de către serverul web iar rezultatul este trimis în formă de text sau cod HTML către browser-ul clientului.



Serverul Web încarcă pagina din sistemul de fișiere propriu și, constatând că este vorba de o pagină PHP o trimite interpretorului PHP. Interpretorul PHP parcurge pagina și execută instrucțiunile PHP întâlnite. Instrucțiunile PHP au ca și scop principal acela de a genera cod HTML. De asemenea, codul HTML din fișierul original este copiat pur și simplu la ieșire. Interpretorul PHP transmite înapoi către server fișierul transformat prin interpretarea codul PHP. Acest fișier va conține doar cod HTML (eventual si Client Side Script etc.) dar nu și cod PHP.

Server web local

Pentru a putea rula scenarii php(scripturi) avem nevoie să programăm calculatorul în calitate de server Web, server de aplicații PHP și nu în ultimul rând server de BD MySQL. Aceste componente software de bază necesare (MySQL, Apache și PHP) sunt gratuite (din categoria "open source") și pot fi descărcate de pe site-ul oficial al producătorului. După instalarea fiecăruia din aceste aplicații apare necesitatea configurării lor, care uneori este dificilă pentru începători. Pentru a evita problemele configurării, putem instala un pachet complet ce conține Apache + PHP + MySQL + PHPMyAdmin care se numește **XAMPP (PHPTriad, EasyPhp, Base, WampServer, Denver, ...)**. Aplicația PHPMyAdmin inclusă în pachetul **XAMPP** permite administrarea comodă a BD MySQL. Înainte de a rula scripturi PHP vom lansa aplicația **XAMPP** și anume din *Start/All Programs/Apache Friends/XAMPP/XAMPP Control Panel*, după care se va lansa panoul de control al aplicației **XAMPP**(vezi imaginea).



Pentru a lansa serverul Apache acționăm butonul **start** la **modulul Apache** și respectiv dacă lucrăm cu BD acționăm butonul **start** la **la modulul MySql**. Pentru a închide serverele deschise vom acționa respectiv pe butoanele **stop** iar pentru a închide aplicația **XAMPP** vom acționa **exit**.

Pentru a verifica dacă funcționează aplicațiile instalate, vom lansa browserul iar în câmpul de adrese scriem:

http://localhost/xampp în loc de *localhost* se poate de scris și *IP* calculatorului sau *127.0.0.1* .Pentru a rula scripturile ele trebuiesc salvate(copiate) în *C:\xampp\htdocs*.

Editoare de cod

Întrucât sintaxa limbajului PHP este relativ simplă, iar structura codului sursă nu este una impusă, codul PHP poate fi scris cu orice editor text, cum ar fi Notepad. În realitate, acest lucru este posibil doar pentru scripturile de câteva linii de cod; pentru fișiere mari și complexe, un editor PHP specializat este aproape indispensabil.

Exista mai multe editoare de cod-sursă ce pot fi folosite pentru PHP, de la IDE-uri complexe pînă la editoare simple. Toate oferă **highlight** pentru cuvintele cheie, completare automată a instrucțiunilor, facilități avansate de căutare și altele. Puteți folosi pe oricare dintre ele, în funcție de nevoile voastre.

1. Dreamweaver - un program fascinant pentru creare de pagini web. Potențialul său este mai larg decît celelalte editoare de pagini web. Dreamweaver vă permite să creați obiecte și pagini web folosind DHTML, sprijină Cascading Style Sheets, precum și straturi și JavaScript.

2. PHP Designer - un mediu de dezvoltare pentru limbaje ca PHP, HTML, XHTML, CSS și SQL (suporta Ingres, Interbase, MSSQL, MySQL, Oracle si Sybase). Pachetul mai include un browser, un client FTP, un file manager, biblioteci de coduri (pentru PHP, SQL si HTML) și un modul pentru conversia documentelor în formate ca LaTeX, HTML si RTF.

3. Notepad ++ - un editor de coduri sursă, care indeplinește foarte bine și funcțiile "Notepad" din windows. Limbajele de programare suportate sunt foarte variate (ASP, Ada, ASCII art, Assembly, AutoIt, CSS, doxygen, FORTRAN, HTML, Haskell, Java, JavaScript, BAT, C, C#, C++, Caml, KiXtart, Lisp, Lua, makefile, Matlab, Objective-C, Pascal, Perl, PHP, Smalltalk, SQL, Tcl, TeX, Verilog, VHDL, VB/VBScript, XML, PostScript, Python, Ruby, Scheme, Unix Shell Script), suportă "drag and drop" și poate fi folosit în locul clasicului "Notepad" pentru editarea de texte fără nici un fel de problemă. Programul nu necesita resurse hardware mai mari decăt "Notepad-ul" din Windows.

Un plus în ceea ce privește editarea codurilor sursă este faptul că programul poate modifica (edita) același fișier în mai multe formate de vizualizare avînd loc sincronizarea fișierului în ambele formate și de asemenea se pot tipări codurile sursă pe hîrtie.

Notepad++ - un notepad mult mai complex. Proiectul include foarte multe plug-in-uri ajutatoare pentru anumite editari complexe. Softul este gratuit (licenta GPL).

4. ConTEXT - un editor mic, rapid și puternic de text freeware, dezvoltat pentru a servi ca un instrument secundar pentru dezvoltatorii de software.

5. Zend Studio - un mediu de dezvoltare pentru limbajul de programare PHP, care include optiuni pentru editare cod, diagnosticare și testare.

6. Eclipse - o platformă integrată de dezvoltare (IDE) extensibilă, ce dispune de o multitudine de funcționalități. Folosită inițial pentru dezvoltarea în limbajul Java, a fost ulterior extinsă și la alte limbaje de programare. Eclipse PDT (Eclipse PHP Development Tools) reprezinta la ora actuala cel mai avansat editor PHP. Este gratuit și poate rula pe orice sistem de operare. Mai multe informații pot fi gasite pe site-ul proiectului **Eclipse PDT**.

7. Netbeans - vine din zona programării în Java. Este un IDE complet ce a fost extins recent pentru lucrul în PHP. Ca și Eclipse poate fi folosit pe orice sistem de operare întrucît rulează pe platforma Java. Poate fi descărcat gratuit de la <http://netbeans.org/downloads/> (alegeti pachetul PHP).

Modul de lucru în Netbeans este similar cu cel din Eclipse. Avem și aici conceptul de **proiect** iar pentru a putea crea scripturi PHP este nevoie să creați în prealabil un proiect de tip PHP. Toate celelalte observații făcute la editorul Eclipse rămîn valabile și aici.

8. Sublime Text - Sublime Text 3 este un editor de text care e la modă de ceva timp, datorită faptului că reușește să îmbine o mulțime de aspecte pe care ți le-ai dori de la un editor într-un program mic, dar foarte puternic. Sublime Text 3 folosește un sistem de API Python pentru pluginuri și teme, care sunt incredibil de folositoare.

9. Atom - Atom este un editor gratuit și open-source dezvoltat de cei de la GitHub ca răspuns la o serie de editoare extrem de populare cum ar fi Sublime Text, TextWrangler, TextMate si Textastic, preluând o parte din

feature-urile acestora. Spre diferență de acestea însă, Atom oferă posibilitatea de a modifica aproape toate aspectele editorului de către utilizator.

Inițiere în PHP

Taguri PHP, instrucțiuni, punct și virgulă

Codul PHP trebuie inclus în fișierele ce vor fi deservite utilizatorilor finali și trebuie delimitat de restul conținutului prin tagul de început `<?php` și tagul de sfârșit `?>`. Practic tot ce se află în interiorul acestor etichete va fi interpretat ca fiind cod-sursa PHP (cod ce va fi executat de interpretorul PHP). Textul din afara celor 2 taguri este lăsat neschimbat, fiind ulterior interpretat de browser. Se pot folosi și alte etichete în funcție de configurația serverului web, dar acestea nu sunt recomandate.

```
1
2 <?php
3 /* forma recomandata */
   ?>
```

```
1
2 <?php
3 /* forma recomandata */
   ?>
```

```
1
2 <script language="php">
3 /* forma disponibila oricand, putin folosita */
   </script>
```

```
1
2 <script language="php">
3 /* forma disponibila oricand, putin folosita */
   </script>
```

```
1
2 <?
3 /* taguri scurte, acceptarea lor de catre web-server depinde de configurare */
   ?>
```

```
1
2 <?
3 /* taguri scurte, acceptarea lor de catre web-server depinde de configurare */
   ?>
```

```
1
2 <%
3 /* taguri in stilul asp, acceptarea lor de catre web-server depinde de configurare */
   %>
```

```
1
2 <%
3 /* taguri in stilul asp, acceptarea lor de catre web-server depinde de configurare */
   %>
```

Codul PHP este format din **instrucțiuni** - comenzi date către interpretor, în urma cărora se execută ceva. Așa cum

s-a mai spus, PHP este folosit în principal pentru a genera cod HTML, așa că de cele mai multe ori instrucțiunile folosite sunt cele de afișare. În aplicații mai complexe se pot însă folosi și instrucțiuni de conectare la bazele de date, de citire/scriere/manipulare fișiere, instrucțiuni de trimitere email-uri și altele. Instrucțiunile se pot grupa în **blocuri** delimitate de acolade { }. Aceste blocuri, numite și "instrucțiuni complexe" se comportă ca și cum ar fi o singură instrucțiune (vezi mai jos exemple de blocuri).

Toate instrucțiunile (în afară de blocuri) trebuie să fie terminate cu **punct și virgulă** (;). Lipsa acestuia generează o eroare.

Instrucțiuni uzuale în PHP: echo, print, printf

În multe din scripturile scrise de începători (și nu numai) conținutul rezultat în urma execuției este un text (care poate fi sau nu cod HTML). Pentru a obține acest text, secvențele de cod trebuie să conțină instrucțiuni explicite care să "spună" interpretorului ce anume trebuie afișat.

Instrucțiunile de afișare sunt **print**, **echo** și **printf**. Există și alte modalități dar aceste 3 instrucțiuni sunt cele mai uzuale și ușor de folosit. Diferențe între ele există, dar ca idee generală **print** și **echo** fac același lucru și poate fi folosită oricare dintre ele. **printf** este folosită mai rar datorită sintaxei oarecum greoaie.

```

1  <?php
2  # cele 2 instructiuni de mai jos sunt echivalente
3  print "Text";
4  echo "Text";
5
6
7  # o instructiune echo poate primi mai multi parametrii
8  echo "Afisez", " un text din ", 4, " bucati";
9
10 # o singura instructiune print poate primi doar unul
11 print "Afisez";
12 print " un text din ";
13 print 4;
14 print " bucati";
15
16 # printf este folosita pentru a formata continutul, la fel ca in C/C++
17 printf( "Am %.2f lei", 102.1234 ); // afiseaza Am 102.12 lei
?>

```

```

1  <?php
2  # cele 2 instructiuni de mai jos sunt echivalente
3  print "Text";
4  echo "Text";
5
6
7  # o instructiune echo poate primi mai multi parametrii
8  echo "Afisez", " un text din ", 4, " bucati";
9
10 # o singura instructiune print poate primi doar unul
11 print "Afisez";
12 print " un text din ";
13 print 4;
14 print " bucati";
15
16 # printf este folosita pentru a formata continutul, la fel ca in C/C++
17 printf( "Am %.2f lei", 102.1234 ); // afiseaza Am 102.12 lei
?>

```

În cazul paginilor statice create cu HTML utilizatorul poate vedea doar imagini și text. Folosind PHP însă vom putea crea așa numitele pagini dinamice în care apare elementul de interactivitate: se pot prelua date din formulare realizate în PHP, citi, prelucra și afișa. De asemenea se poate face interacțiunea cu o bază de date creată pe un server.

Crearea primului script php

```
1
2 <?php
3 echo "Primul script php";
?>
```

```
1
2 <?php
3 echo "Primul script php";
?>
```

Obs: echo - afișează un mesaj

Combinarea codului HTML cu cod PHP

Acest lucru se întâmplă foarte des. În interiorul unui script HTML poate exista script PHP delimitat de etichetele **<?php** și **?>** însă fișierul final trebuie să fie PHP.

Exemplu:

```
1
2 <html>
3 <body>
4 Text combinat HTML cu PHP
5 <?php
6 echo "Text scris cu PHP";
7 ?>
8 </body>
</html>
```

```
1
2 <html>
3 <body>
4 Text combinat HTML cu PHP
5 <?php
6 echo "Text scris cu PHP";
7 ?>
8 </body>
</html>
```

Comentarii în scripturi:

- un tip de comentariu pentru un rând;

// - un alt tip de comentariu pentru un rând;

/* și */ - pentru mai multe rânduri.

Variabile PHP

Variabilele sunt mărimi care pot lua diferite valori. Numele dat unei variabile începe întotdeauna cu simbolul \$ urmat de o literă sau caracterul "_". Nu poate fi precedat niciodată de o cifră. Restul caracterelor care formează numele variabilei pot fi litere, cifre sau liniuțe de subliniere. În PHP, sunt considerate litere toate caracterele cuprinse între 'a' și 'z', cele cuprinse între 'A' și 'Z', precum și cele care au codul ASCII cuprins între 127 și 255.

Definim o variabilă astfel:

\$<variabila>=<valoare>;

Exemplu :

```

1 // variabile numerice
2 $variabila = 1;
3 $numar = 0;
4
5 // variabile text (stringuri)
6 $text = "Salut";
7 $text2 = 'Ce faci?';
8
9 // variabile logice (boolean)
10 $stiuPHP = true;
11 $uitRepede = false;
12
13 // vector
14 $vectorGol = array();
15 $vectorS = array( 10,11,12,13 ); // vector simplu
16
17 // vector asociativ
18 $vector = array(
19     'luni' => 'Monday',
20     'marti' => 'Tuesday',
21     'miercuri' => 'Wednesday'
22 );
23
24 // definirea unei noi componente a unui vector asociativ
25 $vector[ 'joi' ] = 'Thursday';
26
27 // vector simplu definit ca asociativ, echivalentul lui $vectorS
28 $vectorS2 = array(
29     0 => 10,
30     1 => 11,
31     2 => 12,
32     3 => 13
33 );

```

```

1 // variabile numerice
2 $variabila = 1;
3 $numar = 0;
4
5 // variabile text (stringuri)
6 $text = "Salut";
7 $text2 = 'Ce faci?';
8
9 // variabile logice (boolean)
10 $stiuPHP = true;
11 $uitRepede = false;
12
13 // vector
14 $vectorGol = array();
15 $vectorS = array( 10,11,12,13 ); // vector simplu
16
17 // vector asociativ
18 $vector = array(
19     'luni' => 'Monday',
20     'marti' => 'Tuesday',
21     'miercuri' => 'Wednesday'
22 );
23
24 // definirea unei noi componente a unui vector asociativ
25 $vector[ 'joi' ] = 'Thursday';
26
27 // vector simplu definit ca asociativ, echivalentul lui $vectorS
28 $vectorS2 = array(
29     0 => 10,
30     1 => 11,
31     2 => 12,
32     3 => 13
33 );

```

```

1
2 <?php
3     $a = "PHP5";
4     $b = &$a;
5     $b = "PHP e foarte bun";
6     echo $a;
7     echo "<br>";
8     echo $b;
9 ?>

```

```

1
2 <?php
3     $a = "PHP5";
4     $b = &$a;
5     $b = "PHP e foarte bun";
6     echo $a;
7     echo "<br>";
8     echo $b;
9 ?>

```

```

1
2 <?php
3     $nume = "Ciobanu";
4     $prenume = "Andrei";
5     $virsta = "22 ani";
6     $tara = "Moldova";
7     echo "Ma numesc ".$nume.$prenume.", am ".$virsta.", si locuiesc in ".$tara;
8 ?>

```

```

1
2 <?php
3     $nume = "Ciobanu";
4     $prenume = "Andrei";
5     $virsta = "22 ani";
6     $tara = "Moldova";
7     echo "Ma numesc ".$nume.$prenume.", am ".$virsta.", si locuiesc in ".$tara;
8 ?>

```

Atentie: PHP face distinctie între literele mari și mici. Astfel, cele 3 variabile de mai jos sunt TOTAL diferite:

```

1
2 $variabila = "negru";
3 $vaRiabila = "alb";
4 $Variabila = 1;

```

```

1
2 $variabila = "negru";
3 $vaRiabila = "alb";
4 $Variabila = 1;

```

Atribuirea de valori

Se realizează la fel ca și declararea prin intermediul instrucțiunii de atribuire.

```

1 # atribuirea unei valori anume
2 $variabila = 2;
3
4 # copierea valorii de la o alta variabila
5 $text = $variabila;
6
7
8 # copierea valorii functioneaza si in cazul vectorilor
9 # in urma copierii cei 2 vectori vor fi identici

```

```

9
10 $vectorS = $vectorS2; # toate componentele lui $vectorS2 sunt copiate in $vectorS
11
12 # in cazul vectorilor se pot atribui valori separat fiecarei componente
13 $vectorS[ 0 ] = 100;
14 $vectorS[ 3 ] = 10;
15
16 $vector[ 'luni' ] = 'Lundi';
   $vector[ 'joi' ] = 'Jeudi';

1
2 # atribuirea unei valori anume
3 $variabila = 2;
4
5 # copierea valorii de la o alta variabila
6 $text = $variabila;
7
8 # copierea valorii functioneaza si in cazul vectorilor
9 # in urma copierii cei 2 vectori vor fi identici
10 $vectorS = $vectorS2; # toate componentele lui $vectorS2 sunt copiate in $vectorS
11
12 # in cazul vectorilor se pot atribui valori separat fiecarei componente
13 $vectorS[ 0 ] = 100;
14 $vectorS[ 3 ] = 10;
15
16 $vector[ 'luni' ] = 'Lundi';
   $vector[ 'joi' ] = 'Jeudi';

```

Stergerea variabilelor

De obicei nu e nevoie că variabilele să fie șterse după ce au fost folosite. Totuși, lucrul acesta se poate face în următoarele moduri:

```

1
2 unset( $variabila );
   $variabila = null;

1
2 unset( $variabila );
   $variabila = null;

```

Afișarea variabilelor

Afișarea se realizează utilizând oricare din instrucțiunile de afișare:

```

1
2 $variabila = "Text";
3 $randNou = "<br>";
4
5 print $variabila; print $randNou;
   echo $variabila, $randNou;

1
2 $variabila = "Text";
3 $randNou = "<br>";
4
5 print $variabila; print $randNou;
   echo $variabila, $randNou;

```

Afișarea tipului de date a variabilei

Afișarea tipului de date a variabilei se realizează utilizând funcția **gettype(\$var)**:

```
1
2 <?php
3 echo gettype(102). '<br>';
4 echo gettype(true). '<br>';
5 echo gettype(' '). '<br>';
6 echo gettype(null). '<br>';
7 echo gettype(array()). '<br>';
8 echo gettype(new stdClass());
?>
```

```
1
2 <?php
3 echo gettype(102). '<br>';
4 echo gettype(true). '<br>';
5 echo gettype(' '). '<br>';
6 echo gettype(null). '<br>';
7 echo gettype(array()). '<br>';
8 echo gettype(new stdClass());
?>
```

Output :

integer
boolean
string
NULL
array
object

Variabile globale

\$GLOBALS - pot fi accesate toate variabilele globale care sunt accesibile script-ului PHP curent; acest vector este indexat chiar prin numele variabilelor globale. Această variabilă superglobală este accesibilă începând cu versiunea 3.0.0

\$_SERVER - conține o serie de variabile ale căror valori sunt setate de server-ul web; majoritatea valorilor variabilelor din acest vector depind de mediul de execuție al script-ului curent. A fost introdusă începând de la versiunea 4.1.0. În versiunile anterioare se folosea **\$HTTP_SERVER_VARS**.

Este o variabilă care conține informații cum ar fi headere, locația scriptului, sau informații despre vizitator și browser. Este posibil ca în funcție de setările fiecărui server și browser să nu fie active toate informațiile specificate de această variabilă. Această variabilă este de asemenea globală, adică se poate folosi oriunde în script, fără apelarea ei ca funcție globală (adică nu trebuie să o cereți cu **global \$_SERVER** în funcțiile dvs.) În schimb **\$HTTP_SERVER_VARS** conține aceleași informații dar nu este o funcție globală (atenție **\$HTTP_SERVER_VARS** și **\$_SERVER** sunt două variabile diferite, chiar dacă aparent fac aceleași lucruri) Dacă directiva *register_globals* este ON atunci aceste variabile vor deveni la rîndul lor variabile globale în cadrul scriptului. Elementele acestor variabile sunt cele de mai jos (atenție: în funcție de setările serverului și browserului unele elemente pot lipsi)

' PHP_SELF ' - conține numele fișierului unde se execută scriptul curent, relativ la rădăcină (document root) De exemplu, **\$_SERVER['PHP_SELF']** într-un script la adresa *http://example.com/test.php/foo.bar* va fi */test.php/foo.bar*. Constanta **__FILE__** conține toată calea și numele fișierului curent.

- ' **argv** ' - Un array al argumentului trimis către script. Când scriptul rulează în linie de comandă, va da acces C-style la parametrii liniei de comanda. Când este accesat prin metoda GET, acesta va conține un sir de interogare.
- ' **argc** ' - Conține un număr al parametrilor liniei de comandă trimise către script (daca este rulat în linie de comanda).
- ' **GATEWAY_INTERFACE** ' - Returneaza versiunea CGI utilizată pe server (exemplu ' CGI/1.1 ').
- ' **SERVER_NAME** ' - Numele server-ului de hosting unde se execută scriptul curent. Dacă scriptul rulează pe un host virtual, aceasta va conține valoarea definită pentru acel host virtual.
- ' **SERVER_SOFTWARE** ' - Returnează un șir de identificare al server-ului dat în header ca raspuns la cerere.
- ' **SERVER_PROTOCOL** ' - Numele și versiunea protocolului prin care este cerută pagina (exemplu ' HTTP/1.0 ')
- ' **REQUEST_METHOD** ' - returnează metoda utilizată pentru accesarea paginii (exemplu ' GET ', ' HEAD ', ' POST ', ' PUT ')
- ' **REQUEST_TIME** ' - Valoarea *timestamp* dată la pornirea cererii. Este valabilă începînd de la versiunea PHP 5.1.0.
- ' **QUERY_STRING** ' - Sirul de interogare, dacă este, de la pagina care a fost accesată.
- ' **DOCUMENT_ROOT** ' - Directorul rădăcină unde este rulat scriptul curent, așa cum este definit în fișierul de configurație al serverului.
- ' **HTTP_ACCEPT** ' - Conținutul header-ului acceptat de la cererea curentă, dacă este unul.
- ' **HTTP_ACCEPT_CHARSET** ' - Conține setul de caractere acceptate și care este trimis de header-ul cererii respective, dacă există. Exemplu: ' iso-8859-1,*,utf-8 '.
- ' **HTTP_ACCEPT_ENCODING** ' - Conține tipul de encodare trimis de cerera curentă prin header, dacă există.. Exemplu: ' gzip '.
- ' **HTTP_ACCEPT_LANGUAGE** ' - Conține limbaajul trimis de cererea curentă prin header, dacă exista. Exemplu: ' en '.
- ' **HTTP_CONNECTION** ' - Conține informații despre conexiunea trimisă de cererea curentă prin header, dacă există. Exemplu: ' Keep-Alive '.
- ' **HTTP_HOST** ' - Conține informații despre host, header-ul de la cererea curentă, dacă există.
- ' **HTTP_REFERER** ' - Adresa paginii (dacă este) de unde a venit userul în pagina curentă. Aceasta este setată de către browser. Nu toate browserele setează această valoare, și chiar unele au posibilitatea de a modifica *HTTP_REFERER* așa cum vor. Pe scurt, nu este de încredere.
- ' **HTTP_USER_AGENT** ' - Conține semnătura browser-ului trimisă prin header la cererea curentă, dacă există. Un exemplu tipic este: Mozilla/4.5 [en] (X11; U; Linux 2.2.9 i586) . În afară de asta, puteți utiliza **get_browser()** pentru a vedea ce capabilitați are browserul folosit de user și astfel să returnați pagina specifică pentru acel browser.
- ' **HTTPS** ' - Setează o valoare dacă scriptul a fost cerut printr-un protocol securizat HTTPS
- ' **REMOTE_ADDR** ' - Adresa de IP a userului care accesează pagina curentă.
- ' **REMOTE_HOST** ' - Numele hostului userului care vede pagina curentă. Este inversul DNS-ului userului bazat pe *REMOTE_ADDR*. Atenție: Serverul web trebuie configurat pentru a crea această variabila. De exemplu în Apache va trebui *HostnameLookups On* în fișierul de configurare *httpd.conf*. Pentru alte detalii vedeți și funcția **gethostbyaddr()** .
- ' **REMOTE_PORT** ' - Portul prin care userul comunică cu serverul web.
- ' **SCRIPT_FILENAME** ' - Calea absolută unde se execută scriptul curent. Atenție: Dacă scriptul este rulat cu CLI, ca și cale relativă, cum ar fi file.php sau ../file.php , \$_SERVER['SCRIPT_FILENAME'] va conține calea relativă specificată de către user.
- ' **SERVER_ADMIN** ' - Valoarea dată de directiva *SERVER_ADMIN* (pentru Apache) în fișierul de configurare al web serverului. Dacă scriptul rulează pe un host virtual, acesta va conține valoarea definită pentru acel host virtual.

- ' **SERVER_PORT** ' - Portul de pe server care este folosit pentru comunicare pe web. Pentru setările standard, acesta este ' 80 '; pentru SSL, de exemplu, se va schimba cu oricare port definit în transmisia securizată HTTP.
- ' **SERVER_SIGNATURE** ' - Conține un șir care cuprinde versiunea serverului și numele hostului virtual dacă este.
- ' **PATH_TRANSLATED** ' - Calea de bază a sistemul de fișiere (nu rădăcina documentului) de la scriptul curent, după ce serverul a terminat maparea oricăror cai (virtual-to-real mapping) Atenție: Începînd de la PHP 4.3.2, **PATH_TRANSLATED** nu mai este setat implicit în Apache 2 SAPI în contrast cu situația din Apache 1, unde este setată aceeași valoare ca și la **SCRIPT_FILENAME**. Această schimbare a fost făcută pentru a fi compatibilă cu specificațiile CGI în care **PATH_TRANSLATED** trebuie să existe doar dacă **PATH_INFO** este definit. Utilizatorii Apache 2 pot utiliza *AcceptPathInfo = On* în fișierul de configurație *httpd.conf* pentru a defini **PATH_INFO** .
- ' **SCRIPT_NAME** ' - Conține calea către scriptul curent. Acesta este util pentru pagini care trebuiesc definite către ele însuși. Constanta `__FILE__` conține calea completă și numele de fișier al scriptului curent.
- ' **REQUEST_URI** ' - URI care este dat pentru accesarea paginii curente, de exemplu ' /index.html '.
- ' **PHP_AUTH_DIGEST** ' - Cînd rulați sub Apache ca modul, acțiunea *Digest HTTP*, valoarea acestei variabile este setată în 'Authorization' header trimis către client (care ar trebui utilizat pentru a face o validare corespunzătoare).
- ' **PHP_AUTH_USER** ' - Cînd este rulat sub Apache sau IIS (ISAPI pe PHP 5) ca modul, acțiunea *HTTP authentication*, această variabilă conține numele userului trimis de către user.
- ' **PHP_AUTH_PW** ' - Cînd este rulat sub Apache sau IIS (ISAPI pe PHP 5) ca modul, acțiunea *HTTP authentication*, această variabilă conține parola userului trimisă de către user.
- ' **AUTH_TYPE** ' - Cînd este rulat sub Apache ca modul acțiunea *HTTP authenticated* , această variabilă seteaza tipul de autentificare.

Variabilele pentru trimiterea datelor

\$_GET este o variabilă array globală. A fost introdusă în versiunea 4.1.0. Pentru versiunile anterioare se folosește **\$HTTP_GET_VARS** care nu este variabilă globală. Puteți folosi **\$_GET** pentru a trimite variabile cu valori prin intermediul linkurilor.

\$_POST este de asemenea o variabilă array globală. A fost introdusă în versiunea 4.1.0. Pentru versiunile anterioare se folosește **\$HTTP_POST_VARS** care nu este variabilă globală. Puteți folosi **\$_POST** pentru a trimite variabile cu valori prin intermediul formularelor.

\$_GET și **\$_POST** conțin variabile primite de script prin intermediul unor transferuri care folosesc metodele HTTP *get*, respectiv *post*. De exemplu, prin intermediul acestor vectori pot fi accesate valorile câmpurilor dintr-un formular care a fost completat și transmis folosind una dintre cele doua metode.

Variabilele HTTP cookies

\$_COOKIE conține valorile variabilelor care conțin informații referitoare la cookie-urile păstrate pe calculatorul utilizatorului care accesează pagina web. A fost introdus în versiunea 4.1.0. În versiunile anterioare, utilizați **\$HTTP_COOKIE_VARS** .

Variabile pentru uploadul fișierelor

\$_FILES conține variabile primite de script prin intermediul încărcărilor de fișiere prin metoda *post*. A fost introdus în versiunea 4.1.0. În versiunile anterioare se folosește **\$HTTP_POST_FILES** care nu este variabilă globală.

Variabilele de mediu \$_ENV

\$_ENV conține variabile disponibile prin intermediul mediului în care este executat. A fost introdus în versiunea 4.1.0. În versiunile anterioare, utilizați **\$HTTP_ENV_VARS** .

Variabilele pentru cereri

\$_REQUEST conține variabile disponibile prin intermediul oricărui tip de mecanism cu ajutorul căruia utilizatorul poate introduce date. Este de tip array și conține valorile variabilelor **\$_GET** , **\$_POST** , și **\$_COOKIE** . A fost introdusă în versiunea 4.1.0. Începând de la versiunea 4.3.0, informațiile din **\$_FILES** au fost de asemenea incluse în **\$_REQUEST**

Variabile de sesiune

\$_SESSION conține variabile care corespund sesiunii curente a script-ului, sub formă de array. A fost introdusă în versiunea 4.1.0 și este de asemenea globală. În versiunile anterioare folosiți **\$HTTP_SESSION_VARS**.

Variabile de mesaje de eroare

\$php_errormsg - este o variabilă ce conține ultimul mesaj generat de către PHP Această variabilă este folosită în scopul de a vedea ce eroare apare și doar dacă opțiunile de configurare a erorilor **track_errors** este pornita (standard este pe OFF)

Tipuri de variabile

Variabilele pot fi de mai multe tipuri, nu doar numere. PHP are opt tipuri de variabile. Patru dintre acestea sunt tipuri scalare (*boolean*, *integer*, *float* și *string*), două sunt tipuri compuse (*array* și *object*), iar alte două sunt tipuri speciale (*resource* și *null*).

În PHP, de obicei, tipul unei variabile nu este specificat de către programator, ci este stabilit în timpul execuției în funcție de contextul în care este folosită variabila.

Observație: Tipul de date este specificat prin valoarea atribuită variabilei inițiale.

Exemplu :

```
1
2 $a=1; // număr întreg (zecimal)
3 $b=3.14; // număr real
4 $c="exemplu de text"; //șir de caractere
5 $c='exemplu de text'; //șir de caractere
6 $matrice = array('alb','rosu','verde'); //matrice
   $a=TRUE; //variabilă booleana
```

```
1
2 $a=1; // număr întreg (zecimal)
3 $b=3.14; // număr real
4 $c="exemplu de text"; //șir de caractere
5 $c='exemplu de text'; //șir de caractere
6 $matrice = array('alb','rosu','verde'); //matrice
   $a=TRUE; //variabilă booleana
```

```
1
2 $a=1;
   echo "Numarul ", $a,"<br>";
```

```

1 $a=1;
2 echo "Numarul ", $a, "<br>";

```

Se va afișa : Numarul 1

În cazul unui șir de caractere definit prin ghilimele duble se pot folosi următoarele secvențe:

Secvența	Semnificație
\n	rînd nou
\r	sfîrșit de linie
\t	tab orizontal
\\	backslash
\\$	simbol dolar
\"	ghilimele duble

Constante PHP

O constantă stochează o valoare, cum este și o variabilă, dar a această valoare, după ce a fost stabilită nu mai poate fi modificată în script. Constantele sunt create cu ajutorul funcției *define()*, iar numele constantelor este scris cu Majuscule, aceasta optiune nu este obligatorie, însă va face codul dumneavoastră mai frumos și mai lizibil. O diferență importantă între constante și variabile, este faptul că o constantă nu are în fața ei semnul \$.

Exemplu:

```

1 define("PI", 3.14);
2 define('NUME', "Ciobanu");
3 define('VARSTA', 40);
4 define("ADEVARAT", true);

```

```

1 define("PI", 3.14);
2 define('NUME', "Ciobanu");
3 define('VARSTA', 40);
4 define("ADEVARAT", true);

```

```

1 <?php
2 define('AUTOR', 'Eminescu');
3 define('VARSTA', '20 ani');
4 echo AUTOR. '<br>'.VARSTA;
5 ?>

```

```

1 <?php
2 define('AUTOR', 'Eminescu');
3 define('VARSTA', '20 ani');
4 echo AUTOR. '<br>'.VARSTA;
5 ?>

```

Este recomandat ca numele constantelor sa fie scris cu majuscule (pentru a fi mai ușor identificate) deși nu este

obligatoriu. Atunci cînd se folosesc, se specifică doar numele, fără ghilimele sau \$:

```
1 | print "Ma numesc " .NUME;
2 | print " si am " .VARSTA." ani";

1 | print "Ma numesc " .NUME;
2 | print " si am " .VARSTA." ani";
```

Pentru a verifica dacă o constantă este definită, se folosește funcția **defined**

```
1 | if( defined('NUME' ) ) {
2 | print "Ma numesc " . NUME;
3 | }
4 | if (!defined( 'VARSTA' ) ) {
5 | define('VARSTA', 2 );
6 | }
7 | print " Am ". VARSTA ." ani";

1 | if( defined('NUME' ) ) {
2 | print "Ma numesc " . NUME;
3 | }
4 | if (!defined( 'VARSTA' ) ) {
5 | define('VARSTA', 2 );
6 | }
7 | print " Am ". VARSTA ." ani";
```

Operatori PHP

Interpretorul PHP permite folosirea a nouă tipuri diferite de operatori. Aceștia operează asupra unor expresii (una, două sau trei) și furnizează ca rezultat o altă expresie (rezultatul operației corespunzătoare).

Operatori aritmetici

Toți operatorii aritmetici operează asupra a două expresii (operandi). Există cinci astfel de operatori:

+	Adunare	\$a + \$b
-	Scădere	\$a - \$b
*	Înmulțire	\$a * \$b
/	Împărțire	\$a / \$b
%	Modulo(rest)	\$a % \$b

Aceste operații se aplică asupra unor valori care reprezintă tipuri numerice. Dacă unul dintre operandi nu are tip numeric, atunci el va fi convertit automat la o valoare întreagă. Dacă cel puțin unul dintre operandi (după efectuarea eventualelor conversii necesare) este un număr real, atunci rezultatul operației va fi tot un număr real, cu excepția operatorului rest; rezultatul operației de determinare a restului este întotdeauna un număr întreg. Dacă ambii operandi sunt numere întregi, atunci rezultatul va fi un număr întreg, cu excepția împărțirii al cărei rezultat este întotdeauna un număr real. Dacă operatorul rest este aplicat asupra unor numere reale, atunci rezultatul este un număr întreg. Nu este permisă împărțirea la valoarea 0. Dacă al doilea operand asupra căruia se

aplică unul dintre operatorii '/' sau '%' are valoarea 0, va fi semnalată o eroare.

Exemple:

```
1
2 <?php
3     $a = '12';
4     $b = '8';
5     $rezultat = $a + $b;
6     echo 'Rezultatul adunarii lui '.$a.' cu '.$b.' este '.$rezultat.' ';
?>
```

```
1
2 <?php
3     $a = '12';
4     $b = '8';
5     $rezultat = $a + $b;
6     echo 'Rezultatul adunarii lui '.$a.' cu '.$b.' este '.$rezultat.' ';
?>
```

Operatori de comparare

Interpretorul PHP pune la dispozitie mai mulți operatori care pot fi folosiți pentru compararea a două valori. Expresiile în care apar astfel de operatori au ca rezultat valori logice (true sau false).

Cel mai des întâlnit operator este cel de atribuire, definit prin semnul =

Dacă scriem \$x = 1 nu înseamnă că \$x este egal cu 1 ci că i s-a acordat valoarea 1. Dacă scriem \$rezultat = \$x + \$y nu înseamnă că \$rezultat este egal cu suma celor două, ci că i-am atribuit (acordat) valoarea sumei celor două.

Alți operatori folosiți pentru compararea valorilor variabilelor sunt:

'=='	Egal	\$a==\$b	rezultatul este true dacă cele două expresii au aceeași valoare
'==='	Identice	\$a=== \$b	rezultatul este true dacă cele două expresii au aceeași valoare și au același tip;
'!=' sau '<>'	Diferit	a!=\$b sau \$a<>\$b	rezultatul este true dacă cele două expresii au valori diferite;
'!=='	Neidentice	\$a!== \$b	rezultatul este true dacă cele două expresii au valori diferite și au tipuri diferite
'<'	Mai mic	\$a<\$b	rezultatul este true dacă valoarea primei expresii este mai mică decât valoarea celei de-a doua expresii
'>'	Mai mare	\$a>\$b	rezultatul este true dacă valoarea primei expresii este mai mare decât valoarea celei de-a doua expresii
'<=	Mai mic sau egal	\$a<=\$b	rezultatul este true dacă valoarea primei expresii este mai mică sau egală cu

			valoarea celei de-a doua expresii;
'>='	Mai mare sau egal	\$a>=\$b	rezultatul este true dacă valoarea primei expresii este mai mare sau egală valoarea celei de-a doua expresii

Exemplu:

```
1 |
2 | <?php
3 |     $x=1;
4 |     $y=87;
5 |
6 |     if ($x == $y) {
7 | print "$x este egal cu $y"; }
8 |
9 |     if ($x != $y) {
10 | print "$x este diferit de $y"; }
    ?>

1 |
2 | <?php
3 |     $x=1;
4 |     $y=87;
5 |
6 |     if ($x == $y) {
7 | print "$x este egal cu $y"; }
8 |
9 |     if ($x != $y) {
10 | print "$x este diferit de $y"; }
    ?>

1 |
2 | <?php
3 |     $x=5;
4 |     $y=4;
5 |
6 |     if ($x>$y) {
7 | print "$x este mai mare ca $y"; }
8 |
9 |     if ($x <= $y) {
10 | print "$x este mai mic sau egal cu $y"; }
    ?>

1 |
2 | <?php
3 |     $x=5;
4 |     $y=4;
5 |
6 |     if ($x>$y) {
7 | print "$x este mai mare ca $y"; }
8 |
9 |     if ($x <= $y) {
10 | print "$x este mai mic sau egal cu $y"; }
    ?>
```

Operatori pentru șiruri de caractere

Operatorii pentru stringuri sunt '.' (punct) pentru concatenare și '!=' (punct și egal) pentru atribuirea concatenării. Acești operatori sunt folosiți pentru a uni stringuri, în felul urmator:

```

1
2 <?php
3 // concatenarea stringurilor
4 $test='Am scris un '.'text'.'.oarecare';
5 print $test;
6 // pe ecran va fi afisat "Am scris un text oarecare"
7
8 //concatenarea stringurilor cu variabile
9 $nr=5;
10 print'Am scris numarul '.$nr.' si un text '.'oarecare';
11 // pe ecran va fi afisat "Am scris numarul 5 si un text oarecare"
12
13 // atribuirea concatenarii
14 $nr = 6;
15 $text = "Un text ";
16 $text.= "oarecare ";
17 $text.="si numarul ".$nr;
18 print $text;
19 // pe ecran va fi afisat "Un text oarecare si numarul 6"
20 ?>

```

```

1
2 <?php
3 // concatenarea stringurilor
4 $test='Am scris un '.'text'.'.oarecare';
5 print $test;
6 // pe ecran va fi afisat "Am scris un text oarecare"
7
8 //concatenarea stringurilor cu variabile
9 $nr=5;
10 print'Am scris numarul '.$nr.' si un text '.'oarecare';
11 // pe ecran va fi afisat "Am scris numarul 5 si un text oarecare"
12
13 // atribuirea concatenarii
14 $nr = 6;
15 $text = "Un text ";
16 $text.= "oarecare ";
17 $text.="si numarul ".$nr;
18 print $text;
19 // pe ecran va fi afisat "Un text oarecare si numarul 6"
20 ?>

```

Operatori pe biți și operatori logici

Interpretorul PHP pune la dispoziție șase operatori care operează asupra biților unui număr întreg sau ai unui șir de caractere. Aceștia sunt:

- conjuncție ('&') - SI (AND);
- disjuncție ('|') - SAU (OR);
- disjuncție exclusivă ('^') - SAU exclusiv (XOR);
- negație ('~') - NU (NOT);
- deplasare la stînga ('<<');
- deplasare la dreapta ('>>').

Operatorii logici vă vor veni la îndemînă în execuția scriptului atunci cînd aveți nevoie să lucrați cu valori de adevăr. Să presupunem că la intrarea în secțiunea de administrare avem un formular care cere numele și parola de acces în secțiune. Pentru contruirea expresiilor logice operanzii trebuie să aibă tipul boolean; dacă operatorii au alt tip, valoarea este convertită la o valoare booleană. Scriptul PHP ar putea verifica aceste informații pentru a autoriza accesul în secțiune folosind operatorii logici astfel:

- operatorul **'xor'** (SAU exclusiv)

Expresia în care apare operatorul **'xor'** va avea valoarea **true** dacă exact unul dintre operanzi are valoarea **true**

- operatorul **'!'** (NOT)

if (!parola_e_buna) ... parola nu este bună, accesul este interzis

if (!parola_nu_e_buna) ... parola e bună, accesul este permis

Operatorul **'!'** returnează **TRUE** dacă valoarea inițială de adevăr e **FALSE** și **FALSE** dacă valoarea inițială este **TRUE**.

- operatorul **'||'** (OR)

if (numele_este_valid || parola_este_buna) ...

verifică dacă numele sau parola sunt valide și dacă oricare dintre ele este, returnează valoarea de adevăr **TRUE**.

În acest exemplu de pseudocod dacă numele ar fi valid dar parola nu, i-am acorda utilizatorului acces mai departe, ceea ce nu este de dorit. Trebuie să fim siguri că și numele și parola sunt valide. Operatorul **'||'** returnează **TRUE** dacă oricare din valorile verificate e **TRUE**. Returnează **FALSE** doar dacă amîndouă sunt **FALSE**.

- operatorul **'&&'** (AND)

if (numele_este_valid && parola_este_buna) ...

dacă atît numele cît și parola sunt valide putem acorda utilizatorului acces în secțiunea de administrare.

Operatorul **'&&'** returnează **TRUE** doar dacă ambele valori verificate sunt **TRUE**. El returnează **FALSE** dacă oricare dintre ele este **FALSE** (sau dacă amîndouă sunt **FALSE**).

Operatori de atribuire

Primul operand asupra căruia este aplicat un astfel de operator de atribuire trebuie să fie o variabilă, iar al doilea poate fi o expresie.

Operația	Varianta scurtă	Varianta standard
adunare ('+=')	\$a += 2	\$a = \$a + 2
scădere ('-=')	\$a -= 2	\$a = \$a - 2
înmulțire ('*=')	\$a *= 2	\$a = \$a * 2
împărțire ('/=')	\$a /= 2	\$a = \$a / 2
rest ('%=')	\$a %= 2	\$a = \$a % 2
conjuncție ('&=')	\$a &= 2	\$a = \$a & 2
disjuncție (' =')	\$a = 2	\$a = \$a 2
disjuncție exclusivă ('^=')	\$a ^= 2	\$a = \$a ^ 2
deplasare la stînga ('<=<=')	\$a <<= 2	\$a = \$a << 2
deplasare la dreapta ('>>=')	\$a >>= 2	\$a = \$a >> 2
concaternare ('.=')	\$a .= "2"	\$a = \$a . "2"

Exemplu:

```

1  <?php
2  $b = 2;
3  $a = 1;
4  echo "<TT>$a += $b = ";
5  echo $a += $b;
6  $b = 2;
7  $a = 1;
8  echo "<br>$a -= $b = ";
9  echo $a -= $b;
10 $b = 2;
11 $a = 1;
12 echo "<br>$a *= $b = ";
13 echo $a *= $b;
14 $b = 2;
15 $a = 1;
16 echo "<br>$a /= $b = ";
17 echo $a /= $b;
18 $b = 2;
19 $a = 1;
20 echo "<br>$a %= $b = ";
21 echo $a %= $b;
22 $b = 2;
23 $a = 1;
24 echo "<br>$a &= $b = ";
25 echo $a &= $b;
26 $b = 2;
27 $a = 1;
28 echo "<br>$a |= $b = ";
29 echo $a |= $b;
30 $b = 2;
31 $a = 1;
32 echo "<br>$a ^= $b = ";
33 echo $a ^= $b;
34 $b = 2;
35 $a = 1;
36 echo "<br>$a <=<= $b = ";
37 echo $a <=<= $b;
38 $b = 2;
39 $a = 1;
40 echo "<br>$a >>= $b = ";
41 echo $a >>= $b;
42 $b = 2;
43 $a = 1;
44 echo "<br>$a .= $b = ";
45 echo $a .= $b;
46 echo "</TT>";
47 ?>

```

```

1  <?php
2  $b = 2;
3  $a = 1;
4  echo "<TT>$a += $b = ";
5  echo $a += $b;
6  $b = 2;
7  $a = 1;
8  echo "<br>$a -= $b = ";
9  echo $a -= $b;
10 $b = 2;
11 $a = 1;
12 echo "<br>$a *= $b = ";
13 echo $a *= $b;
14 $b = 2;
15 $a = 1;
16 echo "<br>$a /= $b = ";
17 echo $a /= $b;
18 $b = 2;
19 $a = 1;
20 echo "<br>$a %= $b = ";
21 echo $a %= $b;

```

```

22
23 $b = 2;
24 $a = 1;
25 echo "<br>$a &= $b = ";
26 echo $a &= $b;
27 $b = 2;
28 $a = 1;
29 echo "<br>$a |= $b = ";
30 echo $a |= $b;
31 $b = 2;
32 $a = 1;
33 echo "<br>$a ^= $b = ";
34 echo $a ^= $b;
35 $b = 2;
36 $a = 1;
37 echo "<br>$a <<= $b = ";
38 echo $a <<= $b;
39 $b = 2;
40 $a = 1;
41 echo "<br>$a >>= $b = ";
42 echo $a >>= $b;
43 $b = 2;
44 $a = 1;
45 echo "<br>$a .= $b = ";
46 echo $a .= $b;
47 echo "</TT>";
?>

```

Rezultatul codului de mai sus este:

```

1 += 2 = 3
1 -= 2 = -1
1 *= 2 = 2
1 /= 2 = 0.5
1 %= 2 = 1
1 &= 2 = 0
1 |= 2 = 3
1 ^= 2 = 3
1 <<= 2 = 4
1 >>= 2 = 0
1 .= 2 = 12

```

Operatori de incrementare și decrementare

Operatorul de incrementare (**++**) are ca efect creșterea cu 1 a valorii unei variabile, iar cel de decrementare (**--**) are ca efect scăderea cu 1 a valorii variabilei. Tipul variabilei poate fi întreg sau real. Dacă operatorul precede variabila, atunci rezultatul expresiei este valoarea obținută după incrementare sau decrementare. Dacă variabila precede operatorul, atunci rezultatul expresiei este valoarea variabilei înaintea incrementării sau decrementării. Cu alte cuvinte, dacă operatorul precede variabila, atunci valoarea variabilei este mai întâi modificată și apoi utilizată, iar dacă variabila precede operatorul, atunci valoarea ei este mai întâi utilizată și apoi modificată. Acești operatori pot fi utilizați și pentru variabile care conțin șiruri de caractere. Operatorul de incrementare duce la creșterea cu 1 a codului ASCII a ultimului caracter din șir dacă acesta este o literă sau cifră. În cazul în care litera este 'z', respectiv 'Z', sau cifra este '9', atunci ea devine 'a', respectiv 'A', sau cifra '0' și se încearcă incrementarea penultimului caracter. Dacă aceasta este tot '9', 'z' sau 'Z' se aplică același procedeu și se trece la antepenultimul caracter. Procedeul continuă până în momentul în care se ajunge la un caracter care nu este '9',

'z' sau 'Z' sau se ajunge la începutul șirului. În acest ultim caz, la începutul șirului se adaugă caracterul '1', 'a' sau 'A'. Incrementarea nu are nici un efect pentru caracterele care nu sunt cifre sau litere.

Decrementarea șirurilor de caractere nu are nici un efect, valorile variabilelor rămânând nemodificate. Există o singură excepție și anume șirurile care reprezintă numere întregi sau reale. Acestea sunt incrementate sau decrementate potrivit regulilor pentru valorile numerice.

Exemplu:

```

1  <?php
2  $a=7;
3  echo "<tt>a=$a";
4  echo ": </tt><i>Rezultatul operatiei </i><tt>a++</tt><i> este </i><tt>";
5  echo $a++;
6  echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
7  echo "<br>a = $a";
8  echo ": </tt><i>Rezultatul operatiei </i><tt>++a</tt><i> este </i><tt>";
9  echo ++$a;
10 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
11 echo "<br>a=$a";
12 echo ": </tt><i>Rezultatul operatiei </i><tt>a--</tt><i> este </i><tt>";
13 echo $a--;
14 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
15 echo "<br>a=$a";
16 echo ":</tt><i>Rezultatul operatiei </i><tt>--a</tt><i> este </i><tt>";
17 echo --$a;
18 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
19 $a=7.5;
20 echo "<br>a=$a";
21 echo ":</tt><i>Rezultatul operatiei </i><tt>a++</tt><i> este </i><tt>";
22 echo $a++;
23 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
24 echo "<br>a=$a";
25 echo ":</tt><i>Rezultatul operatiei </i><tt>++a</tt><i> este </i><tt>";
26 echo ++$a;
27 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
28 echo "<br>a=$a";
29 echo ":</tt><i>Rezultatul operatiei </i><tt>a--</tt><i> este </i><tt>";
30 echo $a--;
31 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
32 echo "<br>a=$a";
33 echo ":</tt><i>Rezultatul operatiei </i><tt>--a</tt><i> este </i><tt>";
34 echo --$a;
35 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i>este </i><tt>$a.";
36 $a = "PHP";
37 echo "<br>a = $a";
38 echo ":</tt><i>Rezultatul operatiei </i><tt>a++</tt><i> este </i><tt>";
39 echo $a++;
40 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
41 echo "<br>a=$a";
42 echo ":</tt><i>Rezultatul operatiei </i><tt>++a</tt><i> este</i><tt>";
43 echo ++$a;
44 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
45 echo "<br>a=$a";
46 echo ":</tt><i>Rezultatul operatiei </i><tt>a--</tt><i>este </i><tt>";
47 echo $a--;
48 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i>este </i><tt>$a.";
49 echo "<br>a=$a";
50 echo ":</tt><i>Rezultatul operatiei </i><tt>--a</tt><i>este</i><tt>";
51 echo --$a;
52 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i>este </i><tt>$a.";
53 echo "</tt>";
54 ?>

```

```

1  <?php
2  $a=7;
3  echo "<tt>a=$a";
4  echo ": </tt><i>Rezultatul operatiei </i><tt>a++</tt><i> este </i><tt>";
5  echo $a++;
6

```

```
7  echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
8  echo "<br>a = $a";
9  echo ": </tt><i>Rezultatul operatiei </i><tt>++a</tt><i> este </i><tt>";
10 echo ++$a;
11 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
12 echo "<br>a=$a";
13 echo ": </tt><i>Rezultatul operatiei </i><tt>a--</tt><i> este </i><tt>";
14 echo $a--;
15 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
16 echo "<br>a=$a";
17 echo ": </tt><i>Rezultatul operatiei </i><tt>--a</tt><i> este </i><tt>";
18 echo --$a;
19 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
20 $a=7.5;
21 echo "<br>a=$a";
22 echo ": </tt><i>Rezultatul operatiei </i><tt>a++</tt><i> este </i><tt>";
23 echo $a++;
24 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
25 echo "<br>a=$a";
26 echo ": </tt><i>Rezultatul operatiei </i><tt>++a</tt><i> este </i><tt>";
27 echo ++$a;
28 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
29 echo "<br>a=$a";
30 echo ": </tt><i>Rezultatul operatiei </i><tt>a--</tt><i> este </i><tt>";
31 echo $a--;
32 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
33 echo "<br>a=$a";
34 echo ": </tt><i>Rezultatul operatiei </i><tt>--a</tt><i> este </i><tt>";
35 echo --$a;
36 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i>este </i><tt>$a.";
37 $a = "PHP";
38 echo "<br>a = $a";
39 echo ": </tt><i>Rezultatul operatiei </i><tt>a++</tt><i> este </i><tt>";
40 echo $a++;
41 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
42 echo "<br>a=$a";
43 echo ": </tt><i>Rezultatul operatiei </i><tt>++a</tt><i> este</i><tt>";
44 echo ++$a;
45 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i> este </i><tt>$a.";
46 echo "<br>a=$a";
47 echo ": </tt><i>Rezultatul operatiei </i><tt>a--</tt><i>este </i><tt>";
48 echo $a--;
49 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i>este </i><tt>$a.";
50 echo "<br>a=$a";
51 echo ": </tt><i>Rezultatul operatiei </i><tt>--a</tt><i>este</i><tt>";
52 echo --$a;
53 echo "</tt>. <i>Noua valoare a variabilei </i><tt>a</tt><i>este </i><tt>$a.";
54 echo "</tt>";
?>
```

Rezultatul codului de mai sus este:

- a = 7: Rezultatul operatiei ++a este 7. Noua valoare a variabilei a este 8.
- a = 8: Rezultatul operatiei ++a este 9. Noua valoare a variabilei a este 9.
- a = 9: Rezultatul operatiei a-- este 9. Noua valoare a variabilei a este 8.
- a = 8: Rezultatul operatiei --a este 7. Noua valoare a variabilei a este 7.
- a = 7.5: Rezultatul operatiei ++a este 7.5. Noua valoare a variabilei a este 8.5.
- a = 8.5: Rezultatul operatiei ++a este 9.5. Noua valoare a variabilei a este 9.5.
- a = 9.5: Rezultatul operatiei a-- este 9.5. Noua valoare a variabilei a este 8.5.
- a = 8.5: Rezultatul operatiei --a este 7.5. Noua valoare a variabilei a este 7.5.
- a = PHP: Rezultatul operatiei ++a este PHP. Noua valoare a variabilei a este PHQ.
- a = PHQ: Rezultatul operatiei ++a este PHR. Noua valoare a variabilei a este PHR.
- a = PHR: Rezultatul operatiei a-- este PHR. Noua valoare a variabilei a este PHR.

a = PHR: *Rezultatul operatiei --a este PHR. Noua valoare a variabilei a este PHR.*

Operatori de execuție

Interpretorul PHP permite executarea unor comenzi sistem prin intermediul operatorului (`` ``). Comanda respectivă este cuprinsă între apostroafele inverse (tasta de lângă 1 deasupra tastei tab), iar rezultatul acestei comenzi este un șir de caractere care reprezintă și rezultatul expresiei.

Urmatorul *script* PHP determină execuția pe *server* a comenzii **help** */?* și afișarea rezultatului în fereastra programului de navigare.

```
1
2 <?php
3     echo "<PRE>".`help`/?`."</PRE>";
    ?>
```

```
1
2 <?php
3     echo "<PRE>".`help`/?`."</PRE>";
    ?>
```

Atenție!

Pe unele configurații de server nu se permite executarea de comenzi prin intermediul PHP (ca măsură de siguranță, altfel oricine ar putea da un format sau reboot la server prin intermediul unui script PHP).

Operatori pentru vectori

Singurul operator care poate fi aplicat asupra vectorilor este cel de concatenare (**+**). Rezultatul concatenării a doi vectori este un vector care conține elementele din cei doi vectori. Dacă cei doi vectori conțin elemente cu aceeași cheie, atunci este păstrat doar elementul din vectorul care reprezintă primul operând.

Exemplu:

```
1
2 <?php
3     $x[2]=2;
4     $x[4]=4;
5     $x[5]=5;
6     $x[7]=7;
7     $y[1]=100;
8     $y[2]=200;
9     $y[3]=300;
10    $y[4]=400;
11    $y[5]=500;
12    $z = $x+$y;
13    echo"$z[1]$z[2]$z[3]$z[4]$z[5]$z[7]";
    ?>
```

```
1
2 <?php
3     $x[2]=2;
4     $x[4]=4;
5     $x[5]=5;
6     $x[7]=7;
7     $y[1]=100;
8     $y[2]=200;
9     $y[3]=300;
10    $y[4]=400;
11    $y[5]=500;
12    $z = $x+$y;
13    echo"$z[1]$z[2]$z[3]$z[4]$z[5]$z[7]";
```

```
?>
```

Rezultatul codului de mai sus este:

100 2 300 4 5 7

Exerciții:

1. Se dau două numere reale **x** și **y**. Să se calculeze următoarele expresii:

$A=2+x-y$;

$B=x*A+y$;

$C=A-2*B+x$.

Structuri condiționale

Structura *if*

Una dintre cele mai importante structuri în orice limbaj este cea alternativă. În PHP sintaxa acestei structuri este următoarea:

```
1 | if (conditie) {instructiuni;}
```

```
1 | if (conditie) {instructiuni;}
```

Folosirea unei astfel de structuri indică faptul că instrucțiunile se vor executa dacă și numai dacă valoarea expresiei *conditie* (eventual după conversia la tipul boolean) va fi **true**.

Exemplu

```
1 | <?php
2 | $a = 12;
3 | $b = 8;
4 | $rezultat = $a + $b;
5 | if($rezultat == 20) {echo "Rezultatul este perfect";}
6 | ?>
```

```
1 | <?php
2 | $a = 12;
3 | $b = 8;
4 | $rezultat = $a + $b;
5 | if($rezultat == 20) {echo "Rezultatul este perfect";}
6 | ?>
```

se compară două variabile și se afișează un mesaj:

```
1 | <?php
2 | $a=4;
3 | $b=3;
4 | echo "$a=4<br>";
5 | echo "$b=3<br>";
6 | if ($a>$b) {echo "$a>$b";}
7 | ?>
```

```

1
2 <?php
3 $a=4;
4 $b=3;
5 echo "$a=4<br>";
6 echo "$b=3<br>";
7 if ($a>$b) {echo "$a>$b";}
?>

```

Construcția *else*

În foarte multe cazuri dorim să se execute alte instrucțiuni dacă valoarea expresiei condiție este **false**. În PHP putem utiliza o construcție de tipul **else** în acest scop. Sintaxa este:

```

1
2 if (conditie) {instrucțiuni #1;} else {instrucțiuni #2;}

1
2 if (conditie) {instrucțiuni #1;} else {instrucțiuni #2;}

```

Exemplu

```

1
2 <?php
3 $a = 20;
4 $b = 8;
5 $rezultat = $a + $b;
6 if($rezultat == 20) {echo "Rezultatul este perfect";}
7 else {echo "Rezultatul nu este egal cu cel din conditie";}
?>

1
2 <?php
3 $a = 20;
4 $b = 8;
5 $rezultat = $a + $b;
6 if($rezultat == 20) {echo "Rezultatul este perfect";}
7 else {echo "Rezultatul nu este egal cu cel din conditie";}
?>

```

Construcția *elseif*

Uneori, suntem nevoiți să folosim structuri alternative imbricate în diferite scopuri. Folosind construcții de tipul **if** - **else** vom obține o secvență de tipul:

```

1
2 if (conditie #1) {instrucțiuni #1;}
3 else if (conditie #2) {instrucțiuni #2;}
4 ...
5 else if (conditie #n) {instrucțiuni #n;}
6 else {instrucțiuni #n+1;}

1
2 if (conditie #1) {instrucțiuni #1;}
3 else if (conditie #2) {instrucțiuni #2;}
4 ...
5 else if (conditie #n) {instrucțiuni #n;}
6 else {instrucțiuni #n+1;}

```

Limbaajul PHP permite folosirea unei prescurtări și anume construcția ***elseif***. Practic, aceasta înlocuiește un ***else*** urmat de un ***if***. Folosind această structură, codul anterior poate fi scris astfel:

```
1
2 if (conditie #1) {instructiuni #1;}
3 elseif (conditie #2) {instructiuni #2;}
4 ...
5 elseif (conditie #n) {instructiuni #n;}
6 else {instructiuni #n+1;}
```

```
1
2 if (conditie #1) {instructiuni #1;}
3 elseif (conditie #2) {instructiuni #2;}
4 ...
5 elseif (conditie #n) {instructiuni #n;}
6 else {instructiuni #n+1;}
```

Exemplu:

```
1
2 <?php
3 $a = 20;
4 $b = 8;
5 if ($a > $b) {echo "a este mai mare ca b";}
6 elseif ($a == $b) {echo "a este egal cu b";}
7 else {echo "a este mai mic ca b";}
8 ?>
```

```
1
2 <?php
3 $a = 20;
4 $b = 8;
5 if ($a > $b) {echo "a este mai mare ca b";}
6 elseif ($a == $b) {echo "a este egal cu b";}
7 else {echo "a este mai mic ca b";}
8 ?>
```

Structura *switch*

În anumite cazuri trebuie să efectuăm anumite operații în funcție de anumite valori ale unei expresii. O soluție ar fi folosirea unor structuri ***if*** imbricate sau a uneia singure în care apar mai multe construcții ***elseif***.

Să presupunem că o anumită variabilă poate avea cinci valori distincte și pentru fiecare valoare trebuie executată o anumită instrucțiune. Dacă folosim structura ***if***, secvența de cod va fi următoarea:

```
1
2 if ($variabila == valoare #1)
3 instructiune #1
4 elseif ($variabila == valoare #2)
5 instructiune #2
6 elseif ($variabila == valoare #3)
7 instructiune #3
8 elseif ($variabila == valoare #4)
9 instructiune #4
10 else
11 instructiune #5
```

```
1
2 if ($variabila == valoare #1)
3 instructiune #1
4 elseif ($variabila == valoare #2)
```

```

5 | instructiune #2
6 | elseif ($variabila == valoare #3)
7 | instructiune #3
8 | elseif ($variabila == valoare #4)
9 | instructiune #4
10| else
    instructiune #5

```

Folosind structura **switch**, secvența echivalentă este:

```

1 |
2 | switch ($variabila) {
3 | case valoare #1: instructiune #1; break;
4 | case valoare #2: instructiune #2; break;
5 | case valoare #3: instructiune #3; break;
6 | case valoare #4: instructiune #4; break;
7 | case valoare #5: instructiune #5; break;
  | }

1 |
2 | switch ($variabila) {
3 | case valoare #1: instructiune #1; break;
4 | case valoare #2: instructiune #2; break;
5 | case valoare #3: instructiune #3; break;
6 | case valoare #4: instructiune #4; break;
7 | case valoare #5: instructiune #5; break;
  | }

```

Prezentăm în continuare sintaxa generală a structurii **switch**:

```

1 |
2 | switch (expresie) {
3 | case valoare #1: grup instructiuni #1; break;
4 | case valoare #2: grup instructiuni #2; break;
5 | ...
6 | case valoare #n: grup instructiuni #n; break;
7 | [default: grup instructiuni #n + 1;]
  | }

1 |
2 | switch (expresie) {
3 | case valoare #1: grup instructiuni #1; break;
4 | case valoare #2: grup instructiuni #2; break;
5 | ...
6 | case valoare #n: grup instructiuni #n; break;
7 | [default: grup instructiuni #n + 1;]
  | }

```

Pentru început se determină valoarea expresiei și apoi se verifică dacă aceasta apare ca valoare pentru una dintre construcțiile **case**. În caz afirmativ, toate instrucțiunile începînd cu cea corespunzătoare valorii respective (pînă la sfîrșitul blocului **switch**) vor fi executate. Execuția acestor instrucțiuni poate fi întreruptă dacă este folosită instrucțiunea **break**, care va fi prezentată mai jos. Dacă valoarea expresiei nu corespunde nici uneia dintre valorile corespunzătoare construcțiilor **case**, se execută instrucțiunile corespunzătoare construcției **default**. Dacă aceasta lipsește, atunci nu se execută nici o instrucțiune. Trebuie precizat faptul că valorile corespunzătoare construcțiilor **case** pot fi numere întregi, numere reale sau șiruri de caractere; nu pot fi utilizate obiecte sau vectori.

Exemplu: Transformă numărul zilei în numele ei

```

1 |

```

```

2  <?php
3  $ziua=2;
4  echo $ziua;
5  echo "<br>";
6  switch ($ziua) {
7  case 1: echo "Luni"; break;
8  case 2: echo "Marti"; break;
9  case 3: echo "Miercuri"; break;
10 case 4: echo "Joi"; break;
11 case 5: echo "Vineri"; break;
12 case 6: echo "Sambata"; break;
13 case 7: echo "Duminica"; break;
14 }
    ?>

```

```

1  <?php
2  $ziua=2;
3  echo $ziua;
4  echo "<br>";
5  switch ($ziua) {
6  case 1: echo "Luni"; break;
7  case 2: echo "Marti"; break;
8  case 3: echo "Miercuri"; break;
9  case 4: echo "Joi"; break;
10 case 5: echo "Vineri"; break;
11 case 6: echo "Sambata"; break;
12 case 7: echo "Duminica"; break;
13 }
    ?>

```

```

1  <?php
2  $i=2;
3  Switch($i) {
4  case 0: echo "i este egal cu 0"; break;
5  case 1: echo "i este egal cu 1"; break;
6  case 2: echo "i este egal cu 2"; break;
7  default: echo "i nu este egal cu 0,1,2";
8  }
    ?>

```

```

1  <?php
2  $i=2;
3  Switch($i) {
4  case 0: echo "i este egal cu 0"; break;
5  case 1: echo "i este egal cu 1"; break;
6  case 2: echo "i este egal cu 2"; break;
7  default: echo "i nu este egal cu 0,1,2";
8  }
    ?>

```

```

1  <?php
2  if(!isset($_GET['pag'])) $_GET['pag'] = '';
3  switch($_GET['pag']) {
4  case '': echo 'Pagina switch.php'; break;
5  case 'pagina1': echo 'Pagina switch.php?pag=pagina1'; break;
6  case 'pagina2': echo 'Pagina switch.php?pag=pagina2'; break;
7  }
    ?>

```

```

1  <?php
2  if(!isset($_GET['pag'])) $_GET['pag'] = '';
3  switch($_GET['pag']) {

```

```

5 | case '': echo 'Pagina switch.php'; break;
6 | case 'pagina1': echo 'Pagina switch.php?pag=pagina1'; break;
7 | case 'pagina2': echo 'Pagina switch.php?pag=pagina2'; break;
8 | }
  | ?>

```

Exerciții:

1. Să se afle minimum și maximum din trei numere date **a**, **b**, **c**, cu ajutorul instrucțiunii if, else.
2. Sunt date trei numere **a**, **b**, **c**. De afișat numerele în ordine crescătoare(descreșcătoare).

Structuri repetitive

Structura *while*

Majoritatea script-urilor PHP vor conține cicluri. Pentru a folosi o buclă anterior condiționată vom utiliza structura **while**. Sintaxa acestei structuri este:

```

1 | while (conditie) {instructiuni;}

1 | while (conditie) {instructiuni;}

```

Efectul acestei structuri este executarea instrucțiunii atît timp cît valoarea expresiei *conditie* (eventual după conversia la tipul *boolean*) este *true*. Trebuie observat faptul că este posibil ca instrucțiunile din acoladă să nu fie executată niciodată.

Exemplu:

```

1 | <?php
2 | $numar=1;
3 | while ($numar<=7) {
4 |     echo $numar."<br>";
5 |     $numar++;
6 | }
7 | ?>

```

```

1 | <?php
2 | $numar=1;
3 | while ($numar<=7) {
4 |     echo $numar."<br>";
5 |     $numar++;
6 | }
7 | ?>

```

```

1 | $brush_price = 5;
2 | $counter = 10;
3 | echo "<table border=\"1\" align=\"center\">";
4 |     echo "<tr><th>Quantity</th>";
5 |     echo "<th>Price</th></tr>";
6 |     while ( $counter <= 100 ) {
7 |         echo "<tr><td>";
8 |         $counter;
9 |         echo "</td><td>";
10 |         echo $brush_price * $counter;
11 |         echo "</td></tr>";
12 |

```

```

13 |     $counter = $counter + 10;
14 | }
    echo "</table>";

1 |
2 | $brush_price = 5;
3 | $counter = 10;
4 | echo "<table border=\"1\" align=\"center\">";
5 |     echo "<tr><th>Quantity</th>";
6 |     echo "<th>Price</th></tr>";
7 |     while ( $counter <= 100 ) {
8 |         echo "<tr><td>";
9 |         $counter;
10 |        echo "</td><td>";
11 |        echo $brush_price * $counter;
12 |        echo "</td></tr>";
13 |        $counter = $counter + 10;
14 |    }
    echo "</table>";

```

Structura *do - while*

Uneori, dorim să folosim bucle posterior condiționate. În acest scop, în PHP avem la dispoziție structura ***do - while*** a cărei sintaxă este:

```

1 | do {instructiuni;} while (conditie);

1 | do {instructiuni;} while (conditie);

```

Singura diferență față de structura *while* este faptul că valoarea condiției este determinată, de fiecare dată, după executarea instrucțiunii. Ca urmare, instrucțiunea va fi executată cel puțin o dată. Nu există o sintaxă alternativă pentru această structură.

```

1 |
2 | <?php
3 | $i=1;
4 | do {
5 |     echo $i."<br>";
6 |     $i++;
7 | }
8 | while ($i<=5)
   ?>

1 |
2 | <?php
3 | $i=1;
4 | do {
5 |     echo $i."<br>";
6 |     $i++;
7 | }
8 | while ($i<=5)
   ?>

```

Exercițiu:

1. Se dă șirul 1..20, utilizînd operatorul *while*. Să se afișeze numerele pare apoi și numerele impare din șirul dat.

Structura *for*

Sintaxa este foarte asemănătoare cu cea din limbajele C/C++ și Java și anume:

```
1
2 for (expresie #1; conditie; expresie #2) {
3     instructiuni;
4 }
```

```
1
2 for (expresie #1; conditie; expresie #2) {
3     instructiuni;
4 }
```

Prima expresie este evaluată o singură dată, înainte de începerea execuției ciclului. Expresia *conditie* este testată înainte fiecărei repetări a buclei. Dacă expresia returnează valoarea **false**, repetarea se oprește. *expresie #2* este executată la fiecare repetare a buclei.

instructiuni se execută la fiecare repetare a buclei.

Oricare dintre cele trei expresii poate lipsi. În cazul în care o expresie lipsește, se consideră că ea are valoarea *true*.

Buclea WHILE și FOR sunt identice din punct de vedere funcțional însă bucla FOR este puțin mai complexă.

Exemple:

```
1
2 <?php
3 for ($i=1; $i<=10; $i++) {
4     echo $i."<br>";
5 }
6 ?>
```

```
1
2 <?php
3 for ($i=1; $i<=10; $i++) {
4     echo $i."<br>";
5 }
6 ?>
```

```
1
2 <?php
3 for($i=10; $i>=1; $i--) {
4     echo $i."<br>";
5 }
6 ?>
```

```
1
2 <?php
3 for($i=10; $i>=1; $i--) {
4     echo $i."<br>";
5 }
6 ?>
```

```
1
2 <?php
3 for($a=1; $a<=10; $a++) {
4     $r=$a%2;
5     if($r==1) echo "$a - impar" else echo "$a - par";
6 }
7 ?>
```

```

1
2 <?php
3 for($a=1; $a<=10; $a++) {
4     $r=$a%2;
5     if($r==1) echo "$a - impar" else echo "$a - par";
6 }
7 ?>

```

```

1
2 <?php
3 $i = 0;
4 $s = 0;
5 do {
6     $s = $s + $i;
7     $i++;
8 } while ($i <= 3);
9 echo "Suma este $s";
10 ?>

```

```

1
2 <?php
3 $i = 0;
4 $s = 0;
5 do {
6     $s = $s + $i;
7     $i++;
8 } while ($i <= 3);
9 echo "Suma este $s";
10 ?>

```

```

1
2 <?php
3 echo"<table border=\"1\">\n<tr><td>Celula</td></tr>\n";
4 $culoare="yellow";
5 for($i=1; $i<=10; $i++){
6     if($culoare=="yellow") $culoare="red";
7     else $culoare="yellow";
8     echo "<tr><td bgcolor=\".$culoare.\">".$i."</td></tr>\n";
9 }
10 echo"</table>";
11 ?>

```

```

1
2 <?php
3 echo"<table border=\"1\">\n<tr><td>Celula</td></tr>\n";
4 $culoare="yellow";
5 for($i=1; $i<=10; $i++){
6     if($culoare=="yellow") $culoare="red";
7     else $culoare="yellow";
8     echo "<tr><td bgcolor=\".$culoare.\">".$i."</td></tr>\n";
9 }
10 echo"</table>";
11 ?>

```

```

1
2 <?php
3 for ($i=1; $i<=15; $i++):
4     echo"<b><font size =3 ";
5     echo"color=";
6     if(!($i%2))
7         echo"red";
8     elseif(!($i%3))
9         echo"green";
10    else
11        echo"blue";

```

```

12 | echo">$i</font></b>";
13 | endfor;
    | ?>

1 |
2 | <?php
3 | for ($i=1; $i<=15; $i++):
4 |     echo"<b><font size =3 ";
5 |     echo"color=";
6 |     if(!($i%2))
7 |     echo"red";
8 |     elseif(!($i%3))
9 |     echo"green";
10 |    else
11 |    echo"blue";
12 |    echo">$i</font></b>";
13 | endfor;
    | ?>

```

Exerciții:

1. Sunt date numerele **K** și **N**($N > 0$). De afișat de **N** ori numărul **K**;
2. De elaborat un script ce va afișa tabla înmulțirii;
3. De elaborat un script ce va afișa tabla de șah;
4. De elaborat un script care va calcula numărul exact de ani, luni și zile dela ziua de naștere.
5. Să se scrie un program care să afișeze următoarele valori:

$1*9 + 2 = ?$

$12*9 + 3 = ?$

$123*9 + 4 = ?$

.....

$123456789*9 + 10 = ?$

6. Să se scrie un program care să afișeze următoarea "piramidă" de numere:

1

1 2 3

1 2 3 4 5

.....

1($2n-1$)

Tipul tablou(matrice) în PHP

1.Generalitati: definitie, forme

Variabilele matrice (array, tablou), sunt acel tip de variabile, care pot sa stocheze o matrice de valori (mai multe valori). Variabilele matrice sunt variabilele care pot stoca colectii de valori, organizat ca seturi pereche, **cheie - valoare**.

Tipul acestor valori, poate fi de acelasi tip, dar poate fi si diferita. Variabilele matrice suporta orice tip de date suportate de PHP, inclusiv "tipul de date" matrice, cea ce face posibila crearea de variabile matrice multidimensionale.

În funcție de tipul, natura, cheii folosite, sunt două forme de variabile matrice:

indexate - cheile acestor variabile matrice sunt numere intregi, incepand cu 0

asociative - cheile acestor variabile matrice sunt siruri de caractere

In realitate, PHP trateaza toate variabilele matrice, ca variabile matrice asociative, cu diferenta ca cheile matricii

indexate, sunt numere intregi. In ambele cazuri, cheile sunt unice, prin urmare nu putem sa avem doua elemente cu aceasi cheie. Reciproca nu este valabila, in sensul ca doua elemente cu chei diferite pot sa aiba aceasi valoare. Variabilele matrice in PHP, au o ordine interna a elementelor lor, care este independenta de chei sau de valori. Ordinea normala este data de ordinea in care s-au initializat elementele, dar functiile de sortare despre care vom vorbi mai tarziu, dau posibilitatea schimbarii ordinii interne in functie de chei, de valori, sau oricum doriti.

2.Initializarea variabilelor matrice

Initializarea variabilelor matrice, se poate realiza simplu, initializand fiecare element separat, ca o variabila simpla, sau folosind unele functi interne ale PHP, dedicate pentru acest lucru, cum este **array ()**.

2.1.Initializarea separata a variabilelor matrice

Pentru initializarea variabilelor matrice, cel mai la indemana, este initializarea fiecarui element separat:

Initializarea variabilelor matrice - fiecare element separat

Matrice indexata	Matrice asociativa
<pre>\$a[0]="Vasile"; \$a[1]="Ion"; \$a[2]="Gheorghe";</pre>	<pre>\$a["Popescu"]="Vasile"; \$a["Dascalu"]="Ion"; \$a["Popa"]="Gheorghe";</pre>

Exemplu:

```
1  <?php
2  /* variabile matrice - initializare separata */
3  $a1[0]="Vasile";
4  $a1[1]="Ion";
5  $a1[2]="Gheorghe";
6  echo "<h3>Initializarea separata a variabilelor matrice</h3>";
7  for ($ind=0; $ind<3; $ind++)
8  { echo $a1[$ind]."<br />";}
9  print_r ($a1);
10 echo "<hr />";
11 $a2["Popescu"]="Vasile";
12 $a2["Dascalu"]="Ion";
13 $a2["Popa"]="Gheorghe";
14 echo $a2["Popescu"]."<br />";
15 echo $a2["Dascalu"]."<br />";
16 echo $a2["Popa"]."<br />";
17 print_r ($a2);
18 ?>
```



```
1  <?php
2  /* variabile matrice - initializare separata */
3  $a1[0]="Vasile";
4  $a1[1]="Ion";
5  $a1[2]="Gheorghe";
6  echo "<h3>Initializarea separata a variabilelor matrice</h3>";
7  for ($ind=0; $ind<3; $ind++)
8  { echo $a1[$ind]."<br />";}
9  print_r ($a1);
10
```

```
11     echo "<hr />";
12     $a2["Popescu"]="Vasile";
13     $a2["Dascalu"]="Ion";
14     $a2["Popa"]="Gheorghe";
15     echo $a2["Popescu"]."<br />";
16     echo $a2["Dascalu"]."<br />";
17     echo $a2["Popa"]."<br />";
18     print_r ($a2);
    ?>
```

2.2. Initializarea variabilelor matrice cu functia array

Variabilele de tip tablou se mai pot initializa si prin constructia sau declaratia **array()**, care este, cel putin in cazul matricilor unidimensionale, mai usoara:

Initializarea variabilelor matrice cu constructia **array()**

Matrice indexata	Matrice asociativa
<pre>\$a=array("Vasile", "Ion", "Gheorghe");</pre>	<pre>\$a=array("Popescu"=> "Vasile", "Dascalu"=> "Ion", "Popa"=> "Gheorghe");</pre>

Valorile (in cazul variabilelor cu index), sau perechile cheie - valoare (in cazul tablourilor asociative) sunt separate prin virgula.

Exemplu:

```
1
2 <?php
3 /* variabile matrice - initializare cu array */
4 echo "<h3>Initializarea cu array a variabilelor matrice</h3>";
5 $a1=array ("Vasile", "Ion", "Gheorghe");
6 for ($ind=0; $ind<3; $ind++)
7 { echo $a1[$ind]."<br />";
8   print_r ($a1);
9   echo "<hr />";
10 $a2=array
11 ("Popescu" => "Vasile",
12 "Dascalu"=> "Ion",
13 "Popa" => "Gheorghe");
14 echo $a2["Popescu"]."<br />";
15 echo $a2["Dascalu"]."<br />";
16 echo $a2["Popa"]."<br />";
17 print_r ($a2);
18 echo "<hr />";
    ?>
```

```
1
2 <?php
3 /* variabile matrice - initializare cu array */
4 echo "<h3>Initializarea cu array a variabilelor matrice</h3>";
5 $a1=array ("Vasile", "Ion", "Gheorghe");
6 for ($ind=0; $ind<3; $ind++)
7 { echo $a1[$ind]."<br />";
8   print_r ($a1);
9   echo "<hr />";
10 $a2=array
11 ("Popescu" => "Vasile",
12 "Dascalu"=> "Ion",
13 "Popa" => "Gheorghe");
14 echo $a2["Popescu"]."<br />";
15 echo $a2["Dascalu"]."<br />";
```

```

15
16     echo $a2["Popa"]."<br />";
17     print_r ($a2);
18     echo "<hr />";
    ?>

```

Cu ajutorul constructiei **array ()**, variabilele matrice se mai pot initializa si dupa cum urmeaza:

se defineste o variabila matrice fara valori: **\$var=array()**;

se specifica o cheie de pornire, cu **=>**, si apoi o lista de valori; valorile sunt inserate in matrice, incepand cu cheia specificata

In cazul tablourilor indexate, urmatoarele chei vor fi in ordine crescatoare fata de cheia initiala

In cazul tablourilor a caror cheie de pornire este un sir, cheile urmatoare vor fi numere intregi, prima valoare fiind 0.

Exemplu:

```

1
2 <?php
3 /* variabile matrice - initializare cu array */
4 echo "<h3>Initializarea cu array a variabilelor matrice</h3>";
5 $a1=array();
6 $a1=array (1=>"Vasile", "Ion", "Gheorghe");
7     for ($ind=1; $ind<4; $ind++)
8     {echo $a1[$ind]."<br />";}
9     print_r($a1);
10    echo "<hr />";
11 $a2=array ("Popescu" => "Vasile", "Ion", "Gheorghe");
12 $key=array ("Popescu", "0", "1");
13     echo $a2["Popescu"]."<br />";
14     echo $a2["0"]."<br />";
15     echo $a2["1"]."<br />";
16     print_r($a2);
17     echo "<br />";
18     for ($ind=0; $ind<3; $ind++)
19     {echo $key[$ind]. " => ". $a2[$key[$ind]]."<br />";}
20    echo "<hr />";
    ?>

```

```

1
2 <?php
3 /* variabile matrice - initializare cu array */
4 echo "<h3>Initializarea cu array a variabilelor matrice</h3>";
5 $a1=array();
6 $a1=array (1=>"Vasile", "Ion", "Gheorghe");
7     for ($ind=1; $ind<4; $ind++)
8     {echo $a1[$ind]."<br />";}
9     print_r($a1);
10    echo "<hr />";
11 $a2=array ("Popescu" => "Vasile", "Ion", "Gheorghe");
12 $key=array ("Popescu", "0", "1");
13     echo $a2["Popescu"]."<br />";
14     echo $a2["0"]."<br />";
15     echo $a2["1"]."<br />";
16     print_r($a2);
17     echo "<br />";
18     for ($ind=0; $ind<3; $ind++)
19     {echo $key[$ind]. " => ". $a2[$key[$ind]]."<br />";}
20    echo "<hr />";
    ?>

```

2.3. Adaugarea de valori suplimentare, ulterior initializarii

Daca cunoastem indicele ultimei valori a variabilei, putem adauga noi elemente prin indicarea indexului:

```
$var[4] = "Nat";
```

Totusi, nu intodeauna cunostem cate elemente are o variabila, sau care este cheia ultimului element. PHP va furnizeaza o cale simpla, pentru a adauga variabilei un nou element:

```
$var[] = "val_sup";
```

Exemplu:

```
1  <?php
2  /* variabile matrice - elemente suplimentare */
3  echo "<h3>Setarea cu noi valori a variabilelor matrice</h3>";
4  $a=array ("Vasile", "Ion", "Gheorghe");
5  print_r ($a);
6  echo "<hr />";#####
7  $a [3]="Dinu";
8  print_r ($a);
9  echo "<hr />";#####
10 $a []="Stefan";
11 $a []="Petrache";
12 print_r($a);
13 echo "<hr />";
14 ?>
```

```
1  <?php
2  /* variabile matrice - elemente suplimentare */
3  echo "<h3>Setarea cu noi valori a variabilelor matrice</h3>";
4  $a=array ("Vasile", "Ion", "Gheorghe");
5  print_r ($a);
6  echo "<hr />";#####
7  $a [3]="Dinu";
8  print_r ($a);
9  echo "<hr />";#####
10 $a []="Stefan";
11 $a []="Petrache";
12 print_r($a);
13 echo "<hr />";
14 ?>
```

3.Identificarea elementelor unei variabile matrice

Pentru a accesa valoarea unui anumit element dintr-o variabila matrice, folosim numele variabilei, urmat de cheia elementului intre paranteze drepte:

```
$nume_var[numar]
```

```
$nume_var["sir_cheie"]
```

Cheia, **sir_cheie** poate sa fie, un sir sau un numar intreg. Cand este un numar intreg, este tratat ca atare. Astfel, **\$var[3]** si **\$var["3"]** sunt acelasi element, dar **\$var["03"]** este alt element. Numerele negative sunt chei valide, si ele nu specifica pozitia la sfarsitul tabloului ca in Perl.

Exemplu:

```
1  <?php
2  /* variabile matrice - identificare */
3  $a=array ("Vasile", "Ion", "Gheorghe");
4  echo $a[1]."<br />";
5  $a=array
6  (
7      "0"  => "Vasile",
8      "-1" => "Ion",
9      "02" => "Gheorghe");
10 echo $a["-1"]."<br />";
11 print_r($a);
```

```

1  <?php
2  /* variabile matrice - identificare */
3  $a=array ("Vasile", "Ion", "Gheorghe");
4  echo $a[1]."<br />";
5  $a=array
6  (
7      "0"    => "Vasile",
8      "-1"   => "Ion",
9      "02"   => "Gheorghe");
10 echo $a["-1"]."<br />";
11 print_r($a);
12 ?>

```

Sirurile dintr-un singur cuvânt, nu este obligatoriu sa fie puse intre ghilimele. De exemplu, **\$var[Popescu]** este la fel ca **\$var["Popescu"]**. Totusi, se considera ca o buna practica PHP, sa folositi intodeauna ghilimelele, deoarece cheile fara ghilimele, nu se pot deosebi de constantele introduse ca index:

Exemplu:

```

1  <?php
2  /* variabile matrice - identificare */
3  define ("IDX", 1); //definim o constanta
4  $a=array
5  (
6      "IDX"    => "Vasile",
7      IDX      => "Ion", "Gheorghe");
8  echo $a[IDX]."<br />";
9  echo $a["IDX"]."<br />";
10 print_r($a);
11 ?>

```

```

1  <?php
2  /* variabile matrice - identificare */
3  define ("IDX", 1); //definim o constanta
4  $a=array
5  (
6      "IDX"    => "Vasile",
7      IDX      => "Ion", "Gheorghe");
8  echo $a[IDX]."<br />";
9  echo $a["IDX"]."<br />";
10 print_r($a);
11 ?>

```

Cand aveti nevoie sa recuperati elemente prin indexul matricii (matrice asociativa cu index), trebuie sa folositi ghilimelele duble, pentru ca PHP sa recunoasca variabila indice.

Exemplu:

```

1  <?php
2  /* variabile matrice - identificare */
3  $a['ind1']="Vasile";
4  $a['ind2']="Ion";
5  $a['ind3']="Gheorghe";
6  for ($i=1; $i<=3; $i++)
7  {
8      echo $a["ind$i"]."<br />";
9  }
10 print_r($a);
11 ?>

```

```

1  <?php
2  /* variabile matrice - identificare */
3  $a['ind1']="Vasile";

```

```

5  $a['ind2']="Ion";
6  $a['ind3']="Gheorghe";
7  for ($i=1; $i<=3; $i++)
8  {echo $a["ind$i"]."<br />";}
9  print_r($a);
  ?>

```

4. Variabilele matrice multi-dimensionale

Vom vorbi in particular despre matricile (tablourile) bidimensionale, deoarece sunt mai usor de vizualizat, atat vizual, cat si mental. Vom lua un tabel, valorile sale le vom introduce intr-o variabila matrice, ca sa avem o asociere buna.

Tabel exemplu

-	Nume	Prenume	An nastere
Rand/Col	0	1	2
0	Popescu	Vasile	1964
1	Dascalu	Ion	1982
2	Popa	Gheorghe	1955

Lasand la o parte capul de tabel, precum si randul, respectiv coloana, pentru numerotarea randurilor si coloanelor, avem un tabel cu trei randuri si trei coloane. Valorile acestui tabel se pot stoca intr-o variabila matrice bidimensionala in trei modalitati:

prin initializarea separata a fiecarui element

```
$a[0][0]="Popescu";
```

```
$a[0][1]="Vasile";
```

```
$a[0][2]="1964";
```

```
$a[1][0]="Dascalu";
```

```
$a[1][1]="Ion";
```

etc.

prin initializarea mixta, fiecare rand cu constructia **array**

```
$a[0]=array("Popescu", "Vasile", "1964");
```

```
$a[1]=array("Dascalu", "Ion", "1982");
```

```
$a[2]=array("Popa", "Gheorghe", "1955");
```

doar cu constructia **array**

```
$a=array(array("Popescu", "Vasile", "1964"), array("Dascalu", "Ion", "1982"), array("Popa", "Gheorghe", "1955"));
```

Observam ca matricile bidimensionale, au evident doi indici. Prin extrapolare, putem spune ca o matrice **n** dimensionala trebuie sa aiba **n** indici. Nu am vazut nicaieri vreo restrictie privind dimensiunea unei variabile matrice, eu am folosit in acest curs matrice cu 4 dimensiuni, dar nu este bine sa exagerati.

Recuperarea valorilor, elementelor unei matrici bidimensionale, este identic procedural ca a matricilor unidimensionale, adica prin indicarea cheilor variabilei, deci trebuie sa cunoastem cheile elementului a carei valoare vrem s-o recuperam:

\$nume_var[cheie_1][cheie_2]

Deoarece PHP trateaza cele doua tipuri de variabila matrice, ca variabile asociative, putem sa spunem ca si variabilele asociative se initializeaza si recupereaza similar.

Exemplu:

```
1
2  <?php
3  /* variabile matrice bidimensionale */
4  $a[0]=array("Nume"    =>"Popescu",
5              "Prenume"  =>"Vasile",
6              "An_nastere"=>"1964");
7  $a[1]=array("Nume"    =>"Dascalu",
8              "Prenume"  =>"Ion",
9              "An_nastere"=>"1982");
10 $a[2]=array("Nume"    =>"Popa",
11             "Prenume"  =>"Gheorghe",
12             "An_nastere"=>"1955");
13 for ($i=0; $i<=2; $i++)
14 {echo $a["$i"]["Nume"]." - ".
15      $a["$i"]["Prenume"]." - ".
16      $a["$i"]["An_nastere"]."<br />";}
17 ?>
```

```
1
2  <?php
3  /* variabile matrice bidimensionale */
4  $a[0]=array("Nume"    =>"Popescu",
5              "Prenume"  =>"Vasile",
6              "An_nastere"=>"1964");
7  $a[1]=array("Nume"    =>"Dascalu",
8              "Prenume"  =>"Ion",
9              "An_nastere"=>"1982");
10 $a[2]=array("Nume"    =>"Popa",
11             "Prenume"  =>"Gheorghe",
12             "An_nastere"=>"1955");
13 for ($i=0; $i<=2; $i++)
14 {echo $a["$i"]["Nume"]." - ".
15      $a["$i"]["Prenume"]." - ".
16      $a["$i"]["An_nastere"]."<br />";}
17 ?>
```

5.Functiile specifice variabilelor matrice

Limbajul PHP chiar daca are un numar restrins de instructiuni, are in schimb un numar impresionant de funtii interne, care dau puterea acestui limbaj de programare.

Cele mai importante domenii privind variabilele matrice sunt legate de initializarea (crearea) lor, precum si identificarea elementelor sale. Alte domenii cel putin la fel de importante sunt modificarea si efectuarea de operatii cu elementele variabilelor matrice. Un alt domeniu interesant, este domeniul verificarilor. Functiile interne furnizate de PHP, incearca sa acopere aceste domenii, unele din ele, interferand chiar doua domenii.

5.1.Functii pentru crearea variabilelor matrice

Functiile doar pentru crearea variabilelor matrice sunt:

Functii pentru crearea de noi variabile matrice

Funcția	Descriere

array()	Crează o variabilă matrice.
range()	Întoarce o variabilă matrice, a carui elemente au valorile ordonate de la o valoare minima la una maxima, cu diferenta intre valorile elementelor consecutive egala cu un pas - ordonarea poate fi si descrescatoare.

5.1.1.functia **array()**

Aceasta functie am vazut-o in actiune, asa ca vom vorbi foarte pe scurt despre ea. In primul rand este principala metoda pentru crearea variabilelor matrice.

Sintaxa:

\$nume_var = array ("cheie_1" => "val_1", "cheie_2" => "val_2", ...);

Aceasta este sintaxa generala. Dupa cum am vazut sunt cazuri particulare.

cheie_1, cheie_2 - sunt valorile cheilor viitoarei variabile matrice - sunt optionale

val_1, val_2 - sunt valorile elementelor viitoarei variabile matrice, corespunzator cheii - si ele sunt optionale, daca sunt omise si ele, se defineste o variabila matrice fara valori

=> - defineste corespondenta cheie - valoare - optional, in cazul in care cheile nu sunt definite, chiar trebuie sa lipseasca.

Cheile **cheie_1, cheie_2** pot fi, fie numere intregi, fie sir de caractere. Cand este omisa atunci functia genereaza automat chei numerice incepand cu 0, cu rata de crestere 1.

Descriere si functionare

Aceasta functie creaza (intoarce) o variabila matrice, cu perechi chei valori, conform argumentelor setate.

5.1.2. functia **range()**

Sintaxa:

\$nume_var = range (val_min, val_max, pas);

Argumentele functiei *range()*

Argument	Descriere
val_min	Necesar. Specifica valoarea minima a valorilor elementelor variabilei matrice, care in general se alocă elementului cu indicele 0.
val_max	Necesar. Specifica valoarea maxima (cea mai mare) a valorilor elementelor variabilei matrice, care in general se alocă elementului cu indicele cel mai mare.
pas	Optional. Specifica pasul cu care cresc valorile elementelor variabilei matrice. Predefinit (deci daca nu setam acest argument) este 1.

Descriere si functionare

Aceasta functie creaza o variabila matrice, care contine o clasa de elemente, ale carei valori sunt de la **val_min** pana la inclusiv **val_max**, cu pasul intre valori determinat de **pas**. Altfel spus, intoarce o variabila matrice, a carui elemente au valorile ordonate de la **val_min** la **val_max**, cu diferenta intre valorile elementelor consecutive egala cu **pas**.

Nota: Daca parametrul **val_min** este mai mare decat **val_max** atunci ordonarea se va face de la mai mare spre mai mic.

Exemplu:

```
1
2 <?php
3 /* functia range */
4 $a = range(1,4);
5 print_r ($a);
6 echo "<hr />"; #####
7 $a = range(1,12,3);
8 print_r ($a);
9 echo "<hr />"; #####
10 $a = range("e", "s",2);
11 print_r ($a);
12 echo "<hr />";
13 ?>
```



```
1
2 <?php
3 /* functia range */
4 $a = range(1,4);
5 print_r ($a);
6 echo "<hr />"; #####
7 $a = range(1,12,3);
8 print_r ($a);
9 echo "<hr />"; #####
10 $a = range("e", "s",2);
11 print_r ($a);
12 echo "<hr />";
13 ?>
```

5.2.Functii pentru crearea variabilelor matrice prin procesarea altor variabile

Pentru acest caz, PHP furnizeaza mai multe functii, dintre care am selectat:

Functii pentru crearea de noi variabile matrice

Functia	Descriere
array_pad()	Intoarce o variabila matrice care are inserate un numar specificat de elemente, cu valoare definita, pe langa elementele matricii care se prelucreaza.
array_slice()	Intoarce o variabila matrice, care are ca elemente, unele din elementele selectate ale variabilei matrice argument, fara a o afecta pe aceasta.
array_chunk()	Intoarce o variabila matrice bidimensionala, prin impartirea variabilei matrice argument.
array_keys()	Intoarce o variabila matrice, care are ca elemente, cheile altei variabile matrice.
array_values()	Intoarce o variabila matrice, insa indexata, care are ca elemente, valorile altei variabile matrice, asociative.
explode()	Intoarce o variabila matrice, dintr-un sir de caractere, prin impartirea sirului de caractere, in functie de un separator.

compact()	Intoarce o variabila de matrice, cu valorile unor variabilelor simple.
array_reverse()	Intoarce o variabila matrice, cu valorile elementelor in ordine inversata fata de variabila matrice argument.
array_flip()	Intoarce o variabila matrice, care are ca chei, valorile variabilei matrice argument, iar ca valori, cheile acestei variabile.
array_keys()	Intoarce o variabila matrice care contine valorile tuturor variabilelor matrice care sunt argumentele functiei.
array_chunk()	Aceasta functie compara doua sau mai multe variabile matrice, si intoarce o alta variabila matrice, cu elementele primei variabile (cheile si valorile sale), din care se extrag elementele a caror valoare se gaseste in una din variabilele urmatoare.
array_unique()	Aceasta functie intoarce o variabila matrice, care are valorile elementelor matricei argument, dintre care se inlatura valorile duplicate. Daca doua sau mai multe valori sunt la fel, prima aparitie va fi pastrata, iar celelalte vor fi inlaturate.
array_intersect()	Aceasta functie compara doua sau mai multe variabile matrici, si intoarce o matrice cu cheile si valorile primei matrice, dar doar daca valorile sunt si in celelalte matrice. Daca mai multe chei au aceeasi valoare, prima cheie va fi pastrata.
array_filter()	Aceasta functie trece fiecare valoare a variabilei, unei functi utilizator, de tip comparator, care intoarce fie TRUE fie FALSE. Functia intoarce o variabila matrice doar cu valorile care au intors TRUE.

5.2.1. functia *array_pad()*

Sintaxa

```
$nume_var = array_pad ($array, marime, valoare);
```

Argumentele functiei *array_pad()*

Argument	Descriere
\$array	Necesar. Specifica matricea care se prelucreaza.
marime	Necesar. Specifica numarul de elemente a noii variabile matrice intoarsa de functie.
valoare	Necesar. Specifica valoarea noilor elemente din matrice intoarse de functie.

Descriere si functionare

Aceasta functie, intoarce o variabila matrice, care are ca elemente, elementele variabilei matrice argument, pe langa care sunt inserate un numar specificat de elemente (numarul de elemente este determinat de argumentul **marime**), cu o valoare specificata, (determinata de argumentul **valoare**).

Nota:

nu taie din elementele matricii, cand parametrul **marime**, este mai mic decat numarul original de elemente. daca setam **marime** cu un numar negativ, functia va insera elementele noi in fata elementelor originale.

Exemplu

```

1  <?php
2  /* functia array_pad() */
3  $a=array("Nume"=>"Popescu",
4          "Prenume"=>"Vasile",
5          "An_nastere"=>"1964");
6          echo "variabila care se proceseaza este:<br />";
7          print_r ($a);
8          echo "<hr />";
9
10 $pad= array_pad($a,5,"element nou" );
11 echo "noua variabila este:<br />";
12 print_r ($pad);
13 echo "<hr />";
14
15 echo "noua variabila este:<br />";
16 print_r (array_pad($a,1,"element nou"));
17 echo "<hr />";
18
19 echo "noua variabila este:<br />";
20 print_r (array_pad($a,-5,"element nou"));
21 echo "<hr />";
22
23 echo "variabila care s-a procesat a ramas:<br />";
24 print_r ($a);
25 ?>

```

```

1  <?php
2  /* functia array_pad() */
3  $a=array("Nume"=>"Popescu",
4          "Prenume"=>"Vasile",
5          "An_nastere"=>"1964");
6          echo "variabila care se proceseaza este:<br />";
7          print_r ($a);
8          echo "<hr />";
9
10 $pad= array_pad($a,5,"element nou" );
11 echo "noua variabila este:<br />";
12 print_r ($pad);
13 echo "<hr />";
14
15 echo "noua variabila este:<br />";
16 print_r (array_pad($a,1,"element nou"));
17 echo "<hr />";
18
19 echo "noua variabila este:<br />";
20 print_r (array_pad($a,-5,"element nou"));
21 echo "<hr />";
22
23 echo "variabila care s-a procesat a ramas:<br />";
24 print_r ($a);
25 ?>

```

5.2.2. functia array_slice()**Sintaxa:**

\$nume_var = array_slice (\$array, start, lungime, pastrare);

Argumentele functiei `array_slice ()`

Argument	Descriere
<code>\$array</code>	Necesar. Specifica variabila matrice care se prelucreaza.
<code>start</code>	Necesar. Este o valoare numerica intreaga. Specifica de la al catelea element al variabilei matrice argument (\$array) incepe functia sa preia elemente. 0 - este incepand cu primul element. Daca acest parametru este setat cu un numar negativ, functia va incepe numararea elementelor, de la ultimul element. (-2 inseamna ca va incepe cu penultimul element)
<code>lungime</code>	Optional. Este o valoare numerica intreaga. Specifica numarul de elemente a variabilei matricei intoarse. Daca acest parametru este setat cu un numar negativ, functia nu va mai numara elementele, ci va lua ca ultim element intors, elementul anterior elementului cu numarul setat, dar numarand de la ultimul element (vezi exemplul 5.13). Predefinit functia intoarce toate elementele, incepand de la pozitia setata prin argumentul start .
<code>pastrare</code>	Optional. Valori posibile: true - pastreaza cheile variabilelor matrice indexate conform variabilei argument (\$array) false - predefinit, reface cheile variabilelor matrice indexate, incepand cu 0.

Descriere si functionare

Aceasta functie creaza o variabila matrice, taiand o parte dintr-o alta variabila matrice. Cu alte cuvinte, seteaza cheile si valorile unei noi variabile cu o parte din cheile si valorile corespunzatoare ale matricii argument. Partea cu care este setata noua variabila este determinata de argumentele functiei.

cheia de la care se incepe "taierea" este data de argumentul **start** - de la al **start**-lea element cate elemente se transfera, este data de argumentul **lungime**

Nota:

Predefinit, variabilele matrice asociative, raman variabile matrice asociative.

Exemplu

```
1  <?php
2  /* functia array_slice() */
3
4  $a=array("Popescu", "Vasile", "1964", "Chisinau");
5  $var=array_slice ($a, 2);
6  print_r ($a);           // $a nu se modifica
7  echo "<hr />";          #####
8  echo "1<br />";print_r ($var);  # 1
9  echo "<hr />";          #####
10 $var=array_slice ($a, -3);      # 2
11 echo "2<br />";print_r ($var);
12 echo "<hr />";          #####
```

```
13 $var=array_slice ($a, 2, 2);          # 3
14 echo "3<br />";print_r ($var);
15 echo "<hr />";                          #####
16 $var=array_slice ($a, 2, 2, true); # 4
17 echo "4<br />";print_r ($var);
18 echo "<hr />";                          #####
19 $var=array_slice ($a, 0, 2);          # 5
20 echo "5<br />";print_r ($var);
21 echo "<hr />";                          #####
22 $var=array_slice ($a, -3, 2);        # 6
23 echo "6<br />";print_r ($var);
24 echo "<hr />";                          #####
  ?>

1
2 <?php
3 /* functia array_slice() */
4 $a=array("Popescu", "Vasile", "1964", "Chisinau");
5 $var=array_slice ($a, 2);
6 print_r ($a); // $a nu se modifica
7 echo "<hr />"; #####
8 echo "1<br />";print_r ($var); # 1
9 echo "<hr />"; #####
10 $var=array_slice ($a, -3); # 2
11 echo "2<br />";print_r ($var);
12 echo "<hr />"; #####
13 $var=array_slice ($a, 2, 2); # 3
14 echo "3<br />";print_r ($var);
15 echo "<hr />"; #####
16 $var=array_slice ($a, 2, 2, true); # 4
17 echo "4<br />";print_r ($var);
18 echo "<hr />"; #####
19 $var=array_slice ($a, 0, 2); # 5
20 echo "5<br />";print_r ($var);
21 echo "<hr />"; #####
22 $var=array_slice ($a, -3, 2); # 6
23 echo "6<br />";print_r ($var);
24 echo "<hr />"; #####
  ?>
```

Exemplu si cu functionarea valorii negative a argumentului **lungime**:

```
1
2 <?php
3 /* functia array_slice () */
4 $a=array("Popescu", "Vasile", "1964", "Chisinau");
5 $var=array_slice ($a, 1, -2); # 1
6 echo "1<br />";print_r ($var);
7 echo "<hr />"; #####
8 $var=array_slice ($a, 1, -1); # 2
9 echo "2<br />";print_r ($var);
10 echo "<hr />"; #####
11 $var=array_slice ($a, 0, -2); # 3
12 echo "3<br />";print_r ($var);
13 echo "<hr />"; #####
  ?>

1
2 <?php
3 /* functia array_slice () */
4 $a=array("Popescu", "Vasile", "1964", "Chisinau");
5 $var=array_slice ($a, 1, -2); # 1
6 echo "1<br />";print_r ($var);
7 echo "<hr />"; #####
8 $var=array_slice ($a, 1, -1); # 2
9 echo "2<br />";print_r ($var);
10 echo "<hr />"; #####
11 $var=array_slice ($a, 0, -2); # 3
12 echo "3<br />";print_r ($var);
13 echo "<hr />"; #####
```

```
?>
```

Exemplu cu variabile matrice asociative:

```
1  <?php
2  /* functia array_slice () */
3  $a=array("Nume"      => "Dascalu",
4          "Prenume"   => "Ion",
5          "An_nastere" => "1982",
6          "Localitatea" => "Ialoveni");
7
8  $var=array_slice ($a, 1);
9  print_r ($a);
10 echo "<hr />"; #####
11 $i=0;$i++;echo "$i<br />";
12 print_r ($var);
13 echo "<hr />"; #####
14 print_r (array_slice ($a, 1));
15 echo "<hr />"; #####
16 $i++;echo "$i<br />";
17 print_r (array_slice ($a, 0, 2));
18 echo "<hr />"; #####
19 $i++;echo "$i<br />";
20 print_r (array_slice ($a, -2));
21 echo "<hr />"; #####
22 $i++;echo "$i<br />";
23 print_r (array_slice ($a, 0, -2));
24 echo "<hr />"; #####
25 $i++;echo "$i<br />";
26 print_r (array_slice ($a,0,2,true));
27 echo "<hr />"; #####
28 ?>
```

```
1  <?php
2  /* functia array_slice () */
3  $a=array("Nume"      => "Dascalu",
4          "Prenume"   => "Ion",
5          "An_nastere" => "1982",
6          "Localitatea" => "Ialoveni");
7
8  $var=array_slice ($a, 1);
9  print_r ($a);
10 echo "<hr />"; #####
11 $i=0;$i++;echo "$i<br />";
12 print_r ($var);
13 echo "<hr />"; #####
14 print_r (array_slice ($a, 1));
15 echo "<hr />"; #####
16 $i++;echo "$i<br />";
17 print_r (array_slice ($a, 0, 2));
18 echo "<hr />"; #####
19 $i++;echo "$i<br />";
20 print_r (array_slice ($a, -2));
21 echo "<hr />"; #####
22 $i++;echo "$i<br />";
23 print_r (array_slice ($a, 0, -2));
24 echo "<hr />"; #####
25 $i++;echo "$i<br />";
26 print_r (array_slice ($a,0,2,true));
27 echo "<hr />"; #####
28 ?>
```

5.2.3. functia array_chunk()

Sintaxa

\$nume_var = array_chunk (\$array, marime, pastrare);

Argumentele functiei `array_chunk ()`

Argument	Descriere
\$array	Necesar. Specifica variabila matrice care se prelucreaza.
marime	Necesar. Specifica cate elemente va contine primul element al noi variabile matrice bidimensionale.
pastrare	Optional. Valori posibile: true - pastreaza cheile variabilelor matrice indexate conform variabilei argument false - predefinit, reface cheile variabilelor matrice indexate, incepand cu 0.

Descriere si functionare

Aceasta functie, intoarce o variabila matrice bidimensionala, din elementele altei variabile matrice. Noua variabila matrice va avea primul element cu un numar de **marime** elemente, restul elementelor variabilei matrice argument, vor face parte din al doilea element a variabilei.

Daca argumentul **marime**, are valoarea mai mare decat numarul de elemente a variabilei originale, atunci toate elementele acestei variabile, vor fi alocate primului element a noi variabile.

Exemplu

```
1  <?php
2  /* functia array_chunk () */
3  $a=array("Popescu", "Vasile", "1964", "Chisinau");
4  $var=array_chunk ($a, 2);
5  print_r ($a);          # $a nu se modifica
6  echo "<hr />";          #####
7  echo "1<br />";print_r ($var);  # 1
8  echo "<hr />";          #####
9  $var=array_chunk ($a, 3);      # 2
10 echo "2<br />";print_r ($var);
11 echo "<hr />";          #####
12 $var=array_chunk ($a, 5);      # 3
13 echo "3<br />";print_r ($var);
14 echo "<hr />";          #####
15 $var=array_chunk ($a, 2, true); # 4
16 echo "4<br />";print_r ($var);
17 echo "<hr />";
18 ?>
```



```
1  <?php
2  /* functia array_chunk () */
3  $a=array("Popescu", "Vasile", "1964", "Chisinau");
4  $var=array_chunk ($a, 2);
5  print_r ($a);          # $a nu se modifica
6  echo "<hr />";          #####
7  echo "1<br />";print_r ($var);  # 1
8  echo "<hr />";          #####
9  $var=array_chunk ($a, 3);      # 2
10 echo "2<br />";print_r ($var);
11 echo "<hr />";          #####
12 $var=array_chunk ($a, 5);      # 3
13 echo "3<br />";print_r ($var);
14 echo "<hr />";          #####
15 $var=array_chunk ($a, 2, true); # 4
16 echo "4<br />";print_r ($var);
17 echo "<hr />";
18
```

>

5.2.4. functia `array_keys()`

Sintaxa

`$nume_var = array_keys ($array, val_key, strict);`

Argumentele functiei `array_keys()`

Argument	Descriere
\$array	Necesar. Specifica variabila matrice argument, cea a carei chei le va prelua noua variabila, intoarsa de functie.
valoare	Optional. Puteti specifica o valoare a matricii origine, astfel va fi intoarsa doar cheia cu aceasta valoare.
strict	Optional. Folosit impreuna cu parametrul valoare. Valori posibile: true - Intoarce cheia cu valoarea specificata, in functie de tip: numarul 5 nu este la fel cu sirul "5". false - Valoarea predefinita. Nu depinde de tip, numarul 5 este la fel ca sirul "5".

Descriere si functionare

Aceasta functie, intoarce o variabila matrice, ale carei elemente vor fi cheile variabilei matrice argument `$array`.

Exemplu

```
1  <?php
2  /* functia array_keys () */
3
4  $a=array("Nume"      => "Popescu",
5          "Prenume"   => "Vasile",
6          "An_nastere" => "1964",
7          "Localitatea" => "Chisinau");
8
9  $var=array_keys ($a);
10 print_r ($a);           # $a nu se modifica
11 echo "<hr />";          #####
12 echo "1<br />";print_r ($var);    # 1
13 echo "<hr />";          #####
14 $var=array_keys ($a, 1964);      # 2
15 echo "2<br />";print_r ($var);
16 echo "<hr />";          #####
17 $var=array_keys ($a, 1964, true); # 3
18 echo "3<br />";print_r ($var);
19 echo "<hr />";          #####
20 $var=array_keys ($a, "1964", true); # 4
21 echo "4<br />";print_r ($var);
22 echo "<hr />";
23
24 ?>
```

```
1
2  <?php
3  /* functia array_keys () */
4  $a=array("Nume"      => "Popescu",
5          "Prenume"   => "Vasile",
6          "An_nastere" => "1964",
7          "Localitatea" => "Chisinau");
8
9  $var=array_keys ($a);
10 print_r ($a);           # $a nu se modifica
```

```
10      echo "<hr />";          #####
11      echo "1<br />";print_r ($var);      # 1
12      echo "<hr />";          #####
13      $var=array_keys ($a, 1964);          # 2
14      echo "2<br />";print_r ($var);
15      echo "<hr />";          #####
16      $var=array_keys ($a, 1964, true);      # 3
17      echo "3<br />";print_r ($var);
18      echo "<hr />";          #####
19      $var=array_keys ($a, "1964", true);      # 4
20      echo "4<br />";print_r ($var);
21      echo "<hr />";
?>
```

5.2.5. functia array_values()

Sintaxa

\$nume_var = array_values (\$array);

Descriere si functionare

Aceasta functie, intoarce o variabila matrice indexata, ale carei elemente vor avea valorile variabilei matrice argument **\$array**, care macar pentru logica, trebuie sa fie asociativa.

Exemplu

```
1
2  <?php
3  /*  functia array_values () */
4  $a=array("Nume"      => "Popescu",
5          "Prenume"   => "Vasile",
6          "An_nastere" => "1964",
7          "Localitatea" => "Chisinau");
8  $var=array_values ($a);
9  print_r ($a);          // $a nu se modifica
10     echo "<hr />";
11  print_r ($var);
12     echo "<hr />";
?>
```

```
1
2  <?php
3  /*  functia array_values () */
4  $a=array("Nume"      => "Popescu",
5          "Prenume"   => "Vasile",
6          "An_nastere" => "1964",
7          "Localitatea" => "Chisinau");
8  $var=array_values ($a);
9  print_r ($a);          // $a nu se modifica
10     echo "<hr />";
11  print_r ($var);
12     echo "<hr />";
?>
```

5.2.6. functia explode ()

Sintaxa

\$nume_var = explode (separator, sir, limita);

Argumentele functiei *explode ()*

Argument	Descriere
----------	-----------

separator	Necesar. Specifica separatorul care indica unde se intrerupe sirul.
sir	Necesar. Este sirul care se imparte, pentru a devenii variabila matrice.
limita	Optional. Specifica numarul maxim de elemente a variabilei matrice care va fi intoarsa.

Descriere si functionare

Converteste un sir de caractere (string) intr-o variabila matrice, prin divizarea cu un separator specificat. Intoarce variabila matrice cu subsiruri, creat prin impartirea sirului, ori de cate ori intalneste separatorul.

Nota: Separatorul nu poate fi un sir gol.

Exemplu

```

1  <?php
2  /* functia explode () */
3  $str = "Salut lume. Este o zi minunata, ca fiecare zi, de altfel.";
4  echo "ex 1 <br />";
5  $var=explode(" ", $str);
6  print_r ($var);          ## 1
7  echo "<hr />";          #####
8  echo "ex 2 <br />";
9  print_r (explode(".", $str)); ## 2
10 echo "<hr />";          #####
11 echo "ex 2 <br />";
12 print_r (explode(",", $str, 2)); ## 3
13 echo "<hr />";
14 ?>

```

```

1  <?php
2  /* functia explode () */
3  $str = "Salut lume. Este o zi minunata, ca fiecare zi, de altfel.";
4  echo "ex 1 <br />";
5  $var=explode(" ", $str);
6  print_r ($var);          ## 1
7  echo "<hr />";          #####
8  echo "ex 2 <br />";
9  print_r (explode(".", $str)); ## 2
10 echo "<hr />";          #####
11 echo "ex 2 <br />";
12 print_r (explode(",", $str, 2)); ## 3
13 echo "<hr />";
14 ?>

```

5.2.7. functia compact ()

Sintaxa

\$nume_var = compact (var1, var2,...);

Descriere si functionare

Aceasta functie creaza o variabila matrice, din mai multe variabile simple a caror nume sunt trecute ca argumente ale functiei (**var1, var2, ...**), fara caracterul \$. Argumentele functiei (numele functiilor simple, fara caracterul \$), devin cheile noi variabile matrice, intoarse de functie, iar valorile variabilelor simple, devin valorile corespunzatoare ale noi variabile.

Exemplu

```
1  <?php
2  /* functia compact () */
3  $a1 = "Salut lume.";
4  $a2="Este o zi minunata";
5  $a3="Ca fiecare zi, de altfel.";
6  $comp=compact ("a1", "a2", "a3");
7      echo "$a1<br />";
8      echo "$a2<br />";
9      echo "$a3<br />";
10 echo "<hr />";          #####
11      print_r ($comp);
12 echo "<hr />";
13 ?>
```



```
1  <?php
2  /* functia compact () */
3  $a1 = "Salut lume.";
4  $a2="Este o zi minunata";
5  $a3="Ca fiecare zi, de altfel.";
6  $comp=compact ("a1", "a2", "a3");
7      echo "$a1<br />";
8      echo "$a2<br />";
9      echo "$a3<br />";
10 echo "<hr />";          #####
11      print_r ($comp);
12 echo "<hr />";
13 ?>
```

5.2.8. functia array_reverse ()

Sintaxa

\$nume_var = array_reverse (\$array, pastrare);

Argumentele functiei array_reverse ()

Argument	Descriere
\$array	Necesar. Specifica variabila matrice care se proceseaza.
pastrare	Optional. Specifica daca functia isi pastreaza cheile matricii sau nu. Valori posibile: - true - pastreaza cheile variabilei de origine - false - predefinit - alocu chei noi

Descriere si functionare

Acesta funcție întoarce o nouă variabilă matrice, cu valorile elementelor in ordine inversa decat a variabilei matrice care se prelucreaza.

Exemplu

```
1  <?php
2  /* functia array_reverse () */
3  $var_ar=array("cap1", "cap2", "cap3");
4      print_r ($var_ar);
5      echo "<hr /> ex1 <br />";          #####
6  $b=array_reverse ($var_ar);
7      print_r ($b);                      # 1
8      echo "<hr /> ex2 <br />";          #####
9  
```

```
10 | print_r(array_reverse ($var_ar, true)); # 2
11 |     echo "<hr />"; #####
12 | print_r ($var_ar);
13 |     echo "<hr />";
    | ?>

1 |
2 | <?php
3 | /* functia array_reverse () */
4 | $var_ar=array("cap1", "cap2", "cap3");
5 |     print_r ($var_ar);
6 |     echo "<hr /> ex1 <br />"; #####
7 | $b=array_reverse ($var_ar);
8 |     print_r ($b); # 1
9 |     echo "<hr /> ex2 <br />"; #####
10 | print_r(array_reverse ($var_ar, true)); # 2
11 |     echo "<hr />"; #####
12 | print_r ($var_ar);
13 |     echo "<hr />";
    | ?>
```

5.2.9. functia *array_flip ()*

Sintaxa

\$nume_var = array_flip (\$array);

Descriere si functionare

Aceasta functie creaza o variabila matrice, care preia valorile matricii care se proceseaza ca chei, iar cheile ca valori.

Exemplu

```
1 |
2 | <?php
3 | /* functia array_flip () */
4 | $var_ar=array("cap1", "cap2", "cap3");
5 | print_r ($var_ar);
6 | echo "<hr />";
7 | #####
8 | $b=array_flip ($var_ar);
9 | print_r ($b);
10 | echo "<hr />";
11 | #####
12 | print_r ($var_ar);
13 | echo "<hr />";
    | ?>

1 |
2 | <?php
3 | /* functia array_flip () */
4 | $var_ar=array("cap1", "cap2", "cap3");
5 | print_r ($var_ar);
6 | echo "<hr />";
7 | #####
8 | $b=array_flip ($var_ar);
9 | print_r ($b);
10 | echo "<hr />";
11 | #####
12 | print_r ($var_ar);
13 | echo "<hr />";
    | ?>
```

5.2.10. functia *array_merge ()*

Sintaxa

```
$nume_var = array_merge ($array1, $array2, ...);
```

Descriere si functionare

Aceasta functie uneste una **\$array1** (necesar), sau mai multe **\$array2, ...** (optionale) variabile matrice intr-o noua variabila matrice **\$nume_var**.

Nota: Daca doua sau mai multe elemente a variabilelor matrice au aceasi cheie (in cazul variabilelor asociative), cheia ultimei variabile, se suprapune peste celelalte, rezultand un numar de elemente mai redus.

Daca atribuiti o singura variabila functiei si cheile sunt numere intregi, functia va intoarce o noua variabila cu cheie numere intregi pornind de la 0 si crescand cu 1.

Exemplu

```
1  <?php
2  /* functia array_flip () */
3  $var_ar=array("cap1", "cap2", "cap3");
4  print_r ($var_ar);
5  echo "<hr />";
6  #####
7  $b=array_flip ($var_ar);
8  print_r ($b);
9  echo "<hr />";
10 #####
11 print_r ($var_ar);
12 echo "<hr />";
13 ?>
```

```
1  <?php
2  /* functia array_flip () */
3  $var_ar=array("cap1", "cap2", "cap3");
4  print_r ($var_ar);
5  echo "<hr />";
6  #####
7  $b=array_flip ($var_ar);
8  print_r ($b);
9  echo "<hr />";
10 #####
11 print_r ($var_ar);
12 echo "<hr />";
13 ?>
```

5.2.11. functia array_diff ()

Sintaxa

```
$nume_var = array_diff ($array1, $array2, ...);
```

Descriere si functionare

Aceasta functie intoarce o variabila matrice, care are ca elemente, elementele (chei si valori) matricii argument **\$array1**, din care se retrag elementele a caror valoare se regaseste in una din variabilele matrice argument **\$array2,** Atentie, se compara doar valorile, nu si cheile variabilelor.

Exemplu

```
1  <?php
2  /* functia array_diff () */
3  echo "<h3>functia array_diff ()</h3>";
4  $var1=array("Popescu", "Nicolae", "Cimislia");
5  $var2=array("Popescu", "Vasile", "Chisinau");
6  echo "<h4>var1=</h4>";
7  print_r ($var1);
8  
```

```

9      echo "<h4>var2=</h4>";
10     print_r ($var2);
11     echo "<h4>array diff=</h4>";
12     $b=array_diff ($var1, $var2);
13     print_r ($b);
14     echo "<hr />";
15     #####
16     $var1=array("nume"      => "Popescu",
17                "prenume"   => "Nicolae",
18                "localitatea" => "Cimislia");
19     $var2=array("nume"      => "Popescu",
20                "prenume"   => "Vasile",
21                "an nastere" => "1959");
22     echo "<h4>var1=</h4>";
23     print_r ($var1);
24     echo "<h4>var2=</h4>";
25     print_r ($var2);
26     echo "<h4>array diff=</h4>";
27     $b=array_diff ($var1, $var2);
28     print_r ($b);
29     ?>

```

```

1  <?php
2  /* functia array_diff () */
3  echo "<h3>functia array_diff (</h3>";
4  $var1=array("Popescu", "Nicolae", "Cimislia");
5  $var2=array("Popescu", "Vasile", "Chisinau");
6  echo "<h4>var1=</h4>";
7  print_r ($var1);
8  echo "<h4>var2=</h4>";
9  print_r ($var2);
10 echo "<h4>array diff=</h4>";
11 $b=array_diff ($var1, $var2);
12 print_r ($b);
13 echo "<hr />";
14 #####
15 $var1=array("nume"      => "Popescu",
16            "prenume"   => "Nicolae",
17            "localitatea" => "Cimislia");
18 $var2=array("nume"      => "Popescu",
19            "prenume"   => "Vasile",
20            "an nastere" => "1959");
21 echo "<h4>var1=</h4>";
22 print_r ($var1);
23 echo "<h4>var2=</h4>";
24 print_r ($var2);
25 echo "<h4>array diff=</h4>";
26 $b=array_diff ($var1, $var2);
27 print_r ($b);
28 ?>

```

5.2.12. functia array_unique ()

Sintaxa

\$nume_var = array_unique (\$array);

Descriere si functionare

Aceasta functie intoarce o variabila matrice, care are ca elemente, elementele matricii argument **\$array**, dar pastreaza o singura valoare in cazul in care sunt mai multe elemente cu aceasi valoare.

Nota : Variabila intoarsa va pastra cheia primului element duplicat, al variabilei matrice argument.

Exemplu

```

1  <?php
2  /* functia array_unique () */
3

```

```
4  echo "<h3>functia array_unique ()</h3>";
5  $var=array("nume"      => "Popescu",
6            "prenume"   => "Nicolae",
7            "loc_domic" => "Cimislia",
8            "loc_nast"  => "Cimislia");
9      echo "<h4>var=</h4>";
10     print_r ($var);
11     echo "<h4>array unique=</h4>";
12     $b=array_unique ($var);
13     print_r ($b);
14     echo "<hr />";
15     #####
16     $var=array("Popescu", "Nicolae", "Chisinau", "Chisinau");
17     echo "<h4>var=</h4>";
18     print_r ($var);
19     echo "<h4>array unique=</h4>";
20     $b=array_unique ($var);
21     print_r ($b);
22     echo "<hr />";
    ?>
```

```
1
2  <?php
3  /* functia array_unique () */
4  echo "<h3>functia array_unique ()</h3>";
5  $var=array("nume"      => "Popescu",
6            "prenume"   => "Nicolae",
7            "loc_domic" => "Cimislia",
8            "loc_nast"  => "Cimislia");
9      echo "<h4>var=</h4>";
10     print_r ($var);
11     echo "<h4>array unique=</h4>";
12     $b=array_unique ($var);
13     print_r ($b);
14     echo "<hr />";
15     #####
16     $var=array("Popescu", "Nicolae", "Chisinau", "Chisinau");
17     echo "<h4>var=</h4>";
18     print_r ($var);
19     echo "<h4>array unique=</h4>";
20     $b=array_unique ($var);
21     print_r ($b);
22     echo "<hr />";
    ?>
```

5.2.13. functia *array_intersect* ()

Sintaxa

\$nume_var = array_intersect (\$array1, \$array2, ...);

Descriere si functionare

Aceasta functie intoarce o variabila matrice, care are ca elemente, elementele matricii argument **\$array1** (necesar), dar doar valorile care se regasesc si in celelalte variabile matrice argument **\$array2, ...** (necesar \$array2, celelalte optionale).

Variabila intoarsa, va avea cheile si valorile primei variabile matrice argument, dar doar daca valorile se regasesc si in celelalte variabile matrice. Daca mai multe chei au aceeasi valoare, prima cheie va fi pastrata.

Exemplu

```
1
2  <?php
3  /* functia array_intersect () */
4  echo "<h3>functia array_intersect ()</h3>";
5  $var1=array("trotineta", "bicicleta", "motoreta", "motocicleta");
6  $var2=array(2=>"camion", "autoturism", "motoreta", "motocicleta");
7      echo "<h4>var1=</h4>";
```

```

8     print_r ($var1);
9     echo "<h4>var2=</h4>";
10    print_r ($var2);
11    echo "<h4>array intersect=</h4>";
12    $b=array_intersect ($var1, $var2);
13    print_r ($b);
14    echo "<hr />";
15    ?>

1
2    <?php
3    /* functia array_intersect () */
4    echo "<h3>functia array_intersect ()</h3>";
5    $var1=array("trotineta", "bicicleta", "motoreta", "motocicleta");
6    $var2=array(2=>"camion", "autoturism", "motoreta", "motocicleta");
7    echo "<h4>var1=</h4>";
8    print_r ($var1);
9    echo "<h4>var2=</h4>";
10   print_r ($var2);
11   echo "<h4>array intersect=</h4>";
12   $b=array_intersect ($var1, $var2);
13   print_r ($b);
14   echo "<hr />";
15   ?>

```

5.2.14. functia *array_filter* ()

Sintaxa

\$nume_var = array_filter (\$array, "nume_functie");

Descriere si functionare

Aceasta functie intoarce o variabila matrice **\$nume_var**, din elementele care intorc TRUE ale unei variabile matrice **\$array** atunci cand sunt aplicate unei functii utilizator de tip comparativ **nume_functie**, asociata functiei **array_filter**, ca argument.

Exemplu

```

1
2    <?php
3    /* functia array_intersect () */
4    echo "<h3>functia array_intersect ()</h3>";
5    $var1=array("trotineta", "bicicleta", "motoreta", "motocicleta");
6    $var2=array(2=>"camion", "autoturism", "motoreta", "motocicleta");
7    echo "<h4>var1=</h4>";
8    print_r ($var1);
9    echo "<h4>var2=</h4>";
10   print_r ($var2);
11   echo "<h4>array intersect=</h4>";
12   $b=array_intersect ($var1, $var2);
13   print_r ($b);
14   echo "<hr />";
15   ?>

1
2    <?php
3    /* functia array_intersect () */
4    echo "<h3>functia array_intersect ()</h3>";
5    $var1=array("trotineta", "bicicleta", "motoreta", "motocicleta");
6    $var2=array(2=>"camion", "autoturism", "motoreta", "motocicleta");
7    echo "<h4>var1=</h4>";
8    print_r ($var1);
9    echo "<h4>var2=</h4>";
10   print_r ($var2);
11   echo "<h4>array intersect=</h4>";
12   $b=array_intersect ($var1, $var2);
13   print_r ($b);
14   echo "<hr />";

```

```
?>
```

Structura *foreach*

Această structură poate fi folosită pentru a realiza o iterație printre toate elementele unui vector. Așadar, ea nu poate fi folosită decât împreună cu vectorii; utilizarea sa asupra unei variabile de alt tip duce la apariția de erori. Există două sintaxe acceptate pentru această structură și anume:

```
1 foreach (expresie_vectoriala as $valoare){
2     instructiune;
3 }
```

```
1 foreach (expresie_vectoriala as $valoare){
2     instructiune;
3 }
```

sau

```
1 foreach (expresie_vectoriala as $cheie => $valoare){
2     instructiune;
3 }
```

```
1 foreach (expresie_vectoriala as $cheie => $valoare){
2     instructiune;
3 }
```

Dacă se utilizează prima variantă, atunci la fiecare iterație valoarea elementului curent este atribuită variabilei *\$valoare* și apoi se trece la elementul următor (a cărui valoare va fi atribuită variabilei la următoarea iterație). Execuția ciclului se încheie în momentul în care nu mai există alte elemente în vector. Singura diferență care apare în cazul utilizării celei de-a doua variante este faptul că la fiecare iterație valoarea cheii elementului curent este atribuită variabilei *\$cheie*.

În continuare este un exemplu de folosire a celor două sintaxe ale structurii **foreach**.

```
1 <?php
2 $sir=array("unu", "doi", "trei", "patru", "cinci");
3 foreach($sir as $valoare) {
4     echo "Valoare:".$valoare."<br>\n";
5 }
6 ?>
```

```
1 <?php
2 $sir=array("unu", "doi", "trei", "patru", "cinci");
3 foreach($sir as $valoare) {
4     echo "Valoare:".$valoare."<br>\n";
5 }
6 ?>
```

```
1 <?php
2
```

```

3  $sir=array("unu", "doi", "trei","patru","cinci");
4  foreach($sir as $cheie=> $valoare) {
5  echo "Cheie:".$cheie."Valoare:".$valoare."<br>\n";
6  }
  ?>

1
2  <?php
3  $sir=array("unu", "doi", "trei","patru","cinci");
4  foreach($sir as $cheie=> $valoare) {
5  echo "Cheie:".$cheie."Valoare:".$valoare."<br>\n";
6  }
  ?>

```

Ordonarea a 3 numere în ordine crescătoare.

```

1
2  <?php
3  $n["d"]=7;
4  $n["b"]=45;
5  $n["c"]=3;
6  asort($n);
7  foreach($n as $key=> $vall)
8  {echo"$vall";}
  ?>

1
2  <?php
3  $n["d"]=7;
4  $n["b"]=45;
5  $n["c"]=3;
6  asort($n);
7  foreach($n as $key=> $vall)
8  {echo"$vall";}
  ?>

```

Instrucțiunea *break*

Această instrucțiune poate fi folosită pentru a întrerupe forțat execuția unui ciclu sau a secvenței de instrucțiuni corespunzătoare unei structuri **switch**. Instrucțiunea poate fi urmată de un argument care indică numărul de structuri îmbricate a căror execuție se încheie. Valoarea implicită este 1, deci se întrerupe execuția unei singure structuri. Următoarea secvență de cod PHP realizează parcurgerea elementelor unui vector de numere întregi pînă în momentul în care se întâlnește un număr negativ.

```

1
2  <?php
3  foreach ($a as $v)
4  if($v<0)
5  break;
  ?>

1
2  <?php
3  foreach ($a as $v)
4  if($v<0)
5  break;
  ?>

```

Mai departe aveți cazul în care este întreruptă execuția mai multor cicluri; vom considera că parcurgem elementele unei matrice pătratică cu n elemente și n coloane pînă în momentul în care întâlnim o valoare nulă.

```

1
2 <?php
3 for ($i = 0; $i < $n; $i++)
4 for($j=0;$j<$n;$j++)
5 if(!$a[$i][$j])
6 break 2;
?>

```

```

1
2 <?php
3 for ($i = 0; $i < $n; $i++)
4 for($j=0;$j<$n;$j++)
5 if(!$a[$i][$j])
6 break 2;
?>

```

Instrucțiunea **break** poate fi utilizată pentru întreruperea execuției secvențelor de instrucțiuni corespunzătoare structurilor **for**, **foreach**, **while**, **do - while** și **switch**.

Instrucțiunea *continue*

Această instrucțiune este folosită pentru a întrerupe execuția secvenței de instrucțiuni din interiorul unui ciclu și trecerea la următoarea iterație. În cazul instrucțiunii **for**, înainte de următoarea iterație se evaluează (execută) expresia de incrementare (expresia #3 din sintaxa generală). La fel ca și în cazul instrucțiunii **break**, poate apărea un argument care indică numărul structurilor îmbricate asupra cărora are efect. Exemplul următor realizează afișarea elementelor unui șir de numere întregi care sunt mai mari decât 1000.

Cod

```

1
2 <?php
3 foreach($a as $v)
4 {
5 if($v <=1000)
6 continue;
7 echo $v;
8 }
?>

```

```

1
2 <?php
3 foreach($a as $v)
4 {
5 if($v <=1000)
6 continue;
7 echo $v;
8 }
?>

```

Următorul exemplu ilustrează efectul folosirii argumentelor pentru instrucțiunea **continue**.

Cod:

```

1
2 <?php
3 $i = 0;
4 while($i++ <5)
5 {
6 echo "Ciclul #1 <br>\n";
7 while(1)
8 {

```

```

9      echo "??Ciclul #2 <br>\n";
10     while(1)
11     {
12         echo "??Ciclul #3<br>\n";
13         continue 3;
14     }
15     echo "Acest mesaj nu va fi afisat niciodata.<br>\n";
16 }
17 echo "Nici acest mesaj nu va fi afisat niciodata.<br>\n";
18 }
?>

```

```

1  <?php
2  $i = 0;
3  while($i++ <5)
4  {
5      echo "Ciclul #1 <br>\n";
6      while(1)
7      {
8          echo "??Ciclul #2 <br>\n";
9          while(1)
10         {
11             echo "??Ciclul #3<br>\n";
12             continue 3;
13         }
14         echo "Acest mesaj nu va fi afisat niciodata.<br>\n";
15     }
16     echo "Nici acest mesaj nu va fi afisat niciodata.<br>\n";
17 }
18 }
?>

```

Rezultatul codului de mai sus este:

Ciclul #1
 Ciclul #2
 Ciclul #3
 Ciclul #1
 Ciclul #2
 Ciclul #3
 Ciclul #1
 Ciclul #2
 Ciclul #3
 Ciclul #1
 Ciclul #2
 Ciclul #3
 Ciclul #1
 Ciclul #2
 Ciclul #3

Exerciții:

1. Scrieți codul PHP care să sorteze crescător/decrescător un tablou de valori cu ajutorul algoritmului de sortare numit "metoda bulelor"(BubbleSort).
2. Este dat un tablou cu numere întregi. De calculat:
 - a) suma elementelor;
 - b) elementul maxim/minim și numărul lor;
 - c) numărul de elemente prime.
3. Este dat un tablou cu numere întregi. Scrieți codul PHP care verifica dacă cel puțin 3 numere dintr-un

tablou sunt consecutive. Ex. (0,2,3,4,5) => 3,4,5 - sunt 3 numere consecutive

4. Se consideră un tablou bidimensional $a[1..n, 1..n]$ format din numere întregi. Elaborați un script care determină o permutare a coloanelor tabloului astfel încât suma componentelor de pe diagonala principală să fi e minimă.

Tipul șir de caractere, în PHP

Șirurile de caractere sunt bucati de text, bine delimitate, folosite in codul-sursa pentru diferite scopuri. PHP prezinta particularitati in modul in care sunt folosite string-urile, dupa cum se poate vedea in exemplele de mai jos.

Important! Sirurile de caractere sunt expresii (entitati ce au si returneaza o valoare). Asadar, un string poate fi folosit, pe langa, afisare, in atribuirii, la verificari, etc. In exemplele ce urmeaza s-a optat pentru afisarea sirurilor.

Siruri de caractere delimitate de ghilimele duble

Sirurile delimitate prin ghilimele duble au particularitatea ca pot interpreta variabilele si caracterele speciale din interiorul lor. Astfel, la evaluarea textului, variabilele existente in interiorul lui sunt inlocuite cu valoarea lor iar rezultatul final este returnat.

Exemplu:

```
1
2 <?php
3 $a = "valoarea variabilei"; // definesc o variabila
4 print "Textul definit cu ghilimele contine $a";
5
6 // va afisa
7 // Textul definit cu ghilimele contine valoarea variabilei
?>
```

```
1
2 <?php
3 $a = "valoarea variabilei"; // definesc o variabila
4 print "Textul definit cu ghilimele contine $a";
5
6 // va afisa
7 // Textul definit cu ghilimele contine valoarea variabilei
?>
```

O alta particularitate a acestor siruri o reprezinta folosirea backslash-ului (caracterul \). Acesta are o functie speciala de marcare a anumitor caractere care nu pot fi incluse in mod normal intr-un text (din cauza ca sunt ele inele caractere speciale). Din acest motiv backslash-ul poarta numele de escape character.

Exemplu:

```
1
2 <?php
3 print "Ma numesc \"Ion\"!"; // sir ce contine ghilimele, se foloseste \" pentru a afisa
4 ghilimele
5
6 print "Text afisat pe 3 linii: \n linia 2 si \n linia3";
7 // atentie, textul e afisat pe 2 linii in sursa HTML cu ajutorul caracterului \n (new-line)
8 // pentru verificare vizualizati sursa paginii (alegeti "View Source" din browser)
9
10 print "Am castigat \$30 :D"; // caracterul dolar are un statut aparte (marcheaza variabilele)
11 // pentru a afisa semnul $ asa cum este se foloseste \$
12
13 print "Text cu \\backslash\\."; // intrucat backslash \ este un caracter special el
```

```

14 // poate fi afisat ca atare intr-un text doar daca este dublat
15
16 // daca e folosit doar un singur \ PHP ar trata textul diferit:
17 print "Text fara \backslash- \nu se afiseaza corec\t.";
18
19 ?>
20
21
22 1
23 2 <?php
24 3 print "Ma numesc \"Ion\"!"; // sir ce contine ghilimele, se foloseste \" pentru a afisa
25 4 ghilimele
26 5
27 6 print "Text afisat pe 3 linii: \n linia 2 si \n linia3";
28 7 // atentie, textul e afisat pe 2 linii in sursa HTML cu ajutorul caracterului \n (new-line)
29 8 // pentru verificare vizualizati sursa paginii (alegeti "View Source" din browser)
30 9
31 10 print "Am castigat \$30 :D"; // caracterul dolar are un statut aparte (marcheaza variabilele)
32 11 // pentru a afisa semnul $ asa cum este se foloseste \$
33 12
34 13 print "Text cu \\backslash."; // intrucat backslash \ este un caracter special el
35 14 // poate fi afisat ca atare intr-un text doar daca este dublat
36 15
37 16 // daca e folosit doar un singur \ PHP ar trata textul diferit:
38 17 print "Text fara \backslash- \nu se afiseaza corec\t.";
39
40 ?>

```

Siruri de caractere delimitate de ghilimele simple

Sirurile delimitate prin ghilimele simple nu permit interpretarea variabilelor continute si nici a caracterelor speciale cu exceptia \' si \\

Exemplu:

```

1
2 <?php
3 $a = 'valoarea variabilei'; // definesc o variabila
4 print 'Textul definit cu ghilimele contine $a'; // Textul definit cu ghilimele contine $a
5
6 print 'Porecla ta e Kelu\' ?'; // sir delimitat de ghilimele simple ce contine un apostrof
7 print 'Text cu backslash \\''; // sir delimitat de ghilimele simple ce contine un backslash
8
9 print 'Alte escape chars nu sunt interpretate \n \t \b'; // textul este afisat ca atare
10 ?>
11
12
13 1
14 2 <?php
15 3 $a = 'valoarea variabilei'; // definesc o variabila
16 4 print 'Textul definit cu ghilimele contine $a'; // Textul definit cu ghilimele contine $a
17 5
18 6 print 'Porecla ta e Kelu\' ?'; // sir delimitat de ghilimele simple ce contine un apostrof
19 7 print 'Text cu backslash \\''; // sir delimitat de ghilimele simple ce contine un backslash
20 8
21 9 print 'Alte escape chars nu sunt interpretate \n \t \b'; // textul este afisat ca atare
22 10 ?>

```

Siruri de caractere delimitate cu notatia speciala <<<

Aceste siruri de caractere au avantajul ca nu necesita marcarea (escaparea) delimitatorilor prin \' sau \". In rest, aceste stringuri sunt tratate in acelasi mod ca cele delimitate de ghilimele duble, in sensul ca permit interpretarea variabilelor si a altor caractere speciale.

Exemplu:

```
1
2 <?php
3 $a = 'valorii lor'; // definesc o variabila
4
5 print <<<TXT
6 Text pe mai multe linii. <br />
7 Delimitatorii pot avea orice nume: TXT, START, etc, cu urmatoarele conditii: <br />
8 - ambii delimitatori (de inceput si sfarsit) trebuie sa aiba acelasi nume
9 - inainte de primul delimitator se foloseste <<<
10 - delimitatorul de inceput nu trebuie sa fie urmat de spatiu sau alt caracter (important!)
11 - delimitatorul de final sa fie la inceputul liniei (fara spatii inainte)
12 - dupa delimitatorul final se pune punct si virgula ;
13 - pot contine ghilimele " sau apostrof ' fara nevoia de a le escapa
14 - permit interpretarea variabilelor si afisarea $a
15 TXT;
16 ?>
```

```
1
2 <?php
3 $a = 'valorii lor'; // definesc o variabila
4
5 print <<<TXT
6 Text pe mai multe linii. <br />
7 Delimitatorii pot avea orice nume: TXT, START, etc, cu urmatoarele conditii: <br />
8 - ambii delimitatori (de inceput si sfarsit) trebuie sa aiba acelasi nume
9 - inainte de primul delimitator se foloseste <<<
10 - delimitatorul de inceput nu trebuie sa fie urmat de spatiu sau alt caracter (important!)
11 - delimitatorul de final sa fie la inceputul liniei (fara spatii inainte)
12 - dupa delimitatorul final se pune punct si virgula ;
13 - pot contine ghilimele " sau apostrof ' fara nevoia de a le escapa
14 - permit interpretarea variabilelor si afisarea $a
15 TXT;
16 ?>
```

In versiunile mai noi de PHP (de la 5.3.0) a fost introdusa posibilitatea de a defini siruri prin notatia speciala <<< fara a interpreta variabilele continute. Exemplul de mai jos foloseste un sir astfel definit (vezi diferenta la delimitatorul de inceput).

Exemplu:

```
1
2 <?php
3 $a = 'valoarea'; // definesc o variabila
4
5 print <<<'TXT'
6 Incepand cu versiunea 5.3.0 exista un alt mod de delimitare a sirurilor
7 in care variabilele si caracterele speciale nu sunt interpretate. <br />
8 Asadar $a si \n raman asa cum sunt.
9 TXT;
10 ?>
```

```
1
2 <?php
3 $a = 'valoarea'; // definesc o variabila
4
5 print <<<'TXT'
6 Incepand cu versiunea 5.3.0 exista un alt mod de delimitare a sirurilor
7 in care variabilele si caracterele speciale nu sunt interpretate. <br />
8 Asadar $a si \n raman asa cum sunt.
9 TXT;
10 ?>
```

Operatii cu siruri de caractere

Mai jos sunt prezentate operatiile uzuale cu siruri de caractere si functiile oferite de limbajul PHP pentru

realizarea lor.

Lungimea sirului

Exemplu:

```
1
2 $s = "acesta este un text";
3 $sir = "stiu PHP stiu HTML stiu CSS";
4
5 # ce lungime are sirul? (numarul de caractere)
  print strlen( $s ); // 19
```

```
1
2 $s = "acesta este un text";
3 $sir = "stiu PHP stiu HTML stiu CSS";
4
5 # ce lungime are sirul? (numarul de caractere)
  print strlen( $s ); // 19
```

Cautarea unei secvente

Exemplu:

```
1
2 # verific daca un cuvant sau text (in cazul de fata cuvantul 'PHP') apare in
3 # sirul exprimat prin variabila $sir
4 if( strstr( $sir, 'PHP' ) !== false ) print 'gasit';
5 else print "nu am gasit";
6
7 # pentru a nu tine cont de litere mari/mici se foloseste strstr
  if( strstr( $sir, 'php' ) !== false ) print 'gasit';
```

```
1
2 # verific daca un cuvant sau text (in cazul de fata cuvantul 'PHP') apare in
3 # sirul exprimat prin variabila $sir
4 if( strstr( $sir, 'PHP' ) !== false ) print 'gasit';
5 else print "nu am gasit";
6
7 # pentru a nu tine cont de litere mari/mici se foloseste strstr
  if( strstr( $sir, 'php' ) !== false ) print 'gasit';
```

Afisarea unui subsir

Exemplu:

```
1
2 # afisez o sectiune din sir
3 print substr( $sir, 0, 4); // stiu
4 print substr( $sir, 5 ); // PHP stiu HTML stiu CSS
5 print substr( $sir, 5, -3 ); // PHP stiu HTML stiu
6 print substr( $sir, -3 ); // CSS
7
8 # returnez doar un caracter din string
9 print $sir{5}; // P
  print $sir{ strlen($sir)-1 }; // S
```

```
1
2 # afisez o sectiune din sir
3 print substr( $sir, 0, 4); // stiu
4 print substr( $sir, 5 ); // PHP stiu HTML stiu CSS
5 print substr( $sir, 5, -3 ); // PHP stiu HTML stiu
6 print substr( $sir, -3 ); // CSS
```

```

7
8 # returnez doar un caracter din string
9 print $sir{5}; // P
print $sir{ strlen($sir)-1 }; // S

```

Transformarea sirului

Exemplu:

```

1
2 # inlocuirea unor secvente
3 print str_replace( "stiu", "invat", $sir); // invat PHP invat HTML invat CSS
4
5 # schimb tipul literelor (mari, mici)
6 print strtoupper( $s ); // ACESTA ESTE UN TEXT
7 print strtolower( $sir ); // stiu php stiu html stiu css
8 print ucfirst( $s ); // Acesta este un text
9 print ucwords( $s ); // Acesta Este Un Text
10
11 # sterg spatiile de la inceput si sfarsit: trim, ltrim, rtrim
12 print trim('      ok      '); // ok
13
14 # caractere "enter" transformate in <br />
15 print nl2br( "acesta e afisat pe \n 2 linii" ); // acesta e afisat pe <br /> 2 linii

```

```

1
2 # inlocuirea unor secvente
3 print str_replace( "stiu", "invat", $sir); // invat PHP invat HTML invat CSS
4
5 # schimb tipul literelor (mari, mici)
6 print strtoupper( $s ); // ACESTA ESTE UN TEXT
7 print strtolower( $sir ); // stiu php stiu html stiu css
8 print ucfirst( $s ); // Acesta este un text
9 print ucwords( $s ); // Acesta Este Un Text
10
11 # sterg spatiile de la inceput si sfarsit: trim, ltrim, rtrim
12 print trim('      ok      '); // ok
13
14 # caractere "enter" transformate in <br />
15 print nl2br( "acesta e afisat pe \n 2 linii" ); // acesta e afisat pe <br /> 2 linii

```

Impartirea (spargerea) sirului

Exemplu:

```

1
2 # impart sirul dupa un caracter, cuvant sau un alt sir
3 $output1 = explode( "stiu ", $sir ); // impart dupa stiu<spatiu>
4 /*
5 Array (
6     [0] => PHP
7     [1] => HTML
8     [2] => CSS
9 )
10 */
11
12 # impart sirul dupa o expresie regulata (regex)
13 $output2 = preg_split( '/ /', $s ); // impart dupa spatiu
14 /*
15 Array (
16     [0] => acesta
17     [1] => este
18     [2] => un
19     [3] => text
20 )
21 */

```

```

22
23 # operatia inversa impartirii unui sir:
24 $a = implode( 'invat ', $output1 ); // invat PHP invat HTML invat CSS
    $b = join( '-', $output2 ); // acesta-este-un-text

1
2 # impart sirul dupa un caracter, cuvant sau un alt sir
3 $output1 = explode( "stiu ", $sir ); // impart dupa stiu<spatiu>
4 /*
5 Array (
6     [0] => PHP
7     [1] => HTML
8     [2] => CSS
9 )
10 */
11
12 # impart sirul dupa o expresie regulata (regex)
13 $output2 = preg_split( '/ /', $s ); // impart dupa spatiu
14 /*
15 Array (
16     [0] => acesta
17     [1] => este
18     [2] => un
19     [3] => text
20 )
21 */
22
23 # operatia inversa impartirii unui sir:
24 $a = implode( 'invat ', $output1 ); // invat PHP invat HTML invat CSS
    $b = join( '-', $output2 ); // acesta-este-un-text

```

Notă: implode si join sunt echivalente (nu exista nici o diferenta intre ele), pe cand explode si preg_split sunt diferite.

Concatenarea (legarea) sirurilor

Exemplu:

```

1
2 print 'Text 1' . " legat de " . 'text 2' . "\n"; // Text 1 legat de text 2
3
4 // se pot concatena siruri rezultate din alte functii sau din variabile
print ucfirst($sir) . '!!! ' . $s;

1
2 print 'Text 1' . " legat de " . 'text 2' . "\n"; // Text 1 legat de text 2
3
4 // se pot concatena siruri rezultate din alte functii sau din variabile
print ucfirst($sir) . '!!! ' . $s;

```

Nota: prin concatenarea sirurilor se obtine in final un singur sir, care este tratat ca atare, de sine statator. Altfel spus, prin legarea mai multor siruri se obtine o singura entitate (o singura expresie). Aceasta poate fi transmisa ca parametru unor functii sau instructiuni precum print care accepta un singur argument.

Interpretarea sirului

Exemplu:

```

1
2 # parsez un Query String
3 $str = "first=value&arr[]=foo+bar&arr[]=baz";
4 parse_str($str);
5 print $first; // value
print $arr[0]; // foo bar

```

```

6 |
7 | print $arr[1]; // baz
8 |
9 | parse_str($str, $output);
10 | print $output['first']; // value
11 | print $output['arr'][0]; // foo bar
    | print $output['arr'][1]; // baz

1 |
2 | # parsez un Query String
3 | $str = "first=value&arr[]=foo+bar&arr[]=baz";
4 | parse_str($str);
5 | print $first; // value
6 | print $arr[0]; // foo bar
7 | print $arr[1]; // baz
8 |
9 | parse_str($str, $output);
10 | print $output['first']; // value
11 | print $output['arr'][0]; // foo bar
    | print $output['arr'][1]; // baz

```

Masuri de siguranta

In cazul in care textul provine din surse nesigure (cum ar fi un formular de comentarii), atunci este indicat sa fie "sterilizat" (sanitized), prin eliminarea elementelor ce pot fi daunatoare (tag-uri HTML, caractere speciale, etc).

Exemplu:

```

1 |
2 | print addslashes( "Baiatu' ia vino-ncoa'!" ); # Baiatu\' ia vino-ncoa\'!
3 | # functia inversa este stripslashes();
4 |
5 | print htmlspecialchars("<a href='test'>Test</a>", ENT_QUOTES);
6 | # afiseaza <a href='test'>Test</a>
7 | # functia inversa este htmlspecialchars_decode()
8 |
9 | print strip_tags( "<p>E <b>bold</b></p>" ); // E bold
    | print strip_tags( "<p>E <b>bold</b></p>", '<b>' ); // E <b>bold</b>

1 |
2 | print addslashes( "Baiatu' ia vino-ncoa'!" ); # Baiatu\' ia vino-ncoa\'!
3 | # functia inversa este stripslashes();
4 |
5 | print htmlspecialchars("<a href='test'>Test</a>", ENT_QUOTES);
6 | # afiseaza <a href='test'>Test</a>
7 | # functia inversa este htmlspecialchars_decode()
8 |
9 | print strip_tags( "<p>E <b>bold</b></p>" ); // E bold
    | print strip_tags( "<p>E <b>bold</b></p>", '<b>' ); // E <b>bold</b>

```

Exerciții:

1. Dintr-o listă ce conține **n** candidați trebuie alese **m** persoane care vor fi incluse în echipa de fotbal a unui raion. Elaborați un script care afișează la ecran toate modalitățile de selecție a celor **m** persoane.
2. Se dă șirul \$sir = "Ion Vasile Petru Vitalie Dumitru";. De transformat șirul dat într-o matrice folosind funcția **explode()**.

Funcții în PHP

În limbajul PHP, funcțiile pot fi definite de către utilizator folosind următoarea sintaxă:

```

1 |
  | function numef ($param1, $param2, ..., $paramN) {

```

```

2
3 | // instructiuni
    }

1
2 | function numef ($param1, $param2, ..., $paramN) {
3 | // instructiuni
    }

```

În PHP o funcție poate fi definită oriunde în cadrul script-ului și în interiorul unei funcții poate să apară orice secvență validă de cod care include definirea de alte funcții și definiții de clase. Argumentele unei funcții trebuie separate prin virgulă, și, implicit, acestea sunt transmise prin valoare. Pentru ca funcția să returneze un rezultat se folosește construcția **return** care primește ca parametru o expresie care reprezintă valoarea funcției. În momentul în care este întâlnită construcția **return**, execuția funcției se încheie. În exemplul următor se calculează cu ajutorul unei funcții PHP, pătratul unui număr.

Cod:

```

1
2 | <?php
3 | function patrat($n)
4 | {
5 |     return $n*$n;
6 | }
7 | echo "4^2=<b>".patrat(4)."</b>";
    ?>

1
2 | <?php
3 | function patrat($n)
4 | {
5 |     return $n*$n;
6 | }
7 | echo "4^2=<b>".patrat(4)."</b>";
    ?>

```

Rezultatul codului va fi:

4^2=16

Transmiterea parametrilor prin referință

Pentru a transmite parametri unei funcții prin referință, fapt care implică modificarea valorii parametrilor și păstrarea noii valori după ce execuția funcției s-a încheiat, se folosește operatorul '&' înaintea numelui parametrului formal, în momentul definirii funcției. Următorul exemplu indică modul în care se modifică valoarea unei variabile în interiorul unei funcții și modul în care această modificare este resimțită în exteriorul acesteia:

Cod:

```

1
2 | <?php
3 | function modificInt ($s)
4 | {
5 |     $s.="prima functie.";
6 |     echo "<b>In modificInt:</b>".$s."<br>";
7 | }
8 | function modificExt(&$s)
9 | {
10 |     $s.="a doua functie.";
11 |     echo "<b>In modificExt: </b>".$s."<br>";
12 | }

```

```

13 <br />
14 $s="Iesire din "; <br />
15 echo "<b>In script: </b>".$s."<br>";
16 modificInt ($s)
17 echo "<b>In script: </b>".$s."<br>";
18 modificExt ($s);
19 echo "<b>In script:</b>".$s."<br>";
?>

1 <?php
2 function modificInt ($s)
3 {
4     $s.="prima functie.";
5     echo "<b>In modificInt:</b>".$s."<br>";
6 }
7 function modificExt(&$s)
8 {
9     $s.="a doua functie.";
10    echo "<b>In modificExt: </b>".$s."<br>";
11 }
12 }
13 <br />
14 $s="Iesire din "; <br />
15 echo "<b>In script: </b>".$s."<br>";
16 modificInt ($s)
17 echo "<b>In script: </b>".$s."<br>";
18 modificExt ($s);
19 echo "<b>In script:</b>".$s."<br>";
?>

```

Rezultat:

In script: Iesire din

In modificInt: Iesire din prima functie.

In script: Iesire din

In modificExt: Iesire din a doua functie.

In script: Iesire din a doua functie.

Nu există posibilitatea de supraîncărcare a unei funcții, de redefinire a ei după ce aceasta a fost definită în cadrul scriptului respectiv și nu există nici posibilitatea de anulare a unei funcții.

Parametri cu valori implicite

În PHP parametrii formali pot avea valori implicite, și, în cazul în care parametrul actual lipsește, atunci se va considera că are valoarea implicită. Următorul exemplu ilustrează modul de folosire al funcțiilor când acestea au parametri formali cu valori implicite:

Cod:

```

1 <?php
2 function comanda($s='cafea')
3 {
4     return"Ati comandat ;".$s.".";
5 }
6 echo comanda();
7 echo "<br>";
8 echo comanda("suc");
9 ?>

1 <?php
2 function comanda($s='cafea')
3 {
4

```

```

5 | return"Ati comandat ;".$s.".";
6 | }
7 | echo comanda();
8 | echo "<br>";
9 | echo comanda("suc");
  | ?>

```

Rezultat:

Ati comandat cafea.

Ati comandat suc.

În cazul în care se folosesc parametri cu valori implicite este necesar ca orice parametru care are o valoare implicită să se afle în partea dreaptă a tuturor parametrilor pentru care nu se folosesc valori implicite, în caz contrar interpretorul PHP nu poate să decidă cărui parametru să-i atribuie valoarea de pe o anumită poziție din lista de parametri. De exemplu, dacă avem o funcție a cărei declarație este

```

1 | function transform ($baza=10, $nr)

```

```

1 | function transform ($baza=10, $nr)

```

și care returnează rezultatul transformării lui *\$nr* din baza 16 în baza *\$baza*, a cărei valoare implicită este 10, dacă se apelează *transform(50)*, interpretorul nu atribuie valoarea 50 parametrului *\$nr*, ci parametrului *\$baza* și generează o eroare deoarece lipsește valoarea parametrului *\$nr*.

Funcții cu număr variabil de parametri

O altă facilități a limbajului PHP este aceea că oferă programatorului posibilitatea de a utiliza funcții care au un număr nedeterminat de parametri. Funcțiile care folosesc un număr variabil de parametri nu au nici o particularitate în ceea ce privește definirea lor. Aceste funcții se definesc la fel ca cele prezentate anterior, dar pentru a putea accesa parametri se vor folosi următoarele funcții predefinite:

- **func_num_args()** - această funcție returnează numărul parametrilor funcției care a apelat-o; dacă această funcție este apelată din exteriorul unei funcții definite de utilizator se va genera un mesaj de avertizare;
- **func_get_arg(arg_num)** - returnează valoarea parametrului care se află pe poziția *arg_num* în lista de parametri; primul parametru are numărul de ordine 0; dacă este apelată din exteriorul unei funcții definite de utilizator se va genera un mesaj de avertizare;
- **func_get_args()** - returnează un tablou unidimensional care conține valorile parametrilor pe care funcția apelantă i-a primit; dacă această funcție este apelată din exteriorul unei funcții definite de utilizator se va genera un mesaj de avertizare.

În continuare aveți 2 exemple de utilizare a acestor funcții. Primul exemplu afișează lista parametrilor funcției folosind funcția **func_num_args** și **func_get_arg**, iar al doilea exemplu afișează aceeași listă folosind numai funcția **func_get_args**.

Cod:

```

1 |
2 | <?php
3 | function lista_parametri()
4 | {
5 |     for ($i=0;$i<func_num_args();$i++)
6 |     {

```

```
7 | print_r(func_get_arg($i));
8 | echo "<br>";
9 | }
10| }
11| echo
    lista_parametri("Comanda:",1,"calculator",2,"procesoare","configuratie",array("local",2,3));
    ?>

1 |
2 | <?php
3 | function lista_parametri()
4 | {
5 |     for ($i=0;$i<func_num_args();$i++)
6 |     {
7 |         print_r(func_get_arg($i));
8 |         echo "<br>";
9 |     }
10| }
11| echo
    lista_parametri("Comanda:",1,"calculator",2,"procesoare","configuratie",array("local",2,3));
    ?>
```

Cod:

```
1 |
2 | <?php
3 | function lista_parametrii()
4 | {
5 |     foreach(func_get_args()as $i)
6 |     {
7 |         print_r($i);
8 |         echo "<br>";
9 |     }
10| }
11| echo lista_parametrii
    ("Comanda:",1,"calculator",2,"procesoare","configuratie",array("local",2,3));
    ?>

1 |
2 | <?php
3 | function lista_parametrii()
4 | {
5 |     foreach(func_get_args()as $i)
6 |     {
7 |         print_r($i);
8 |         echo "<br>";
9 |     }
10| }
11| echo lista_parametrii
    ("Comanda:",1,"calculator",2,"procesoare","configuratie",array("local",2,3));
    ?>
```

Rezultat:

Comanda:

1
calculator
2
procesoare
configuratie
Array ([0] => local [1] => 2 [2] => 3)

Valorile returnate de funcții

Rezultatul obținut după apelarea unei funcții poate avea orice tip. O funcție poate să returneze chiar și liste sau obiecte. În PHP există un caz special de rezultat numit referință. Pentru ca o funcție să poată returna o referință, aceasta trebuie declarată folosindu-se operatorul '&' înaintea numelui funcției. Acest operator trebuie să apară înaintea numelui funcției și în momentul când o variabilă primește ca valoare referința rezultată din apelul funcției. În exemplul următor se definește o funcție al cărui rezultat îl constituie o referință:

Cod:

```
1
2 <?php
3 function&refer()
4 {
5     global $s;
6     return $s;
7 }
8 $s="Acesta este continutul variabilei referite cu ajutorul functiei.";
9 $z=&refer();
10 echo $z;
?>
```

```
1
2 <?php
3 function&refer()
4 {
5     global $s;
6     return $s;
7 }
8 $s="Acesta este continutul variabilei referite cu ajutorul functiei.";
9 $z=&refer();
10 echo $z;
?>
```

Rezultat:

Acesta este continutul variabilei referite cu ajutorul functiei.

Spre deosebire de majoritatea limbajelor de programare moderne, o funcție PHP poate să returneze o referință la o variabilă care a fost declarată în interiorul funcției, însă acest lucru nu este indicat deoarece, în anumite cazuri, poate duce la efecte neașteptate ale executării unui script PHP. În alte limbaje de programare efectele devin uneori fatale.

Variabilele de tip funcție

O altă facilitate a limbajului PHP în ceea ce privește funcțiile este aceea că suportă variabile de tip funcție. Acest lucru este util atunci când se folosesc liste de funcții pentru prelucrarea anumitor tipuri de date. Pentru a atribui un nume de funcție unei variabile în PHP se folosește aceeași construcție ca în cazul atribuirii unui șir de caractere, și anume, o variabilă va primi ca valoare numele funcției scris între ghilimele simple sau duble. În cazul în care interpretorul PHP găsește un nume de variabilă urmată de o listă de parametri, acesta caută funcția pe care variabila o referă și în cazul în care există, o execută. Variabilele de tip funcție nu funcționează cu construcții ale limbajului ca **echo**, **unset**, **isset**, **empty**, **include** etc. Mai jos aveți un exemplu care ilustrează modul de lucru cu variabilele de tip funcție.

Cod:

```
1
2 <?php
3 function produs($a,$b)
4 {
5     return $a*$b;
```

```

6     }
7     function suma($a,$b)
8     {
9         return $a+$b;
10    }
11    $operatie='produs';
12    $rez=$operatie(4,5);
13    echo "4*5=<b>".$rez."</b><br>";
14    $operatie='suma';
15    $rez=$operatie(4,5);
16    echo"4+5=<b>".$rez."</b><br>";
    ?>

```

```

1    <?php
2    function produs($a,$b)
3    {
4        return $a*$b;
5    }
6    function suma($a,$b)
7    {
8        return $a+$b;
9    }
10   $operatie='produs';
11   $rez=$operatie(4,5);
12   echo "4*5=<b>".$rez."</b><br>";
13   $operatie='suma';
14   $rez=$operatie(4,5);
15   echo"4+5=<b>".$rez."</b><br>";
16   ?>

```

Rezultat:

4 * 5 =20

4 + 5 =9

Exerciții:

De elaborat 3 funcții care va calcula numărul permutărilor P_n , numărul aranjamentelor A_{mn} și numărul combinațiilor C_{mn} .

De elaborat funcția care calculează $n!$. Scriptul PHP va apela funcția dată pentru a calcula suma $S=1!+3!+\dots+(2*n-1)!$.

Se dă vectorul $a[1..n]$ de numere întregi. De elaborat o funcție care calculează media componentelor unui vector.

De elaborat o funcție care calculează maximul dintre două numere x și y . De elaborat un program care afișează maximul dintre numerele a, b, c, d, e .

De elaborat o funcție care convertește literele minuscule în majuscule și viceversa.

Funcțiile predefinite în PHP

Funcțiile predefinite sunt funcțiile care aparțin **limbajului PHP**.

Aici sunt prezentate, câteva din cele mai utilizate funcții PHP ce trebuie cunoscute de către cei care sunt pasionați de acest limbaj al programarii web sau lucrează în acest domeniu.

Funcții pentru Array

Aici gasiti cateva din cele mai utile functii PHP pentru Array (matrice).

1.array_change_key_case(array, case)

- Transforma cheile din primul parametru (array) in majuscule sau litere mici, dupa cum e specificat la "case" (**CASE_UPPER** sau **CASE_LOWER**). Default e CASE_LOWER. Daca "array" nu e o matrice, returneaza FALSE.

2.array_chunk(array, nr)

- Imparte "array" intr-un numar de sub-matrici, precizat la "nr", indexate de la 0. Daca "array" nu are elemente, returneaza NULL.

3.array_combine(array_k, array_v)

- Creaza un array folosind valorile din "array_k" pentru chei, iar elementele din "array_v" pentru valorile lor. Numarul de elemente din cele 2 matrice trebuie sa fie egal, in caz contrar returneaza FALSE.

4.array_count_values(array)

- Returneaza o matrice cu numarul de repetari a fiecarui element din "array".

5.array_diff(array1, array2)

- Returneaza o matrice cu elementele din "array1" care nu se gasesc in "array2".

6.array_flip(array)

- Schimba valorile si cheile intre ele. Din valorile care se repeta o pastreaza pe ultima.

7.array_intersect(array1, array2, ...)

- Returneaza o matrice cu toate valorile din "array1" care se gasesc in toati ceilalti "array" dati ca parametri.

8.array_key_exists(cheie, array)

- Returneaza TRUE daca "cheie" este o cheie (index) in matricea "array", in caz contrar returneaza FALSE.

9.array_keys(array, val)

- Returneaza o matrice cu toate cheile din "array". Parametrul "val" e optional, daca e specificat, va returna doar cheile cu valoarea "val".

10.array_map("functie", array)

- Aplica functia pentru fiecare element din "array"

11.array_merge(\$array1, \$array2, ...)

- Uneste mai multe array in unul singur. Daca au cheie de tip sir identice, va fi pastrata valoarea ultimei chei.

12.array_pop(array)

- Elimina ultimul element din "array", care va fi transferat ca valoarea returnata de "array_pop()". Daca parametru nu e matrice sau e gol, returneaza NULL.

13.array_product(array)

- Returneaza produsul elementelor din "array"

14.array_rand(array, nr)

- Preia aleator un "nr" de chei din "array", respectand ordinea lor. Daca "nr" e 1, va returna o cheie aleasa aleator. Altfel, returneaza o matrice cu acele chei preluate aleator.

15.array_reverse(array)

- Returneaza "array" cu ordinea elementelor inversata.

16.array_search("val", array)

- Returneaza cheia din "array" care are valoarea "val", sau FALSE daca nu exista. Cautarea e "case-sensitive"

17.array_shift(array)

- Elimina primul element din "array", care va fi transferat ca valoarea returnata de "array_shift()". Cheile vor fi renumerotate de la 0. Daca parametru nu e matrice sau e gol, returneaza NULL.

18.array_slice(array, start, nr)

- Returneaza un "nr" din elementele lui "array", incepand de la elementul numarul "start" (*primul e 0*), cate elemente arata "nr". Daca "nr" nu e specificat, va fi considerat pana la ultimul.

19.array_sum(array)

- Returneaza suma elementelor din "array"

20.array_unique(array)

- Returneaza matricea "array" fara valorile duplicate, acestea fiind luate o singura data.

21.array_values(array)

- Returneaza o matrice cu toate valorile din "array", ordonate numeric de la 0.

22.arsort(array)

- Sorteaza un "array" cu valorile ordonate in ordine inversa alfabetica, pastrand corelatia dintre chei si valori. In caz de succes returneaza TRUE, altfel, FALSE.

23.asort(array)

- Sorteaza un "array" cu valorile ordonate in ordine alfabetica, pastrand corelatia dintre chei si valori. In caz de succes returneaza TRUE, altfel, FALSE.

24.count(array)

- Returneaza numarul de elemente din "array"

25.current(array);

- Returneaza valoarea elementului curent din "array", daca nu are elemente returneaza FALSE. Fiecare matrice are un pointer intern, care prima data e setat la primul element.

26.end(array);

- Muta pointerul din "array" la ultimul element si returneaza valoarea lui, daca nu are elemente returneaza FALSE.

27.implode("separator", array);

- Transforma "array" in sir (string), adaugand valorile intr-un sir, separate de ceea ce e specificat la "separator".

28.in_array("val", array)

- Verifica daca o matrice contine o anumita valoare. Returneaza TRUE daca vreunul din elementele din "array" are valoarea "val", in caz contrar FALSE.

29.key(array)

- Returneaza cheia elementului curent la care se afla pointerul

30.krsort(array)

- Sorteaza un "array" dupa chei ordonate in ordine inversa alfabetica, pastrand corelatia dintre chei si valori. In caz de succes returneaza TRUE, altfel, FALSE.

31.ksort(array)

- Sorteaza un "array" dupa chei ordonate in ordine alfabetica, pastrand corelatia dintre chei si valori. In caz de succes returneaza TRUE, altfel, FALSE.

32.natcasesort(array)

- Sorteaza un "array" in ordine alfabetica a valorilor, case-insensitive, folosind ordinea naturala a numerelor. In caz de succes returneaza TRUE, altfel, FALSE.

33.next(array);

- Avanseaza pointerul din "array" inainte cu un element si returneaza valoarea, daca nu are elemente returneaza FALSE.

34.prev(array);

- Muta pointerul din "array" inapoi cu un element si returneaza valoarea, daca nu are elemente returneaza FALSE.

35.reset(array);

- Readuce pointerul din "array" la primul element si returneaza valoarea lui, daca nu are elemente returneaza FALSE.

36.rsort(array)

- Sorteaza un "array" in ordine inversa alfabetica a valorilor (literele mici inaintea celor mari), stergand cheile initiale, devenind un Array asociativ cu chei de la 0. In caz de succes returneaza TRUE, altfel, FALSE.

37.sort(array)

- Sorteaza un "array" in ordine alfabetica a valorilor (literele mari inaintea celor mici), stergand cheile initiale, devenind un Array asociativ cu chei de la 0. In caz de succes returneaza TRUE, altfel, FALSE.

38. **shuffle(array)**

- Amesteca aleator elementele din "array". Sterge cheile initiale, devenind un Array asociativ cu chei de la 0. In caz de succes returneaza TRUE, altfel, FALSE.

Functii matematice

Aici gasiti functii PHP matematice, in special pentru lucrul cu numere.

1. **abs(nr)**

- Returneaza valoarea absoluta (*cu +*) a lui "nr".

2. **base_convert(nr, baza1, baza2)**

- Converteste un numar, "nr", din baza1 in baza2. Rezultatul returnat este un sir cu "nr" convertit in baza2.

3. **bindec('sir_binar')**

- Returneaza echivalentul numeric (decimal) al sirului binar din parametrul "sir_binar".

4. **ceil(nr)**

- Returneaza valoarea rotunjita a lui "nr" la urmatorul (mai mare) intreg.

5. **cos(radian)**

- Returneaza valoarea cosinus a gradelor de cerc specificate la "radian".

6. **decbin(nr)**

- Returneaza un sir cu valoarea binara a numarului "nr".

7. **dechex(nr)**

- Returneaza un sir cu valoarea hexazecimala a numarului "nr".

8. **decoct(nr)**

- Returneaza un sir cu valoarea octala a numarului "nr".

9. **floor(nr)**

- Returneaza valoarea rotunjita a lui "nr" la anteriorul (mai mic) intreg.

10. **fmod(x, y)**

- Returneaza valoarea ramasa a lui "x" impartit la "y", care nu mai e multiplu de "y".

11. **hexdec('hex')**

- Returneaza valoarea zecimala a sirului hexazecimal "hex".

12. **hypot(c1, c2)**

- Returneaza lungimea ipotenozei unui triunghi dreptunghic care are lungimea catetelor "c1" si "c2".

13. **is_infinite(val)**

- Returneaza TRUE daca valoarea lui "val" este un numar infinit (*precum $\log(0)$*), altfel, returneaza FALSE.

14. **lcg_value()**

- Returneaza un numar aleator intre 0 si 1.

15. **log10(nr)**

- Returneaza logaritmul in baza 10 a lui "nr"..

16. **log(nr, baza)**

- Returneaza logaritm de "nr" in baza "baza". Daca al doilea parametru nu e specificat, va returna logaritmul natural (in baza e).

17. **max(nr1, nr2, ...)**

- Returneaza cel mai mare numar dintre cele date la parametri.

18. **min(nr1, nr2, ...)**

- Returneaza cel mai mic numar dintre cele date la parametri.

19.**mt_rand(min, max)**

- Returneaza un numar aleator cuprins intre numerele "min" si "max". Similara este si functia **rand()**, dar "mt_rand()" este de 4 ori mai rapida.

20.**octdec('oct')**

- Returneaza valoarea zecimala a sirului octal "oct".

21.**pi()**

- Returneaza valoarea aproximativa a lui PI. La fel ca si constanta **M_PI**.

22.**pow(nr, putere)**

- Returneaza valoarea obtinuta din "nr" la puterea "putere".

23.**round(nr)**

- Returneaza valoarea rotunjita a lui numar la intregul ce mai apropiat.

24.**sin(radian)**

- Returneaza valoarea sinus a gradelor de cerc specificate la "radian".

25.**sqrt(nr)**

- Returneaza radical din "nr".

26.**tan(radian)**

- Returneaza valoarea tangenta a gradelor de cerc specificate la "radian".

27.**is_numeric(val)**

- Returneaza TRUE daca "val" e o valoare numerica, altfel, FALSE.

Funcții pentru lucru cu functii

Aici gasiti functii PHP ce pot fi folosite in lucrul cu functii create /definite in script.

1.**call_user_func("func", arg)**

- Apeleaza functia "func" dandu-i valoarea argumentului "arg". Returneaza rezultatul functiei apelate, sau FALSE.

2.**call_user_func_array("func", array_arg)**

- Apeleaza o functie ("func") dandu-i valoarea argumentelor dintr-un Array (array_arg). Returneaza rezultatul functiei apelate, sau FALSE.

3.**create_function("argumente", "cod_functie")**

- Creaza dinamic o functie ce va avea argumentele "argumente" si codul ce-l executa "cod_functie". Returneaza functia creata (*poate fi asociata cu un nume de variabila*), sau FALSE. ("*argumente*" si "*cod_functie*" trebuie sa fie siruri)

4.**func_num_args()**

- Returneaza numarul de argumente primite de o functie. Se foloseste in interiorul acelei functii.

5.**func_get_arg(nr)**

- Returneaza argumentul cu index-ul "nr" din cele primite de o functie (*primul argument are index 0*). Se foloseste in interiorul acelei functii. Daca numarul de argumente e mai mic decat "nr", returneaza FALSE.

6.**func_get_args()**

- Returneaza o matrice cu argumentele primite de o functie. Se foloseste in interiorul acelei functii..

7.**function_exists("function")**

- Returneaza TRUE daca functia "function" (*numele ei intre ghilimele*) exista, altfel, returneaza FALSE.

8.**register_shutdown_function("func", param1, param2, ...)**

- Inregistreaza functia "func" sa fie executata la terminarea scriptului (dupa executarea comenzilor din el), sau

dupa apelarea lui "exit()". "param1", "param2" sunt optionali si reprezinta parametri ce vor fi transferati la "func".

Functii pentru siruri (strings)

Aici gasiti cateva din cele mai utile functii PHP pentru siruri (strings).

1.addslashes("sir")

- Aadauga caracterul \ (backslash) inaintea ghilimelelor simple si duble, a lui backslash si NULL (NULL byte). Ca protectie, de exemplu cand sirul e adaugat intr-o baza de date.

2.bin2hex('sir')

- Returneaza forma hexadecimale a lui 'sir'

3.chr(ascii)

- Returneaza caracterul reprezentat de codul ASCII din parametru.

4.chunk_split("sir", nr, 'sep')

- Imparte "sir" in mai multe sub-siruri, avand lungimea de caractere "nr" si separate prin "sep".

5.echo 'sir'

- Afiseaza in browser continutul 'sir'.

6.explode('sep', "sir")

- Creaza un Array (matrice) cu partile din "sir" separate prin "sep". Fiecare sub-sir delimitat de "sep" devine un element in matrice. *Este inversul lui "implode()".*

7.html_entity_decode("sir")

- Decodeaza entitatile html din "sir". E opusul lui "htmlentities()"

8.htmlentities("sir")

- Converteste toate caracterele specifice din sir in entitati HTML.

9.htmlspecialchars_decode("sir")

- Converteste entitatile HTML, ce pot fi codificate cu "htmlspecialchars()", inapoi la forma de caracter.

10.htmlspecialchars("sir")

- Converteste doar caracterele ghilimele, & < si > din "sir" in entitati HTML.

11.md5("sir")

- Codifica sirul cu algoritmul MD5, returnand un sir codat cu 32 de caractere.

12.rtrim("sir", 'characters')

- Sterge din partea dreapta a sirului caracterele specificate la "characters", care poate fi un singur caracter sau un sir. Daca al doilea parametru nu e specificat, va sterge spatiile libere din dreapta.

13.sha1("sir")

- Similar cu "md5()", aceasta functie codifica sirul cu algoritmul SHA1 (*US Secure Hash Algorithm 1*), returnand un sir codat cu 40 de caractere.

14.str_ireplace("old", "new", "sir")

- Inlocuieste in "sir" caracterele specificate la "old" cu cele de la "new", fara a tine cont de majuscule sau litere mici. "old" si "new" pot fi si de tip Array.

14.str_replace("old", "new", "sir")

- Similar cu "str_ireplace()", inlocuieste in "sir" caracterele specificate la "old" cu cele de la "new", dar este case-sensitive (tine cont de majuscule si litere mici). "old" si "new" pot fi si de tip Array.

15.str_repeat("sir", nr)

- Returneaza un sir in care parametrul "sir" e repetat de "nr" ori.

16.str_shuffle("sir")

- Amesteca aleator caracterele din "sir".

17.**str_split("sir", nr)**

- Transforma "sir" in Array, impartind pentru fiecare element sub-siruri cu lungimea de "nr" caractere. Daca "nr" nu e specificat, fiecare caracter din sir va deveni un element in matrice.

18.**str_word_count("sir")**

- Returneaza numarul de cuvinte din sir.

19.**strcasecmp("sir1", "sir2")**

- Compara cele doua siruri date ca parametri, fara diferenta intre majuscule si litere mici. Returneaza 0 daca sirurile sunt egale, < 0 daca "sir1" e mai mic decat "sir2" sau > 0 daca "sir1" e mai mare decat "sir2"

20.**strcmp("sir1", "sir2")**

- Similar cu "strcasecmp()", diferenta fiind ca face o comparatie ce tine cont de majuscule si litere mici.

21.**strcspn("sir", 'car')**

- Returneaza lungimea primului segment din "sir" ce nu contine nici un caracter specificat la 'car'

22.**strip_tags("sir")**

- Sterge tag-urile HTML si PHP din "sir".

23.**stripslashes("sir")**

- Returneaza "sir" fara caracterele backslashes adaugate. Inversul lui "addslashes()"

24.**stripos("sir", "sub-sir")**

- Returneaza pozitia de unde incepe "sub-sir" in "sir" (*incepand de la 0*), fara diferenta intre litere mici si majuscule. Daca "sub-sir" nu e gasit, returneaza FALSE.

25.**strpos("sir", "sub-sir")**

- Returneaza pozitia de unde incepe "sub-sir" in "sir" (*incepand de la 0*), case-sensitive, tinand cont de litere mici si majuscule. Daca "sub-sir" nu e gasit, returneaza FALSE.

26.**stristr("sir", 'car')**

- Returneaza tot sub-sirul din "sir", care incepe de la prima aparitie a lui 'car', fara diferenta intre majuscule si litere mici. Daca nu e gasit 'car', returneaza FALSE.

27.**strstr("sir", 'car')**

- Returneaza tot sub-sirul din "sir", care incepe de la prima aparitie a lui 'car', case-sensitive (cu deosebire intre majuscule si litere mici). Daca nu e gasit 'car', returneaza FALSE.

28.**strlen("sir")**

- Returneaza lungimea lui "sir" (numarul de caractere).

29.**strrchr("sir", "sub-sir")**

- Returneaza bucata ramasa din "sir" incepand de la ultimul caracter din "sub-sir" gasit in "sir". Functia e case-sensitive. Daca "sub-sir" nu se gaseste in "sir", returneaza FALSE.

30.**strrev("sir")**

- Inverseaza sir-ul.

31.**strtolower("sir")**

- Transforma toate literele din sir in litere mici.

32.**strtoupper("sir")**

- Transforma toate literele din sir in majuscule (litere mici).

33.**substr_count("sir", "sub_sir")**

- Returneaza numarul de cate ori "sub_sir" se gaseste in "sir". Functia este case-sensitive.

34.**substr("sir", start, lungime)**

- Returneaza portiunea din "sir" care incepe de la "start", cu lungimea de caractere "lungime". Daca parametrul "lungime" nu e specificat, se considera pana la sfarsitul sirului.

35. **trim("sir", 'del')**

- Sterge din marginile sirului caracter-ul (caracterele) specificate la "del". Daca al doilea parametru, "del", nu e adaugat, sterge spatiile libere din marginile sirului.

36. **ucfirst("sir")**

- Transforma primul caracter din "sir" in litera mare.

37. **ucwords("sir")**

- Transforma primul caracter al fiecarui cuvant din "sir" in litera mare.

38. **nl2br("sir")**

- Inlocuieste caracterele de rand nou (`\r\n` si `\n`) cu tagul HTML BR. *Functie utila cand sirul e preluat dintr-un textarea.*

39. **wordwrap("sir", lungime, "sep", true)**

- Imparte "sir" la numarul de caractere specificat de "lungime", separand prin "sep" partile astfel obtinute. Functie utila cand se doreste scurtarea liniilor sau cuvintelor prea lungi. Daca "true" nu e specificat, va imparti doar randurile, lasand cuvintele neafectate.

Funcții pentru Data si Timp

Aici sunt cateva din cele mai utile **functii PHP** pentru lucrul cu **Data** si formate de **Timp**.

1. **checkdate(nr_luna, nr_zi, nr_an)**

- Verifică validitatea datei formate din argumentii din paranteza. O dată este considerată validă dacă fiecare parametru este definit corespunzător. "nr_luna" este numarul lunii, intre 1 și 12 inclusiv, "nr_zi" este numarul zilei cuprinse în domeniul zilelor permise pentru luna respectivă. Anii bisecți sunt luați în considerare, "nr_an" este anul intre 1 și 32767 inclusiv. Returneaza TRUE sau FALSE.

2. **date_create('sir_data')**

- Stil obiect-orientat: **DateTime::__construct('sir_data')**

Creaza o instanta de timp dupa data din "sir_data", ce poate fi folosit in alte functii pt. data si timp care necesita ca parametru o instanta de timp. In caz de esec returneaza FALSE.

3. **date_format(obj_date, 'data_form')**

- Stil obiect-orientat: **DateTime::format('data_form')**

Returneaza data conform formatului din "data_form". "obj_date" e o instanta de timp returnata de functia "date_create()". In caz de esec returneaza FALSE.

4. **date_create_from_format('data_form', 'sir_data')**

- Stil obiect-orientat: **DateTime::createFromFormat(data_form, sir_data)**

Creaza o instanta de timp a datei din "sir_data" care e recunoscuta si trebuie sa corespunda cu formatul de data din "data_form". Instanta obtinuta poate fi folosita pentru a afisa data din "sir_data" cu un alt format, cu functia "date_format()". In caz de esec returneaza FALSE.

5. **date_date_set(obj_date_create, nr_an, nr_luna, nr_zi)**

- Stil obiect-orientat: **DateTime::setDate(nr_an, nr_luna, nr_zi)**

Seteaza o instanta de timp, dupa parametri primiti, care apoi poate fi afisata intr-un format de data, cu functia "date_format()". In caz de esec returneaza FALSE.

6. **date_default_timezone_set('zona_timp')**

- Seteaza timpul folosit in script dupa zona specificata in "zona_timp", identificator din [Lista zonelor orare sustinute](#). Returneaza TRUE sau FALSE.

7. **date_diff(obj_date1, obj_date2)**

- Stil obiect-orientat: **DateTime::diff(obj_date2)**

Returneaza un obiect cu diferenta dintre doua instante de timp sau FALSE.

8.**strong>**`date_modify(obj_date, 'sir_modificare'`

- Stil obiect-orientat: **`DateTime::modify(sir_modificare)`**

Modifica instanta de timp din "obj_date" cu valoarea specificata in "sir_modificare", sau FALSE in caz de esec.

9.**`date_parse_from_format('data_form', 'sir_data')`**

- Returneaza un Array asociativ cu informatii detaliate despre data din "sir_data", aceasta trebuind sa aibe formatul din "data_form".

10.**`date_parse('sir_data')`**

- Returneaza un Array asociativ cu informatii detaliate despre data din "sir_data" (care sa fie intr-un format acceptat de "strtotime()").

11.**`date_sun_info(timp_UNIX, Latitudine, Longitudine)`**

- Returneaza un Array asociativ cu informatii despre rasaritul si apusul soarelui, sfarsitul si inceputul crepusculului din ziua ce se gaseste in timpul UNIX din parametrul "timp_UNIX", din zona data de parametri "Latitudine" si "Longitudine", in grade.

12.**`date_time_set(obj_date, nr_oră, nr_minut, nr_secunde)`**

- Stil obiect-orientat: **`DateTime::setTime(nr_oră, nr_minut, nr_secunde)`**

Reseteaza instanta de timp (din "obj_date"), definindu-i si ora, minutul, si optional secndele, specificate in ceilalti parametri. In caz de esec, FALSE..

13.**`date_timestamp_get(obj_date)`**

- Stil obiect-orientat: **`DateTime::getTimestamp()`**

Obține timpul UNIX din instanta de timp "obj_date".

14.**`date_timestamp_set(obj_date, timp_UNIX)`**

- Stil obiect-orientat: **`DateTime::setTimestamp(timp_UNIX)`**

Modifica instanta de timp "obj_date" cu timpul UNIX din "timp_UNIX".

15.**`date('data_form', timp_UNIX)`**

- Returneaza un sir cu data, avand formatul din "data_form", al timpului UNIX din parametru "timp_UNIX". Daca "timp_UNIX" nu e mentionat, va returna data curenta.

16.**`getdate(timp_UNIX)`**

- Returneaza un Array asociativ conținînd informația despre dată și oră a parametrului "timp_UNIX", sau a datei/orei curente dacă parametrul "timp_UNIX" nu este furnizat.

17.**`localtime(timp_UNIX, bool)`**

- Returneaza un Array cu informatii despre ora locala (a serverului) la timpul specificat in parametrul "timp_UNIX" (sau timpul curent, daca acesta nu e adaugat).

"bool" poate fi true sau false, si e optional. Daca e specificat true, va returna un Array asociativ, in rest, un Array secvential.

18.**`strong>`**`microtime(bool)`

- Returneaza timpul curent Unix cu microsecunde. Dacă parametrul opțional "bool" este stabilit în TRUE, atunci este returnata o valoare **Float** (cu virgula, în secunde), altfel, returneaza sirul "**msec. sec.**", unde '**sec.**' este timpul curent măsurat în numărul de secunde de la Epoca Unix (0:00:00 January 1, 1970 GMT) și '**msec.**' este partea cu microsecunde.

19.**`mktime(ora, minut, secunde, luna, zi, an)`**

- Obține timpul Unix al argumentelor furnizate, care sunt numere întregi ce reprezinta: "ora", "minutul", "secundele", "luna", "ziua" si "anul" din data ce o formeaza. Dacă argumentele nu sunt valide, funcția întoarce FALSE.

20.strtotime('data_form')

- Returneaza timpul UNIX a aproape oricarei descrieri textuale în limba engleză a datei și orei specificate in sirul "data_form".

Dacă anul este specificat cu două cifre, valorile între 00-69 sunt interpretate ca 2000-2069, iar valorile între 70-99 ca 1970-1999.

21.time()

- Returneaza timpul curent, in format UNIX, si anume **numarul de secunde de la 1 Ianuarie 1970 ora 0 (1 Ianuarie 1970 00:00:00 GMT)**. Numarul returnat poate fi folosit cu functia "date()" pentru a obtine data intr-un format cunoscut sau parti din data (ziua, ora, luna, etc.).

Functii pentru fișiere și directoare

Aici gasiti cateva din cele mai utile functii PHP pentru lucru cu sistemul de fisiere si directoare.

1.basename(adresa, "sufix")

- Returneaza partea cu numele fisierului dintr-o adresa URL. "sufix" e optional, daca e specificat, functia elimina si pe acesta de la sfarsitul numelui.

2.chmod(file, mod)

- Modifica prmisunile CHMOD a lui "file" (*cale si nume fiser*) cu cele specificate la "mod" (0777, 0644, ...). Daca modificarea reuseste returneaza TRUE, in caz contrar, FALSE.

3.copy(sursa, destinatie)

- Copie fisierul specificat la "sursa", in locatia "destinatie" (aici poate fi specificata si denumirea fisierului la dstinatie). Aca fisierul deja exista, va fi inlocuit de cel copiat. Funtctia returneaza TRUE in caz de reusita sau FALSE.

4.dirname(adresa)

- Returneaza numele directorului din "adresa", fara numele fisierului sau alte extensii.

5.disk_free_space(cale_dir)

- Returneaza spatiul liber al sistemului in directorul specificat la "cale_dir".

6.disk_total_space(cale_dir)

- Returneaza spatiul total al sistemului in directorul specificat la "cale_dir". Returneaza numarul total de bytes sau FALSE daca functia esueaza.

7.fopen("fisier", "mod")

- Deschide un pointer la "fisier", in modul de lucru "mod". "mod" poate fi ("*b*" e *indicat sa fie adaugat pentru sistemele windows*):

r sau **rb** - pentru citire.

r+ sau **r+b** - citire si scriere.

w sau **wb** - creare si scriere fisier de la 0.

w+ sau **w+b** - citire si scriere fisier de la 0, il creaza daca nu exista.

a sau **ab** - scriere, adauga datele dupa cele deja existente. Creaza fisierul daca nu exista.

a+ sau **a+b** - citire si scriere, adauga datele dupa cele deja existente. Creaza fisierul daca nu exista

x sau **xb** - creare si scriere, daca fisierul exista deja, returneaza eroare.

x sau **x+b** - creare pentru citire si scriere, daca fisierul exista deja, returneaza eroare.

8.fclose(file_pointer)

- Inchide deschiderea unui fisier cu "fopen()" (*transmisa la "file_pointer"*), eliberand memoria folosita de acea deschidere. Returneaza TRUE daca functia reuseste, in caz contrar, FALSE.

9.feof(file_pointer)

- Testeaza sfarsitul unui fisier cu deschiderea specificata la "file_pointer". Functie utila cand fisierul e citit linie cu linie. Returneaza TRUE daca pointer-ul e la sfarsitul fisierului, altfel, returneaza FALSE.

10.fgets("pointer", lungime)

- Citeste numarul de caractere specificat la "lungime" dintr-un fisier deschis si specificat la "pointer". Citirea se face din locul unde a ajuns pointer-ul in fisier, daca "lungime" nu e specificat citirea se va face pana la sfarsitul randului curent (*daca lungimea liniei e mai mare de 8 KB, e indicata specificarea ei*).

11.file_exists("fisier")

- Returneaza TRUE daca "fisier" exista, altfel, returneaza FALSE.

12.file_get_contents("fisier")

- Returneaza tot continutul din "fisier" intr-un sir. Daca "fisier" nu poate fi citit, returneaza FALSE.

13.file_put_contents("fisier", "continut")

- Scrie in "fisier" sirul specificat la "continut". Daca fisierul exista, suprascrie datele existente, daca nu exista, il creaza. Daca nu poate scrie in "fisier", returneaza FALSE.

14.file("fisier", flag)

- Preia continutul fisierului intr-un array, fiecare linie din "fisier" va fi un element din matrice. "flag" e optional, in locul lui se poate adauga: **FILE_IGNORE_NEW_LINES** (nu va prelua si caracterele de linie noua), **FILE_SKIP_EMPTY_LINES** (va sari peste randurile goale).

15.fileatime("file")

- Returneaza timestamp-ul (timpul UNIX) cand a fost accesat ultima data "file", sau in caz de nereusita, FALSE.

16.filectime("file")

- Returneaza timestamp-ul (timpul UNIX) cand a fost modificat ultima data "file", sau in caz de nereusita, FALSE.

Similara cu aceasta e functia **filemtime()**

17.fileperms("file")

- Returneaza permisiunile CHMOD ale "file", sau FALSE.

18.filesize("file")

- Returneaza marimea lui "file", in bytes, sau FALSE.

19.flock(file_pointer, flag)

- Blocheaza / deblocheaza accesul de citire sau scriere la un fisier deschis cu "fopen()", specificat la "file_pointer". Returneaza TRUE, sau daca functia esueaza, FALSE. Modul de blocare, sau deblocarea, se specifica la "flag", acesta putand fi:

LOCK_SH- blocare acces citire.

LOCK_EX- blocare acces scriere.

LOCK_UN- deblocare

20.fseek(file_pointer, pozitie)

- Muta pozitia pointer-ului dintr-un fisier deschis cu "fopen()" la numarul de caractere specificat de "pozitie" (0 e primul caracter). In caz de succes returneaza 0, altfel, -1.

21.ftell(file_pointer)

- Returneaza pozitia curenta a indicatorului (pointer) dintr-un fisier deschis cu "fopen()" (specificat la "file_pointer"). In caz de esec returneaza FALSE.

22.fwrite(file_pointer, 'sir')

- Scrie intr-un fisier deschis cu "fopen()" (specificat la "file_pointer") continutul din "sir".

23.is_dir("nume")

- Returneaza TRUE daca "nume" exista si este un director, altfel returneaza FALSE.

24.is_file("nume")

- Returneaza TRUE daca "nume" exista si este un fisier, altfel returneaza FALSE.

25.is_readable("nume")

- Returneaza TRUE daca "nume" (fisier sau director) exista si poate fi citit, altfel returneaza FALSE.

26.is_uploaded_file("fisier_up")

- Returneaza TRUE daca "fisier_up" a fost incarcat pe server, prin upload si primit prin \$_POST, altfel returneaza FALSE. Parametrul functiei trebuie sa fie de forma "\$_FILES['userfile']['tmp_name']".

27.is_writable("nume")

- Returneaza TRUE daca "nume" (fisier sau director) exista si poate sa scrie in el, altfel returneaza FALSE.

28.mkdir("dir", chmod)

- Returneaza TRUE daca a putut crea directorul "dir" cu permisiunile specificate la "chmod", altfel returneaza FALSE.

29.pathinfo("adresa_file")

- Returneaza o matrice cu informatiile: ale "adresa_file".

30.readfile("fisier")

- Citeste si returneaza la browser (afiseaza) continutul din "fisier". In caz de esec returneaza FALSE.

31.realpath("cale_adresa")

- Returneaza calea reala in sistem a unei adrese din "cale_adresa", rezolvand referinte ca .. / ./ ../.

32.rename("nume", "nume_nou")

- Redenumeste un director sau fisier specificat la "nume", cu "nume_nou". In caz de succes returneaza TRUE, altfel FALSE.

33.rewind(file_pointer)

- Readuce pozitia pointer-ului dintr-un fisier deschis cu "fopen()" (specificat la "file_pointer") la inceputul fisierului (0, primul caracter). In caz de succes returneaza TRUE, altfel FALSE.

34.rmdir("dir")

- Sterge directorul specificat la "dir". Acesta trebuie sa fie gol. In caz de succes returneaza TRUE, altfel FALSE.

35.tempnam("dir", "prefix")

- Creaza un fisier cu nume unic, avand la inceput in nume sirul specificat la "prefix", in directorul "dir". Ii acorda CHMOD 0600. Daca directorul specificat nu exista, creaza fisierul in directorul "temp" al sistemului si returneaza numele fisierului. In caz de nereusita returneaza FALSE.

36.tmpfile()

- Creaza un fisier temporar, cu nume unic. Returneaza un pointer de fisier, ca si "fopen()", cu modul de deschidere "w+". In caz de nereusita returneaza FALSE. Cu "fclose()" fisierul temporar va fi sters.

37.unlink("fisier")

- Sterge fisierul specificat. In caz de succes returneaza TRUE, altfel FALSE.

Functii MySQL

Aici gasiti cateva din cele mai utilizate **functii PHP** pentru **MySQL**.

1.mysql_affected_rows(\$conn)

- Returneaza numarul de linii afectate in ultima operatie PHP-MySQL (INSERT, UPDATE, REPLACE ori DELETE) sau - 1 daca acel "query" esueaza. "\$conn" este optional si reprezinta conexiunea de lucru la MySQL.

Specificatie:*Cand se foloseste REPLACE, aceasta intai sterge inregistrările cu aceeasi cheie primara si dupa aia adauga noile inregistrari. Functia "mysql_affected_rows()" va returna numarul de inregistrari sterse plus cele adaugate.*

2.mysql_client_encoding(\$conn)

- Returneaza tipul de codare a formatului de caractere folosit in conexiunea curenta (\$conn).

3.mysql_connect('server_mysql', 'nume_utilizator', 'parola_mysql')

- Deschide o conexiune la MySQL. La argumentul "server_mysql" se poate adauga si portul de conectare, sintaxa devine astfel: 'server_mysql:port'.

4.mysql_close(\$conn)

- Inchide conexiunea la MySQL "\$conn", daca nu e precizat "\$conn", va inchide conexiunea curenta.

5.mysql_list_dbs(\$conn)

- Returneaza o lista cu bazele de date accesibile in conexiunea curenta (\$conn), altfel, FALSE.

6.mysql_errno(\$conn)

- Returneaza numarul erorii de la ultima functie MySQL, sau 0 (zero) daca nu e vreo eroare.

7.mysql_error(\$conn)

- Returneaza mesajul de eroare (sir text) dat de MySQL la ultima functie executata, sau un sir gol daca nu sunt erori.

8.mysql_query("Comanda_mysql", \$conn)

- Trimite o interogare SQL scrisa in "Comanda_mysql" la serverul MySQL. Returneaza un identificator in care e stocat raspunsul de la server, in functie de comanda SQL trimisa, sau in caz de esec (eroare), FALSE.

9.mysql_fetch_array(sql_rezult, tip)

- Returneaza un Array ce contine rezultatul dat de MySQL in urma unei interogari facute cu "mysql_query()" si transmis in "sql_rezult". In caz de esec, FALSE.

Daca "tip" are valoarea **MYSQL_ASSOC**, va returna un Array asociativ in care cheile sunt numele coloanelor din tabelul MySQL (ca si "mysql_fetch_assoc()"). Daca "tip" are valoarea **MYSQL_NUM** se va obtine o matrice secventiala, cu cheile numere consecutive (ca si "mysql_fetch_row()"). Iar daca "tip" are valoarea **MYSQL_BOTH** (care e default, daca tip nu e precizat) se va obtine un Array cu ambele tipuri.

10.mysql_fetch_assoc(sql_rezult)

- Returneaza un Array asociativ (in care cheile sunt numele coloanelor din tabelul MySQL) cu rezultatul dat de MySQL in urma unei interogari facute cu "mysql_query()" si continut in "sql_rezult". In caz de esec, FALSE.
- Muta si pointerul intern din tabelul MySQL deasupra.

11.mysql_fetch_row(sql_rezult)

- Returneaza un Array secvential (in care cheile sunt numere consecutive, de la 0) cu rezultatul dat de MySQL in urma unei interogari facute cu "mysql_query()" si continut in "sql_rezult". In caz de esec, FALSE.
- Muta si pointerul intern din tabelul MySQL deasupra.

12.mysql_fetch_object(sql_rezult)

- Returneaza un obiect cu rezultatul dat de MySQL in urma unei interogari facute cu "mysql_query()" si continut in "sql_rezult". Proprietatile acestui obiect sunt numele coloanelor. In caz de esec, FALSE.
- Muta si pointerul intern din tabelul MySQL deasupra.

13.mysql_fetch_field(sql_rezult)

- Obtine un obiect cu o lista de informatii despre coloanele dintr-o interogare cu "mysql_query()".

14.mysql_field_len(sql_rezult, nr_camp)

- Returneaza lungimea campului de la "nr_camp", acest parametru este numarul de ordine al coloanelor dintr-o interogare cu "mysql_query()" (a carui raspuns e continut in "sql_rezult"). Prima coloana are numarul 0. In caz de esec, FALSE.

15.mysql_field_name(sql_rezult, nr_camp)

- Obtine numele campului de la "nr_camp", acest parametru este numarul de ordine al coloanelor dintr-o interogare cu "mysql_query()" (a carui raspuns e continut in "sql_rezult"). Prima coloana are numarul 0. In caz de

`esec, FALSE.`

16. `mysql_field_type(sql_rezult, nr_camp)`

- Obține tipul campului de la "nr_camp" ("string", "int", "blob", sau altele ...), acest parametru este numărul de ordine al coloanelor dintr-o interogare cu "mysql_query()" (a cărui răspuns e conținut în "sql_rezult"). Prima coloană are numărul 0.

17. `mysql_free_result(sql_rezult)`

- Eliberează memoria serverului după prelucrarea informațiilor obținute cu o interogare "mysql_query()" (stocată în "sql_rezult").

18. `mysql_get_client_info()`

- Returnează un șir cu date despre versiunea MySQL activă pe server. Nu necesită conectare la MySQL.

19. `mysql_get_host_info($conn)`

- Da informații despre gazda serverului MySQL din conexiunea \$conn.

20. `mysql_insert_id($conn)`

- Obține ultimul ID generat de o coloană "AUTO_INCREMENT" în ultima interogare.

Prelucrarea datei și orei

Limbaajul PHP oferă soluții simple pentru prelucrarea datei și orei. În versiunile mai noi sunt introduse facilități avansate cum ar fi DST (Daylight Saving Time), Timezones, Date Intervals, s.a. Înainte de acestea însă lucrul cu timpul se realiza folosind Unix timestamp. Conceptul de Unix timestamp desemnează numărul de secunde scurse de la 1 ianuarie 1970 (când se considera că a început era Unix).

Fiind vorba de un număr pozitiv ce are mereu aceeași referință, este ușor să se realizeze operații cum ar fi scăderea, adunarea sau compararea a două date reprezentate printr-un Unix timestamp. Mai jos sunt câteva aplicații menite să evidențieze cât de ușor se pot manipula date și ore în PHP.

```

1
2  <?php
3  // data curenta
4  print date( 'd.m.Y H.i.s' )."<br>";
5
6  // data in format Unix (UNIX timestamp)
7  print time()."<br>";
8
9  // alegerea unui fus orar
10 date_default_timezone_set( 'Europe/Bucharest' );
11
12 // parsarea unei date
13 print strtotime("now")."<br>";
14 print strtotime('1st January 2004');
15 ?>

```

```

1
2  <?php
3  // data curenta
4  print date( 'd.m.Y H.i.s' )."<br>";
5
6  // data in format Unix (UNIX timestamp)
7  print time()."<br>";
8
9  // alegerea unui fus orar
10 date_default_timezone_set( 'Europe/Bucharest' );
11
12 // parsarea unei date
13 print strtotime("now")."<br>";
14 print strtotime('1st January 2004');
15 ?>

```

Cum se formatează data sau ora?

Formatarea datei sau a orei se poate realiza ușor folosind funcția `date`. Cu ajutorul acesteia se pot "extrage" doar acele părți din data care sunt necesare și se poate specifica formatul dorit. Spre exemplu, folosind funcția **`date`** putem afișa data în format lung (de genul: "marti, 30 septembrie 2017") sau scurt (30.09.2017) sau cu aceeași funcție putem afișa ora sau oricare combinație a celor două. Formatul se specifică print-un text format din caractere ce au o semnificație anume, transmis ca parametru funcției **`date`**. Pentru detalii complete despre fiecare caracter în parte accesați <http://www.php.net/manual/ro/function.date.php>.

```

1  <?php
2  # se preia data/ora curenta
3  $acum = time();
4
5
6  # formatarea se realizeaza cu functia date
7  # daca nu se specifica altfel, functia date va formata data/ora curenta
8  echo date("H:i:s") . "<br>";
9  echo date("Y-m-d") . "<br>";
10 echo date("F j, Y, g:i a") . "<br>";
11 echo date("l") . "<br>";
12
13 # se poate specifica un al doilea parametru ce reprezinta data/ora care se formateaza
14 echo date("d-m-Y H:i:s", $acum) . "<br>";
15 echo date("D, M d, Y", 1072915200) . "<br>";
16
17 # combinat cu mktime, se pot obtine informatii despre zile din trecut sau viitor
18 # de exemplu, ce zi a fost pe 4 iulie 2000
19 $iulie4 = mktime( 0, 0, 0, 7, 4, 2000 ); # aflu timestamp-ul
20 echo date( 'l', $iulie4 ); # formatez acel timestamp - extrag doar ziua
21 // afiseaza Tuesday
22
23 ?>

```

```

1  <?php
2  # se preia data/ora curenta
3  $acum = time();
4
5
6  # formatarea se realizeaza cu functia date
7  # daca nu se specifica altfel, functia date va formata data/ora curenta
8  echo date("H:i:s") . "<br>";
9  echo date("Y-m-d") . "<br>";
10 echo date("F j, Y, g:i a") . "<br>";
11 echo date("l") . "<br>";
12
13 # se poate specifica un al doilea parametru ce reprezinta data/ora care se formateaza
14 echo date("d-m-Y H:i:s", $acum) . "<br>";
15 echo date("D, M d, Y", 1072915200) . "<br>";
16
17 # combinat cu mktime, se pot obtine informatii despre zile din trecut sau viitor
18 # de exemplu, ce zi a fost pe 4 iulie 2000
19 $iulie4 = mktime( 0, 0, 0, 7, 4, 2000 ); # aflu timestamp-ul
20 echo date( 'l', $iulie4 ); # formatez acel timestamp - extrag doar ziua
21 // afiseaza Tuesday
22
23 ?>

```

Cum determin ce zi și oră va fi peste 480 ore?

Pentru a efectua ușor operații cu zile și ore, se va folosi formatul de dată Unix (UNIX timestamp), returnat de funcția **`time()`**. Practic, data este afișată ca numărul de secunde scurs de la 1 ianuarie 1970, ora 00:00. Având această nouă perspectivă (de a privi datele ca un număr de secunde), operațiile cu date (de genul 'cât va fi peste

2 zile', 'ce oră a fost acum xxx minute', etc.) devin extrem de simple luând forma unor simple scăderi și adunări de secunde.

```

1
2 <?php
3 // cerinta este: "cat va fi ora peste 480 de ore, adica peste (480 * 60 *60) secunde
4
5 $acum = time();
6 $maiTarziu = 480 /*ore*/ * 60 /*minute*/ * 60 /*secunde*/;
7
8 print "Data ceruta este: " . date( 'd.m.Y H.i.s', $acum + $maiTarziu );
9 // Afiseaza Data ceruta
?>

```

```

1
2 <?php
3 // cerinta este: "cat va fi ora peste 480 de ore, adica peste (480 * 60 *60) secunde
4
5 $acum = time();
6 $maiTarziu = 480 /*ore*/ * 60 /*minute*/ * 60 /*secunde*/;
7
8 print "Data ceruta este: " . date( 'd.m.Y H.i.s', $acum + $maiTarziu );
9 // Afiseaza Data ceruta
?>

```

Cum afișez luna curentă și toate cele 12 luni în limba română?

Nota: această secvență de cod depinde de configurările serverului pe care este executată. Limba română trebuie instalată pe server pentru a avea rezultatele scontate.

```

1
2 <?php
3 setlocale( LC_TIME , 'ROM_ROM' );
4
5 // afiseaza numele celor 12 luni
6 for( $m = 1; $m <= 12; $m++ ) {
7     print strftime( "%B", mktime( 0,0,0, $m, 1, 2008 ) ) . "<br>";
8 }
9
10 // afiseaza luna curenta
11 print 'Luna curenta este: ' . strftime( "%B", time() );
?>

```

```

1
2 <?php
3 setlocale( LC_TIME , 'ROM_ROM' );
4
5 // afiseaza numele celor 12 luni
6 for( $m = 1; $m <= 12; $m++ ) {
7     print strftime( "%B", mktime( 0,0,0, $m, 1, 2008 ) ) . "<br>";
8 }
9
10 // afiseaza luna curenta
11 print 'Luna curenta este: ' . strftime( "%B", time() );
?>

```

Rezultatul este posibil să nu fie în limba română!

Clase și obiecte în PHP

O clasă este o colecție de variabile și funcții care operează asupra variabilelor respective. Sintaxa folosită pentru declararea unei clase în PHP este:

Cod:

```

1
2 <?php
3 class nume_clasa
4 {
5     //date membre
6     var nume_variabila_1
7     ...
8     var nume_variabila_m
9     //metode
10    function nume_functie_1(parametri)
11    {
12        ...//definitia functiei
13    }
14    ...
15    function nume_functie_n (parametri)
16    {
17        ...//definirea functiei
18    }
19 }
20 ?>

```

```

1
2 <?php
3 class nume_clasa
4 {
5     //date membre
6     var nume_variabila_1
7     ...
8     var nume_variabila_m
9     //metode
10    function nume_functie_1(parametri)
11    {
12        ...//definitia functiei
13    }
14    ...
15    function nume_functie_n (parametri)
16    {
17        ...//definirea functiei
18    }
19 }
20 ?>

```

Pentru numele unei clase poate fi utilizat orice identificador permis în PHP cu o singură excepție: ***sdtclass***; acest identificador este folosit de PHP în scopuri interne.

În PHP funcțiile ale căror identicatori încep cu "__"(două caractere '_') sunt considerate funcții magice și utilizarea acestora nu este recomandată.

În PHP, datele membre nu pot fi inițializate decât cu valori constante. Pentru a inițializa variabilele cu valori care nu sunt constante trebuie folosit un constructor. Mai jos aveți un exemplu de clasă în care inițializările ***nu sunt corecte***:

Cod:

```

1
2 <?php
3 class Nepermis
4 {
5     var $data = date("Y-m-d");
6     var $nume = $prenume;
7     var $dest = 'Mihai'. 'Claudiu';
8     var $obiecte = array("minge", "pantof");
9 }
10 ?>

```

```

1
2 <?php
3 class Nepermis
4 {
5     var $data = date("Y-m-d");
6     var $nume = $prenume;
7     var $dest = 'Mihai'. 'Claudiu';
8     var $obiecte = array("minge", "pantof");
9 }
10 ?>

```

Obiecte

În PHP clasele sunt considerate a fi tipuri de date; ele pot fi privite ca fiind "ampretele" variabilelor propriu-zise. Pentru a crea o variabilă al cărei tip este o clasă trebuie utilizat operatorul **new**. În continuare vom defini o clasă **Aritmetica** cu două date membre x și y care sunt numere întregi și două metode care realizează adunarea, respectiv înmulțirea lor.

Cod:

```

1
2 <?php
3 class Aritmetica
4 {
5     var $x=2;
6     var $y=3;
7     function Suma()
8     {
9         return $this -> x+$this->y;
10    }
11    function Produs()
12    {
13        return $this ->x*$this->y;
14    }
15 }
16 ?>

```

```

1
2 <?php
3 class Aritmetica
4 {
5     var $x=2;
6     var $y=3;
7     function Suma()
8     {
9         return $this -> x+$this->y;
10    }
11    function Produs()
12    {
13        return $this ->x*$this->y;
14    }
15 }
16 ?>

```

Pentru a crea un obiect de tipul **Aritmetica** vom utiliza o instrucțiune de tipul:

```

1
2 $aritm = new Aritmetica;

1
2 $aritm = new Aritmetica;

```

Acum putem utiliza metodele clasei; pentru a afișa suma sau produsul celor două numere vom putea apela cele două metode astfel:

```
1
2 echo $aritm -> Suma ( );
  echo $aritm -> Produs ( );

1
2 echo $aritm -> Suma ( );
  echo $aritm -> Produs ( );
```

Vom obține rezultatele 5, respectiv 6. Valorile datelor membre pot fi și ele modificate prin instrucțiuni de tipul:

```
1
2 $aritm -> x = 5;
  $aritm -> y = 4;

1
2 $aritm -> x = 5;
  $aritm -> y = 4;
```

Dacă, în urma modificării apelam din nou metodele **Suma()** și **Produs()**, rezultatele vor fi 9, respectiv 20.

Cod:

```
1
2 <?php
3 class PHP4
4 {
5     var $salut="Salut PHP4!";
6     function Salut()
7     {
8         return $this->salut;
9     }
10 }
11 $salutare =new PHP4;
12 echo $salutare ->Salut(). "<br>";
  ?>

1
2 <?php
3 class PHP4
4 {
5     var $salut="Salut PHP4!";
6     function Salut()
7     {
8         return $this->salut;
9     }
10 }
11 $salutare =new PHP4;
12 echo $salutare ->Salut(). "<br>";
  ?>
```

Rezultatul codului de mai sus este:

```
1
  Salut PHP4!

1
  Salut PHP4!
```

În acest exemplu a fost utilizată pseudo-variabila **\$this**. Aceasta este folosită pentru a indica faptul că se operează asupra unei date membre a obiectului curent.

Extinderea claselor

Deseori este necesară definirea unor clase cu proprietăți (date membre) și metode asemănătoare. Pentru a usura definirea unor astfel de clase a fost introdus conceptul de **extindere** (derivare) a claselor.

O clasă derivată va păstra toate proprietățile și metodele clasei pe care o extinde și poate conține diferite proprietăți și metode noi. Nu există nici o posibilitate de a elimina din clasa derivată anumite proprietăți sau metode ale clasei de bază. O anumită clasă poate avea o singură clasă **părinte**; așadar, în PHP nu este permisă **moștenirea** multiplă. Pentru a extinde o anumită clasă se utilizează cuvântul cheie **extends**.

În următorul exemplu voi extinde clasa *Aritmetica*; o să adaug încă o variabilă și o să creez două noi funcții: una pentru calculul sumei celor trei variabile și una pentru calcularea produsului lor:

Cod:

```
1
2 <?php
3 class Aritmetica3 extends Aritmetica
4 {
5     var z=4;
6     function Suma3()
7     {
8         return $this->x+$this->y+$this->z;
9     }
10    function Produs3()
11    {
12        return $this->x*$this->y*$this->z;
13    }
14    }
15    ?>
```

```
1
2 <?php
3 class Aritmetica3 extends Aritmetica
4 {
5     var z=4;
6     function Suma3()
7     {
8         return $this->x+$this->y+$this->z;
9     }
10    function Produs3()
11    {
12        return $this->x*$this->y*$this->z;
13    }
14    }
15    ?>
```

Dacă definim un obiect prin intermediul unei instrucțiuni de genul:

```
1
2 $aritm3 = new Aritmetica3;

1
2 $aritm3 = new Aritmetica3;
```

Atunci pentru acest obiect vom putea utiliza atât metodele definite în cadrul clasei *Aritmetica3*: *Suma3()* și *Produs3()*, cât și metodele definite în cadrul clasei de bază *Aritmetica*: *Suma()* și *Produs()*.

În continuare aveți un exemplu care ilustrează modul în care pot fi create și utilizate clasele derivate.

Cod:

```

1  <?php
2  class Aritmetica
3  {
4  {
5  var $x=2;
6  var $y=3;
7  function Suma()
8  {
9  return $this->x+$this->y;
10 }
11 function Produs()
12 {
13 return $this->x*$this->y;
14 }
15 }
16 class Aritmetica3 extends Aritmetica
17 {
18 var $z=4;
19 function Suma3()
20 {
21 return $this->x+$this->y+$this->z;
22 }
23 function Produs3()
24 {
25 return $this->x*$this->y*$this->z;
26 }
27 }
28 $aritm3 = new Aritmetica3;
29 echo "<b>Inainte de modificare.</b>";
30 echo "<br>";
31 echo "Suma primelor doua numere: ";
32 echo $aritm3->Suma()."<br>";
33 echo "Produsul primelor doua numere: ";
34 echo $aritm3->Produs()."<br>";
35 echo "Suma celor trei numere:";
36 echo $aritm3->Suma()."<br>";
37 echo "Produsul celor trei numere:";
38 echo $aritm3->Produs()."<br>";
39 $aritm3->x=5;
40 $aritm3->y=4;
41 $aritm3->z=3;
42 echo "<br><br>";
43 echo "<b>Dupa modificare.</b><br>";
44 echo "<br>";
45 echo "Suma primelor doua numere:";
46 echo $aritm3->Suma3()."<br>";
47 echo "Produsul primelor doua numere:";
48 echo $aritm3->Produs3()."<br>";
49 echo "Suma celor trei numere:";
50 echo $aritm3->Suma3()."<br>";
51 echo "Produsul celor trei numere:";
52 echo $aritm3->Produs3()."<br>";
53 ?>

```

```

1  <?php
2  class Aritmetica
3  {
4  {
5  var $x=2;
6  var $y=3;
7  function Suma()
8  {
9  return $this->x+$this->y;
10 }
11 function Produs()
12 {
13 return $this->x*$this->y;
14 }
15 }
16 class Aritmetica3 extends Aritmetica

```

```

17 {
18 var $z=4;
19 function Suma3()
20 {
21 return $this->x+$this->y+$this->z;
22 }
23 function Produs3()
24 {
25 return $this->x*$this->y*$this->z;
26 }
27 }
28 $aritm3 = new Aritmetica3;
29 echo "<b>Inainte de modificare.</b>";
30 echo "<br>";
31 echo "Suma primelor doua numere: ";
32 echo $aritm3->Suma()."<br>";
33 echo "Produsul primelor doua numere: ";
34 echo $aritm3->Produs()."<br>";
35 echo "Suma celor trei numere:";
36 echo $aritm3->Suma()."<br>";
37 echo "Produsul celor trei numere:";
38 echo $aritm3->Produs()."<br>";
39 $aritm3->x=5;
40 $aritm3->y=4;
41 $aritm3->z=3;
42 echo "<br><br>";
43 echo "<b>Dupa modificare.</b><br>";
44 echo "<br>";
45 echo "Suma primelor doua numere:";
46 echo $aritm3->Suma3()."<br>";
47 echo "Produsul primelor doua numere:";
48 echo $aritm3->Produs3()."<br>";
49 echo "Suma celor trei numere: ";
50 echo $aritm3->Suma3()."<br>";
51 echo "Produsul celor trei numere:";
52 echo $aritm3->Produs3()."<br>";
?>

```

Rezultatul codului de mai sus este:

Inainte de modificare.

Suma primelor doua numere: 5

Produsul primelor doua numere: 6

Suma celor trei numere: 5

Produsul celor trei numere: 6

Dupa modificare.

Suma primelor doua numere: 12

Produsul primelor doua numere: 60

Suma celor trei numere: 12

Produsul celor trei numere: 60

În PHP clasele trebuie definite înaintea utilizării lor; aşadar clasa părinte va fi definită întotdeauna înaintea clasei fiu.

Constructorii

Un constructor este o metodă (funcție) a unei clase care este apelată automat în momentul în care este creată o nouă instanță a clasei (cu ajutorul operatorului **new**). În PHP, este considerată ca fiind un constructor, orice funcție care are același nume cu clasa în interiorul căreia este definită.

Constructorii pot fi folosiți pentru inițializarea datelor membre cu valori care nu sunt constante. Ei pot avea argumente, iar acestea pot fi optionale. Pentru a putea utiliza clasa fără a specifica nici un parametru în

momentul creării unui obiect, se recomandă stabilirea unor valori implicite pentru toate argumentele constructorului. În cazul în care nu este definit un constructor pentru o anumită clasă, se utilizează constructorul clasei de bază, dacă aceasta există. De exemplu, pentru următoarea secvență de cod, în momentul creării obiectului corespunzător variabilei **\$b**, va fi apelat constructorul clasei **A**.

Cod:

```

1  <?php
2  class A
3  {
4      function A()
5      {
6          echo "Constructorul clasei A<br>";
7      }
8      function B()
9      {
10         echo "O functie obisnuita a clasei A.<br>";
11     }
12 }
13
14 class B extends A
15 {
16     function C()
17     {
18         echo "O functie obisnuita a clasei B.<br>";
19     }
20 }
?>

```

```

1  <?php
2  class A
3  {
4      function A()
5      {
6          echo "Constructorul clasei A<br>";
7      }
8      function B()
9      {
10         echo "O functie obisnuita a clasei A.<br>";
11     }
12 }
13
14 class B extends A
15 {
16     function C()
17     {
18         echo "O functie obisnuita a clasei B.<br>";
19     }
20 }
?>

```

În PHP apelul constructorului clasei de bază trebuie să fie explicit dacă este necesară executarea operațiilor corespunzătoare. În majoritatea limbajelor de programare există funcții speciale numite destructori care sunt apelate automat în momentul "distrugerii" unui obiect. În PHP nu există destructori.

Operatorul ::

Uneori este utilă folosirea unor metode sau variabile ale clasei de bază sau ale unei clase care nu a fost instanțiată încă. În acest scop a fost introdus operatorul ::

Pentru a descrie modul de utilizare al acestui operator voi prezenta mai întâi un exemplu:

Cod:

```

1

```

```

2  <?php
3  class A
4  {
5  function exemplu()
6  {
7  echo "Functia clasei de baza.<br>";
8  }
9  }
10 class B extends A
11 {
12 function exemplu()
13 {
14 echo "Functia redefinita<br>\n";
15 A::exemplu();
16 }
17 }
18 A::exemplu();
19 $b=new B;
20 $b->exemplu();
21 $b=new B;
    ?>

```

```

1  <?php
2  class A
3  {
4  {
5  function exemplu()
6  {
7  echo "Functia clasei de baza.<br>";
8  }
9  }
10 class B extends A
11 {
12 function exemplu()
13 {
14 echo "Functia redefinita<br>\n";
15 A::exemplu();
16 }
17 }
18 A::exemplu();
19 $b=new B;
20 $b->exemplu();
21 $b=new B;
    ?>

```

Prin intermediul instrucțiunii

```

1  A :: exemplu( );

```

```

1  A :: exemplu( );

```

este apelată metoda *exemplu()* a clasei A, așadar se afișează mesajul 'Functia clasei de baza' cu toate că nu există nici un obiect care este o instanță a acestei clase, deci nu putem scrie o instrucțiune de tipul *\$a -> exemplu()*;

În schimb apelăm metoda:

```

1  $b -> exemplu( );

```

```

1  $b -> exemplu( );

```

ca "o funcție a clasei" și nu ca "o funcție a unui obiect". Putem avea funcții ale claselor, dar nu putem avea variabile ale claselor. De fapt, în momentul unui astfel de apel nu se creează nici un obiect care este instanță a clasei respective. Ca urmare, o funcție a unei clase nu poate opera asupra unor proprietăți ale clasei, dar poate utiliza variabile locale sau globale. În plus, o astfel de funcție nu poate utiliza pseudo-variabila **\$this**.

În exemplul anterior, în cadrul clasei **B** este redefinită funcția *exemplu()*. Așadar, definiția "originală" (din cadrul clasei **A**) nu poate fi accesată în interiorul clasei B decât dacă ne referim la ea explicit prin intermediul operatorului `::`.

Accesarea clasei de baza

În exemplul anterior am utilizat o funcție a clasei de bază. În locul utilizării denumirii clasei de baza poate fi folosită denumirea specială **parent** care este o referință la clasa de bază definită în cadrul construcției *extends*. Folosirea denumirii speciale este utilă în cazul în care ierarhia de clase se modifică. În acest caz este suficientă o singură modificare în cadrul construcției *extends*, fără a mai fi necesare modificări în interiorul clasei derivate. Așadar, definiția clasei **B** poate fi rescrisă astfel:

Cod:

```

1
2 <?php
3 class B extends A
4 {
5     function exemplu()
6     {
7         "Funcția redefinită<br>\n";
8         parent :: exemplu();
9     }
10 }
    ?>

```

```

1
2 <?php
3 class B extends A
4 {
5     function exemplu()
6     {
7         "Funcția redefinită<br>\n";
8         parent :: exemplu();
9     }
10 }
    ?>

```

Serializarea obiectelor

Prin serializare se înțelege crearea unui șir de octeți care conțin reprezentarea internă (binară) a variabilei respective. Așadar, serializarea permite "salvarea" valorilor unei variabile. Dacă este serializat un obiect sunt salvate doar proprietățile acestuia (variabilele membre) și numele clasei din care face parte, nu și metodele, deoarece funcțiile nu reprezintă valori.

Pentru a serializa un obiect este utilizată funcția **serialize()** care returnează șirul de octeți care conține reprezentarea binară.

Pentru a deserializa un obiect se folosește funcția pereche **unserialize()**.

Pentru ca o astfel de operație să funcționeze corect este necesară definirea clasei din care face parte obiectul respectiv. Funcția returnează valoarea variabilei serializate. În exemplul următor aveți prezentat modul în care poate fi serializat și deserializat un obiect. Șirul de octeți obținut în urma serializării va fi scris într-un fișier și va

fi citit din fișierul respectiv pentru efectuarea deserializării. De obicei serializarea și deserializarea sunt realizate în documente php diferite deoarece aeste operații nu au aproape nici o utilitate dacă sunt folosite în cadrul aceluiași document. Primul document în care se realizează serializarea trebuie să conțină o secvență asemănătoare cu următoarea:

Cod:

```

1
2 <?php
3 class A
4 {
5     var $msg ="Salutare lume";
6     function scrie()
7     {
8         echo $this->msg;
9     }
10 }
11 $a=new A;
12 $s=serialize($a);
13 //salvarea sirului intr-un fisier
14 $fp=fopen("fisier","w");
15 fputs($fp,$s);
16 fclose($fp);
17 ?>

```

```

1
2 <?php
3 class A
4 {
5     var $msg ="Salutare lume";
6     function scrie()
7     {
8         echo $this->msg;
9     }
10 }
11 $a=new A;
12 $s=serialize($a);
13 //salvarea sirului intr-un fisier
14 $fp=fopen("fisier","w");
15 fputs($fp,$s);
16 fclose($fp);
17 ?>

```

Pentru deserializare al doilea document va conține următoarea secvență:

Cod:

```

1
2 <?php
3 class A
4 {
5     var $msg="Salutare lume";
6     function scrie()
7     {
8         echo $this->msg;
9     }
10 }
11 //citirea sirului din fisier
12 $s=implode("",@file("fisier"));
13 $a=unserialize($s);
14 //dupa deserializare obiectul poate fi folosit
15 $a->show_one();
16 ?>

```

```

1
2 <?php
3 class A
4 {

```

```

5  var $msg="Salutare lume";
6  function scrie()
7  {
8      echo $this->msg;
9  }
10 }
11 //citirea sirului din fisier
12 $s=implode("",@file("fisier"));
13 $a=unserialize($s);
14 //dupa deserializare obiectul poate fi folosit
15 $a->show_one();
16 ?>

```

Referințele pot fi utilizate pentru a accesa conținutul unei variabile folosind mai multe nume. Spre deosebire de limbajul C, în PHP referințele nu sunt pointeri, ci alias-uri într-o tabela de simboluri. În PHP denumirile variabilelor și conținutul acestora nu sunt unul și același lucru. Așadar este posibil ca același conținut să aibă denumiri diferite.

Utilizarea referințelor

Referințele PHP permit unor variabile cu denumiri diferite să corespundă unui același conținut. Cu alte cuvinte, instrucțiunea

```

1  $a = &$b

1  $a = &$b

```

are ca efect faptul că **\$a** și **\$b** referă aceeași variabilă. În această situație **\$a** și **\$b** au același statut. Nu se poate spune că **\$a** referă **\$b** sau invers. O altă posibilitate de utilizare a referințelor este transmiterea prin referință a parametrilor unei funcții. Efectul unei astfel de transmisii este crearea unei variabile locale care referă spre același conținut ca variabila din contextul apelant. Să luăm în considerare următorul exemplu:

Cod:

```

1  <?php
2  function inc (&$var)
3  {
4      $var++;
5  }
6  $a=5;
7  inc($a);
8  ?>

1  <?php
2  function inc (&$var)
3  {
4      $var++;
5  }
6  $a=5;
7  inc($a);
8  ?>

```

Inițial valoarea variabilei **\$a** este 5. După apel variabila locală **\$var** și variabila din contextul apelant **\$a** indică spre același conținut. Valoarea păstrată în locația de memorie respectivă este incrementată (devine 6) prin

intermediul instrucțiunii `$var++;`.

Datorită faptului că cele două variabile au același conținut, valoarea variabilei **\$a** va fi 6 după executarea funcției.

Un parametru transmis prin referință poate fi:

- o variabilă;
- o instrucțiune **new**;
- o referință returnată de o funcție.

Dacă unei astfel de funcții i se transmite ca parametru un alt tip de expresie rezultatul este nedefinit. Așadar, pentru o funcție care are un parametru transmis prin referință nu se poate folosi o constantă în momentul apelului. De exemplu, pentru funcția *inc()* prezentată anterior nu este permis un apel de forma *inc(5)*.

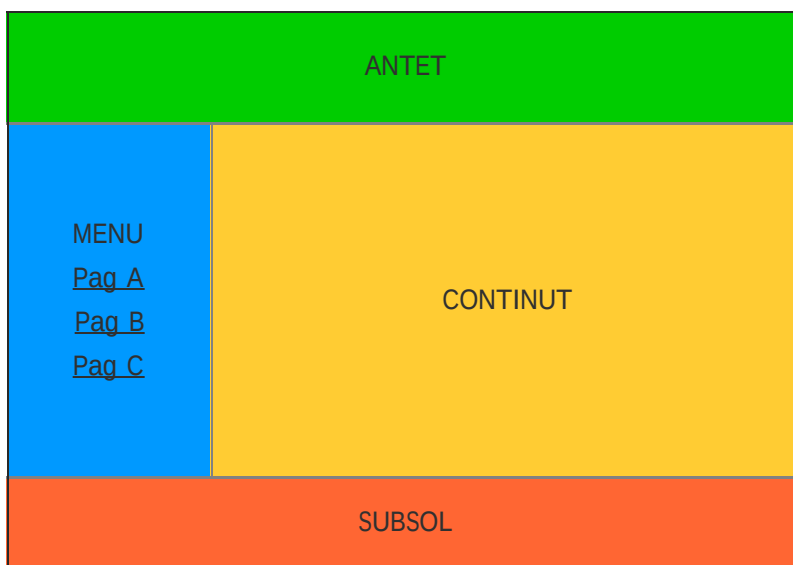
Referințe globale

În momentul declarării unei variabile globale (printr-o instrucțiune de tipul **global \$var**) se creează de fapt o referință spre o variabilă globală. Cu alte cuvinte, această instrucțiune este echivalentă cu `$var = &$GLOBALS["var"];`.

Referința \$this

În cadrul unei metode a unui obiect **\$this** este întotdeauna o referință spre obiectul care utilizează funcția (obiectul curent).

Construiește un site web cu PHP



index.php

```

1  <?php
2  include("antet.html");
3  ?>
4  <tr>
5
6  <?php
7  include("menu.html");
8  $k=$_GET['k'];
9  if (isset($k)) {include($k);}
10 else{
11 include("continut.html");
12 }

```

```
13
14     ?>
15 </tr>
16
17 <?php
18 include("subsol.html");
19 ?>

1
2 <?php
3 include("antet.html");
4 ?>
5 <tr>
6
7 <?php
8 include("menu.html");
9 $k=$_GET['k'];
10 if (isset($k)) {include($k);}
11 else{
12 include("continut.html");
13 }
14 ?>
15 </tr>
16
17 <?php
18 include("subsol.html");
19 ?>
```

antet.html

```
1
2 <html>
3 <head>
4 <title>Site în PHP cu tabele</title>
5 </head>
6
7 <body>
8 <table width="600" border="1" cellspacing="0" align="center">
9 <tr>
10 <td height="78" colspan="2" align="center" bgcolor="#00CC00" > ANTET </td> </tr>

1
2 <html>
3 <head>
4 <title>Site în PHP cu tabele</title>
5 </head>
6
7 <body>
8 <table width="600" border="1" cellspacing="0" align="center">
9 <tr>
10 <td height="78" colspan="2" align="center" bgcolor="#00CC00" > ANTET </td> </tr>
```

menu.html

```
1
2 <td width="60" height="270" align="center" bgcolor="#0099FF">
3 <p><strong>M E N U</p>
4 <hr color="blue">
5 <p><a href="index.php">Home</a></p>
6 <p><a href="index.php?k=p1.html">Pag A</a></p>
7 <p><a href="index.php?k=p2.html">Pag B</a></p>
8 <p><a href="index.php?k=p3.html">Pag C</a></p></strong>
9 </td>

1
2 <td width="60" height="270" align="center" bgcolor="#0099FF">
3 <p><strong>M E N U</p>
```

```

4 <hr color="blue">
5 <p><a href="index.php">Home</a></p>
6 <p><a href="index.php?k=p1.html">Pag A</a></p>
7 <p><a href="index.php?k=p2.html">Pag B</a></p>
8 <p><a href="index.php?k=p3.html">Pag C</a></p></strong>
  </td>

```

continut.html

```

1 <td width="325" align="center" bgcolor="#FFCC33"> CONTINUT </td>

1 <td width="325" align="center" bgcolor="#FFCC33"> CONTINUT </td>

```

subsol.html

```

1 <tr>
2 <td height="58" colspan="2" align="center" bgcolor="#FF6633">SUBSOL</td>
3 </tr>
4 </table>
5 </body>
6 </html>

1 <tr>
2 <td height="58" colspan="2" align="center" bgcolor="#FF6633">SUBSOL</td>
3 </tr>
4 </table>
5 </body>
6 </html>

```

p1.html

```

1 <td width="325" align="center" bgcolor="#FFCC33"> PAGINA A </td>

1 <td width="325" align="center" bgcolor="#FFCC33"> PAGINA A </td>

```

p2.html

```

1 <td width="325" align="center" bgcolor="#FFCC33"> PAGINA B </td>

1 <td width="325" align="center" bgcolor="#FFCC33"> PAGINA B </td>

```

p3.html

```

1 <td width="325" align="center" bgcolor="#FFCC33"> PAGINA C </td>

1 <td width="325" align="center" bgcolor="#FFCC33"> PAGINA C </td>

```

Vezi exemplul

Cîteva idei utile pentru securitatea paginilor web

Regula numărul unu a securității online este: nu vă încredeți niciodată în utilizator. Întotdeauna verificați datele trimise către server și "curățați-le" înainte de a le utiliza. Pentru aceasta trebuie să luați în considerare cîteva posibile "găuri" de securitate. Menționez că cele scrise aici sunt doar noțiuni generale, pentru o securitate ridicată ar trebui să aveți noțiuni specializate de baze de date și de administrare a serverelor web.

1. Variabilele globale

Avînd variabilele globale din *php.ini* setate pe ON puteți accesa datele trimise prin formulare mai simplu: *\$variabila* în loc de *\$_GET['variabila']*, la fel și pentru *POST* sau *FILES*. Dacă scriptul nu este foarte bine gîndit, variabilele globale pot prezenta un risc major de securitate. Din acest motiv *php.ini* este distribuit cu *globals=off* în ultimele versiuni.

2. Ghilimele "magice"

Dacă în *php.ini* *magic_quotes_gpc* sunt OFF folosiți funcția *addslashes* pentru a preceda ghilimelele din datele trimise de utilizatori cu caracterul `\`. Dacă *magic_quotes_gpc* sunt ON, PHP adaugă automat caracterul `\` înainte de ghilimele dar dacă ele sunt OFF, ghilimelele din input vă pot crea probleme serioase.

Ca exemplu, să presupunem că interogarea SQL de verificare a numelui și a parolei pentru înregistrarea pe site este

```
1 SELECT * FROM users WHERE nume='$nume' AND parola='$parola'

1 SELECT * FROM users WHERE nume='$nume' AND parola='$parola'
```

și dacă interogarea este executată cu succes, utilizatorul este logat. În acest caz, folosind parola 'OR' 1=1 oricine poate avea acces pe site deoarece interogarea

```
1 SELECT * FROM user WHERE nume='un nume oarecare' AND parola='' OR '1=1'

1 SELECT * FROM user WHERE nume='un nume oarecare' AND parola='' OR '1=1'
```

este executată cu succes și returnează un rezultat (toate înregistrările din baza de date, de fapt). Mai mult, atunci cînd vă așteptați ca baza de date să returneze un singur rînd, verificați acest lucru și nu dacă interogarea s-a executat cu succes. Implicit în *php.ini* *magic_quotes_gpc* sunt ON pentru a vă proteja de astfel de atacuri, dar este bine să verificați înainte de a renunța la *addslashes*.

3. Includere

Încercați să evitați includerea "vizibilă" a fișierelor în forma `http://site.md/fisier.php?file=cutare.html` pentru a include fișiere în cadrul unei pagini. Cu puțină neatenție din partea voastră atacatorul ar putea accesa astfel de

informații sensibile din cadrul sistemului. Nu includeți fișiere străine. PHP poate "include" fișiere aflate pe alte servere decât cel care rulează dacă setarea URL *fopen wrappers* este activată în *php.ini*. În exemplul de mai sus, un atacator ar fi putut accesa adresa:

`http://site.md/fisier.php?file=http://www.rau.md/scriptultau.php`

pentru a include în fișier un script localizat pe alt server și astfel obține acces către toate resursele sistemului la care are acces PHP, putînd rula comenzi de sistem, afișa informații confidențiale sau șterge baza de date. Nu permiteți includerea fișierelor din altă parte decât de pe serverul vostru. Setati *allow_url_fopen=OFF* în *php.ini*.

4. Formulare

Folosiți metoda *POST* în formulare atunci cînd informația din acestea urmează să fie introdusă în baza de date. Dacă variabilele globale sunt *OFF* în *php.ini* sau metoda de transmitere a formularului este *GET*, un utilizator rău intenționat ar putea accesa adresa dvs. și ar introduce comentariul lui (exemplu "blabla") în baza voastră de date fără să treacă propriu-zis prin site. Așa vă puteți trezi că nu mai aveți spațiu pe server iar în baza de date sunt cîteva milioane de comentarii care spun același lucru "blabla".

Verificarea provenienței cererilor către server este foarte importantă și în alt caz: formularele de loghin. Cineva care știe numele de utilizator ar putea încerca să vă găsească parola foarte ușor. În acest caz va trebui să puneți o protecție suplimentară care să nu permită mai mult de X încercări consecutive eșuate de logare pentru un nume de utilizator. Această problemă se rezolvă ușor folosind sesiunile. În momentul cînd cineva încearcă să trimită numele și parola din formular putem seta o variabilă de sesiune `$_SESSION['login_count']` care să țină minte numărul de încercări. Cînd valoarea acesteia trece de 3 (încercări nereușite) nici măcar nu mai interogați baza de date pentru a verifica datele trimise. Variabila de sesiune va rămîne în memorie cît timp browserul este deschis și sesiunea activă (opțiunea implicită a PHP de menținere a sesiunilor active este de o oră). Dacă atacatorul așteaptă o ora sau își închide browserul, sesiunea va fi închisă și va putea de alte X ori să se logheze. De cele mai multe ori această măsură de siguranță este suficientă pentru a preveni încercările de aflare a parolelor.

5. Extensii

O practică obișnuită este de a acorda extensia *.inc* fișierelor care conțin biblioteci de funcții ce urmează a fi incluse și folosite în cod. PHP nu parsează fișierele cu extensia *.inc* și dacă acestea sunt apelate direct ele sunt trimise plain text către browser. Nu puneți informații sensibile (precum nume și parola) în fișiere cu extensia *.inc*, *.txt*, sau *.html* care pot fi accesate și văzute. Folosiți pentru aceste fișiere extensia *.php* care, dacă sunt accesate direct, vor fi rulate fără să afișeze informațiile conținute în ele.

6. Comanda CHMOD- setarea atributelor

Comanda CHMOD (abreviere pentru **change mode** permissions of a file) este folosită pentru a schimba modul de acces (de permisiune) a fișierelor și directoarelor de pe server. Setarea poate afecta modul cum poate fi citit, sau cum se poate executa un fișier pe server. De exemplu, dacă aveți un fișier *.php* care trebuie să execute o comandă de scriere pe server într-un fișier *.txt* trebuie să îi dați dreptul să poată fi executat iar la fișierul *.txt* trebuie să îi dați dreptul de a se putea scrie în el. Aveți 3 tipuri de acces - *OWNER* - *GROUP* - *WORLD* - fiecare cu cîte 3 setări - *READ* - *WRITE* - *EXECUTE* -

Este bine ca la *WORLD* să nu dați decât acces la citire (pentru a putea accesa paginile de pe site) dar nu și drepturi de scriere sau execuție (pentru că nu doriți ca oricine să vă scrie ce vrea în respectivul fișier *.txt*)

În programul TOTAL COMMANDER (pe care eu îl folosesc și pentru FTP) setarea atributelor pe server se face din meniul FILES - CHANGES ATTRIBUTES. Fiecare program de FTP are în meniul lui setare pentru attributele fișierelor

de pe server.

7. Sesiuni

Dacă aveți o secțiune de administrare pe site, unde accesul este restrictionat doar la membrii de exemplu, este bine ca în toate paginile din această secțiune să includă o pagină de verificare a accesului. Această pagina va verifica la fiecare accesare dacă utilizatorul este înregistrat și are acces pe pagina respectivă și permite rularea paginii doar dacă utilizatorul este înregistrat. Fără această verificare, un utilizator ar putea accesa paginile din secțiunea de administrare fără să treacă prin formularul de înregistrare. După autentificarea propriu-zisă, vom folosi variabile de sesiune pentru a păstra în memorie câteva informații despre autentificare, pentru a le verifica mai târziu, atunci când accesăm alte pagini din cadrul secțiunii de administrare. Pornim întâi sesiunea după care trecem la salvarea informațiilor în ea ca în exemplul de mai jos:

```
1 session_start();
2
3 $_SESSION ['nume_admin'] = $_POST ['nume'];
  $_SESSION ['parola_encriptata'] = $parolaEncriptata;
```

```
1 session_start();
2
3 $_SESSION ['nume_admin'] = $_POST ['nume'];
  $_SESSION ['parola_encriptata'] = $parolaEncriptata;
```

Pe lângă acestea, pentru o și mai mare siguranță, vom salva id-ul sesiunii în altă variabilă. Toate sesiunile au un id unic, un string care seamănă cu rezultatul unei criptări MD5:

```
1 $_SESSION ['key_admin'] = session_id();
```

```
1 $_SESSION ['key_admin'] = session_id();
```

Cu autentificare făcută, spunem scriptului să încarce prima pagină din secțiunea de administrare:

```
1 header ("location: admin.php");
```

```
1 header ("location: admin.php");
```

În continuare pentru a împiedica accesul neautorizat la paginile din această secțiune, scriem un mic script de verificare a datelor sesiunii înainte de a încărca orice pagină din secțiunea de administrare:

```
1 session_start();
2
3 if ($_SESSION ['key_admin'] != session_id ()) {
4     print 'Acces neautorizat!';
5     exit;
6 }
```

```
1 session_start();
2
3 if ($_SESSION ['key_admin'] != session_id ()) {
4     print 'Acces neautorizat!';
```

```
5 | exit;
   | }
```

8. Loguri de acces

Pentru o verificare ulterioară a persoanelor care încearcă să intre în paginile restrictionate de pe site, este bine să salvăm în baza de date (sau într-un fișier text) numele, parola criptată, ora, data, semnătura browserului și ip-ul. Așa veți ști cine a încercat să vă spargă site-ul și puteți crea un script care să verifice de exemplu ip-ul și dacă corespunde cu unul din cele care sunt blocate să nu aibă acces la nici una din pagini (chiar dacă găsește userul și parola corecte)

9. Criptarea parolelor cu MD5

Parolele care sunt salvate în baza de date este recomandat să le țineți criptate, astfel dacă cineva încearcă să citească parolele le va vedea criptate.

```
1 | INSERT INTO admin VALUES ("administrator", md5("parola"));

1 | INSERT INTO admin VALUES ("administrator", md5("parola"));
```

Criptarea folosind md5 nu este reversibilă (și astfel nici dvs., nici altcineva nu o va putea afla chiar dacă are acces la baza de date).

10. Pagina index.php în subdirectoare

Dacă aveți mai multe directoare și subdirectoare pe server este recomandat pentru a preveni accesul la datele aflate în aceste directoare să introduceți în fiecare director o pagină numită *index.html* sau *index.php* care să facă redirectarea automat către pagina principală a site-ului. Astfel minimizați riscul să intre cineva și să vadă tot ce este în aceste subdirectoare.

11. Fișierul .htaccess

Pentru a bloca accesul la un anumit director (sau chiar la tot site-ul) puteți crea un fișier numit *.htaccess* (cu punct înainte) în care să introduceți ip-ul care doriți să îl blocați (sau care să aibă acces)

Order Deny,Allow

Allow from all

În exemplul de mai sus, toată lumea are acces. Dacă de exemplu ip-ul 192.168.1.1 nu doriți să aibă acces la acel director scrieți

Deny from 192.168.1.1

Atenție, să nu vă blocați singuri ip-ul la site, că altfel trebuie să luați legătura cu administratorul serverului să ștergă fișierul ca să puteți intra din nou pe site, sau să scrieți prin FTP un nou fișier *.htaccess*. (la subdirectoare ștergeți directorul sau salvați un alt fișier *.htaccess*) În cazul în care după de ați pus fișierul pe server nu îl vedeți, el este acolo dar serverul setează acest nume de fișier ca fișier ascuns.

12. Roboții de căutare

Motoarele de căutare pot indexa tot ce se află la voi pe site. Dacă doriți ca un fișier sau director să nu fie indexat de către motoarele de căutare, trebuie să creați un fișier numit *robots.txt* care să se afle în directorul principal de pe site (nu în subdirectoare). În el scrieți următoarele comenzi:

robots.txt for http://web.ceiti.md

User-agent: *

Disallow: /cache/

Disallow: /admin

disallow: /search

unde pentru fiecare director sau fișier se specifică locația și numele. Pentru mai multă siguranță, am găsit pe web un mic script care verifică tipul de browser și dacă nu este un browser cunoscut nu permite accesul (dacă cineva încearcă să citească paginile cu programe spion sau gen web offline).

Cum am specificat și la începutul acestui articol, cele spuse mai sus nunt doar cateva indicii și idei pentru a vă putea proteja mai bine paginile.

Preluarea datelor din formulare

Înainte de a vorbi despre preluarea datelor dintr-un formular, trebuie în primul rând să cunoașteți sintaxa HTML pentru crearea unui formular.

Un formular este delimitat de elementul **<form>** care conține alte câteva elemente numite “controale”, care au o varietate de metode de a prelua informații. Fiecare element din formular are un nume cât și o valoare, astfel încât datele transferate pentru procesare să fie sub forma unor perechi nume/valoare.

Elementul <form>

```
1
2 <form [action=url] [method=get/post] [name=nume] >
3
4 Elementele formularului
5
6 </form>
```

```
1
2 <form [action=url] [method=get/post] [name=nume] >
3
4 Elementele formularului
5
6 </form>
```

Elementul <input>

```
1
2 <input [type=text|password|checkbox|radio|submit|image|reset|button|hidden|file] [name=nume]
3 [value=valoare] [checked] [disabled] [readonly] [size=latime] [maxlength=cuvinte_maxime]
4 [src=url]
5 [alt=altText] [usemap=url] [align=left|center|right|justify] [tabindex=numar]
  [accesskey=keyCombo]
  [onfocus=script] [onblur=script] [onselect=script] [onchange=script] [accept=set_caractere]
  [events]>
```

```
1
2 <input [type=text|password|checkbox|radio|submit|image|reset|button|hidden|file] [name=nume]
3 [value=valoare] [checked] [disabled] [readonly] [size=latime] [maxlength=cuvinte_maxime]
4 [src=url]
```

```
5 | [alt=altText] [usemap=url] [align=left|center|right|justify] [tabindex=numar]
   | [accesskey=keyCombo]
   | [onfocus=script] [onblur=script] [onselect=script] [onchange=script] [accept=set_caractere]
   | [events]>
```

Acest element **<input>** este cel mai important în utilizarea formularelor.

Explicarea valorilor atributului *type* ale elementului <input>

Descriere	Valoare	Exemplu	Rezultat
Caseta text	text	<pre>1 <input type="text" name="nume" value="valoare"></pre> <pre>1 <input type="text" name="nume" value="valoare"></pre>	valoare
Casete de introducere a parolei	password	<pre>1 <input type="password" name="nume" value="valoare"></pre> <pre>1 <input type="password" name="nume" value="valoare"></pre>	
Elemente ascunse	hidden	<pre>1 <input type="hidden" name="nume" value="valoare"></pre> <pre>1 <input type="hidden" name="nume" value="valoare"></pre>	
Casete de validare	checkbox	<pre>1 <input type="checkbox" name="nume" value="valoare"></pre> <pre>1 <input type="checkbox"</pre>	☐

		<pre>name="nume" value="valoare"></pre>	
Butoane radio	radio	<pre>1 <input type="radio" name="nume" value="valoare"> 1 <input type="radio" name="nume" value="valoare"></pre>	
Fisiere incluse	file	<pre>1 <input type="file" name="nume" value="valoare"> 1 <input type="file" name="nume" value="valoare"></pre>	
Buton de trimitere	submit	<pre>1 <input type="subbbmit" name="Trimite" value="Trimite"> 1 <input type="subbbmit" name="Trimite" value="Trimite"></pre>	
Butoane personale	button	<pre>1 <input type="button" name="Buton" value="Buton"> 1 <input type="button" name="Buton" value="Buton"></pre>	
Imagini	image	<pre>1 <input type="image" name="buton" src="poza_buton.gif"> 1 </pre>	

		<code><input type="image" name="buton" src="poza_buton.gif"></code>	
Butoane de resetare	reset	<pre>1 <input type="reset" name="Reseteaza" value="Reseteaza"> 1 <input type="reset" name="Reseteaza" value="Reseteaza"></pre>	

Elementul <select>

Acest element este folosit pentru crearea unei liste de opțiuni, fie ca un meniu care se desfășoară, fie ca o casetă cu listă.

```
1 | <select [name=nume] [size=latime] [multiple] [disabled] [tabindex=numar] [onfocus=script]  
2 | [onblur=script]  
3 | [onchange=script] [events]>  
4 |  
5 | Elementele din select  
6 | </select>  
  
1 | <select [name=nume] [size=latime] [multiple] [disabled] [tabindex=numar] [onfocus=script]  
2 | [onblur=script]  
3 | [onchange=script] [events]>  
4 |  
5 | Elementele din select  
6 | </select>
```

Fiecare din opțiunile din listă reprezintă un element <option>.

```
1 | <option [selected] [disabled] [value=valoare] [core] [international] [events]>Nume</option>  
  
1 | <option [selected] [disabled] [value=valoare] [core] [international] [events]>Nume</option>
```

Exemplul 1:

```
1 | <select name="test">  
2 | <option value="optiune 1">Optiune 1</option>  
3 | <option value="optiune 2">Optiune 2</option>  
4 | <option value="optiune 3">Optiune 3</option>  
5 | <option value="optiune 4">Optiune 4</option>  
6 | </select>
```

```
1
2 <select name="test">
3 <option value="optiune 1">Optiune 1</option>
4 <option value="optiune 2">Optiune 2</option>
5 <option value="optiune 3">Optiune 3</option>
6 <option value="optiune 4">Optiune 4</option>
  </select>
```

Optiune 1

Exemplul 2:

```
1
2 <select name="test" multiple size="3">
3 <option value="optiune 1">Optiune 1</option>
4 <option value="optiune 2">Optiune 2</option>
5 <option value="optiune 3">Optiune 3</option>
6 <option value="optiune 4">Optiune 4</option>
7 <option value="optiune 5">Optiune 5</option>
8 <option value="optiune 6">Optiune 6</option>
  </select>

1
2 <select name="test" multiple size="3">
3 <option value="optiune 1">Optiune 1</option>
4 <option value="optiune 2">Optiune 2</option>
5 <option value="optiune 3">Optiune 3</option>
6 <option value="optiune 4">Optiune 4</option>
7 <option value="optiune 5">Optiune 5</option>
8 <option value="optiune 6">Optiune 6</option>
  </select>
```

Optiune 1

Optiune 2

Optiune 3

Optiune 5

Elementul <textarea>

Acest element este asemanator cu cel text numai că aici se poate tasta într-o secțiune mult mai mare decat în cazul text.

```
1
2 <textarea [name=nume] [rows=nr_randuri] [cols=nr_coloane] [disabled] [readonly] [tabindex=numar]
  [onfocus=script] [onblur=script] [onselect=script] [onchange=script] [events]> </textarea>

1
2 <textarea [name=nume] [rows=nr_randuri] [cols=nr_coloane] [disabled] [readonly] [tabindex=numar]
  [onfocus=script] [onblur=script] [onselect=script] [onchange=script] [events]> </textarea>
```

Exemplu textarea:

```
1
  <textarea name="nume" cols="40" rows="10">Text initial</textarea>
```

```
1 | <textarea name="nume" cols="40" rows="10">Text initial</textarea>
```

Text initial

Exemplu 1:

```
1 | <form action="formular.php" method="post">
2 |
3 |
4 | Nume: <input type="text" name="nume" value="" size="30" maxLength="15"> <br><br>
5 | Prenume: <input type="text" name="prenume" value=""> <br><br>
6 | Varsta: sub 16 <input type="radio" name="varsta" value="sub16"> | peste 16
7 |             <input type="radio" name="varsta" value="peste16"> <br><br>
8 | Parola dorita: <input type="password" name="parola" value=""> <br><br>
9 |
10 | <input type="submit" name="Trimite" value="Trimite">
11 | <input type="reset" name="Reseteaza" value="Reseteaza">
12 |
13 | </form>
```

```
1 | <form action="formular.php" method="post">
2 |
3 |
4 | Nume: <input type="text" name="nume" value="" size="30" maxLength="15"> <br><br>
5 | Prenume: <input type="text" name="prenume" value=""> <br><br>
6 | Varsta: sub 16 <input type="radio" name="varsta" value="sub16"> | peste 16
7 |             <input type="radio" name="varsta" value="peste16"> <br><br>
8 | Parola dorita: <input type="password" name="parola" value=""> <br><br>
9 |
10 | <input type="submit" name="Trimite" value="Trimite">
11 | <input type="reset" name="Reseteaza" value="Reseteaza">
12 |
13 | </form>
```

Nume:

Prenume:

Vârsta:

☐ sub 16 ani

☐ peste 16 ani

Parola dorită:

Trimite

Reseteaza

Puneți codul într-un fișier **formular.html**, salvați și apoi vizualizați în browser accesând <http://localhost/formular.html>

Am vorbit până acum despre formulare, cum să le creăm și ce elemente au.

Hai să ne aducem aminte ca în interiorul etichetei **<form>** avem puse atributele **action** și **method** (care poate fi POST sau GET). În atributul **action** se pune calea către scriptul PHP care prelucrează datele iar în **method** se pune metoda prin care se vor transmite datele atunci când butonul **“Trimite”** este apăsat.

POST - această metodă face ca datele trimise prin formular să nu fie vizibile utilizatorului, să fie trimise în spatele paginii web.

GET - prin această metodă, datele trimise prin formular sunt vizibile în URL (URL este adresa ce este afișată în browser).

Acum că am creat formularul, avem pagina **formular.html**, haideți să cream și pagina prin care arătăm datele trimise prin formular.

Creați o pagină **formular.php** și introduceți codul următor:

```
1
2 <?php
3
4 echo $_POST['nume'];
5
?>
```

```
1
2 <?php
3
4 echo $_POST['nume'];
5
?>
```

Salvați și testați în browser accesând <http://localhost/formular.html>, completați câmpul **“Nume”**, și apoi apăsați butonul **“Trimite”**.

Observați că în pagina **formular.php** ne este afișat numele introdus în câmpul **“Nume”** din pagina anterioară, și anume **formular.html**.

Adăugăm în continuare variabilele **\$_POST** corespunzătoare formularului nostru, și apoi testăm din nou.

```
1
2 <?php
3
4 echo 'Nume:      ' . $_POST['nume'] . '      <br>
5      Prenume:   ' . $_POST['prenume'] . '   <br>
6      Varsta:    ' . $_POST['varsta'] . '    <br>
7      Parola:    ' . $_POST['parola'] . '    <br>';
8
?>
```

```
1
2 <?php
3
4 echo 'Nume:      ' . $_POST['nume'] . '      <br>
5      Prenume:   ' . $_POST['prenume'] . '   <br>
6      Varsta:    ' . $_POST['varsta'] . '    <br>
7      Parola:    ' . $_POST['parola'] . '    <br>';
8
?>
```

După cum vedeți, ne folosim de valoarea **POST**, iar datele nu sunt afișate decât dacă le definim noi **\$_POST['nume']** în pagina **formular.php**.

Numele din interiorul variabilei **\$_POST** este numele din interiorul elementelor din formular.

De exemplu, în formularul nostru avem input-ul de la nume așa:

```
<input type="text" name="nume" value="" size="30" maxLength="15">
```

Observați că în câmpul **name** valoarea acestuia este **nume**. Acesta se plasează în variabila **\$_POST**, în cazul nostru **\$_POST['nume']**.

În input-ul pentru prenume avem **<input type="text" name="prenume" value="">** iar acesta se plasează în **\$_POST** așa: **\$_POST['prenume']**, ... și tot așa.

Observați câmpul **value** care nu are nimic definit. Nu este nici o greșeală, veți învăța mai târziu de ce am lăsat gol acel câmp.

Am cam terminat cu metoda **POST**, acum haideți să lucrăm și să ne familiarizăm și cu metoda **GET**.

În același formular din pagina **formular.html**, numai că în eticheta **<form>**, în loc de **method="post"**, vom pune **method="get"**.

Apoi în pagina **formular.php**, în loc de variabilele **\$_POST**, vom pune variabile **\$_GET**.

În caz că nu ați înțeles, deschideți pagina **formular.html**, modificați linia

```
<form action="formular.php" method="post"> în <form action="formular.php" method="get">
```

Apoi deschideți pagina **formular.php** și modificați în:

```
1  <?php
2
3
4  echo 'Nume:      ' . $_GET['nume'] . '      <br>
5      Prenume:   ' . $_GET['prenume'] . '   <br>
6      Varsta:    ' . $_GET['varsta'] . '    <br>
7      Parola:    ' . $_GET['parola'] . '    <br>';
8
9  ?>
```

```
1  <?php
2
3
4  echo 'Nume:      ' . $_GET['nume'] . '      <br>
5      Prenume:   ' . $_GET['prenume'] . '   <br>
6      Varsta:    ' . $_GET['varsta'] . '    <br>
7      Parola:    ' . $_GET['parola'] . '    <br>';
8
9  ?>
```

Observați că rezultatul este același în pagina web, numai că informațiile formularului sunt postate și în adresa din browser (url) sub forma: **http://localhost/formular.php?nume=ciobanu&prenume=andrei&varsta=peste16&parola=altaparola&Trimite=Trimite**

De recomandat este să folosiți metoda **POST**, pentru că este mult mai sigură.

Exemplu 2:

Formularul (**form.html**)

```
1  <!doctype html>
2  <html>
3  <head>
4  <meta charset="utf-8">
5  <title>Formular</title>
6  </head>
7  <body bgcolor="#006699" text="#fff">
```

```
9  <form method="post" action="form.php">
10  <p>Nume: <input type="text" name="nume">
11  </p>
12  <p>Prenume: <input type="text" name="prenume">
13  </p>
14  <p>Virsta: <input type="text" name="virsta">
15  </p>
16  <p>Sex:<br>
17  <input type="radio" name="sex" value="Masculin"> Masculin<br>
18  <input type="radio" name="sex" value="Feminin"> Feminin</p>
19  <p>Limbi cunoscute:<br />
20  <input name="limba1" type="checkbox" value="L.Romana"> L.Româna<br>
21  <input name="limba2" type="checkbox" value="L.Rusa"> L.Rus ă<br>
22  <input name="limba3" type="checkbox" value="L.Engleza"> L.Engleza<br>
23  <input name="limba4" type="checkbox" value="L.Franceza"> L.Franceza
24  </p>
25  <p>
26  Universitatea absolvită:
27  <select name="universitate" size="1">
28  <option value="USM">Universitatea de Stat din Moldova</option>
29  <option value="UST">Universitatea de Stat din Tiraspol</option>
30  <option value="UTM">Universitatea Tehnică din Moldova</option>
31  <option value="ASEM">Academia de Studii Economice</option>
32  </select>
33  </p>
34  <p>Comentarii:<br />
35  <textarea name="comentarii" cols="45" rows="8"></textarea>
36  </p>
37  <p>
38  <input type="submit" name="submit" value="Expediază">
39  </p>
40 </form>
41 </body>
</html>
```

```
1  <!doctype html>
2  <html>
3  <head>
4  <meta charset="utf-8">
5  <title>Formular</title>
6  </head>
7  <body bgcolor="#006699" text="#fff">
8  <form method="post" action="form.php">
9  <p>Nume: <input type="text" name="nume">
10 </p>
11 <p>Prenume: <input type="text" name="prenume">
12 </p>
13 <p>Virsta: <input type="text" name="virsta">
14 </p>
15 <p>Sex:<br>
16 <input type="radio" name="sex" value="Masculin"> Masculin<br>
17 <input type="radio" name="sex" value="Feminin"> Feminin</p>
18 <p>Limbi cunoscute:<br />
19 <input name="limba1" type="checkbox" value="L.Romana"> L.Româna<br>
20 <input name="limba2" type="checkbox" value="L.Rusa"> L.Rus ă<br>
21 <input name="limba3" type="checkbox" value="L.Engleza"> L.Engleza<br>
22 <input name="limba4" type="checkbox" value="L.Franceza"> L.Franceza
23 </p>
24 <p>
25 Universitatea absolvită:
26 <select name="universitate" size="1">
27 <option value="USM">Universitatea de Stat din Moldova</option>
28 <option value="UST">Universitatea de Stat din Tiraspol</option>
29 <option value="UTM">Universitatea Tehnică din Moldova</option>
30 <option value="ASEM">Academia de Studii Economice</option>
31 </select>
32 </p>
33 <p>Comentarii:<br />
34 <textarea name="comentarii" cols="45" rows="8"></textarea>
35 </p>
36 <p>
37 <input type="submit" name="submit" value="Expediază">
38 </p>
```

```

39
40 </form>
41 </body>
    </html>

```

Nume:

Prenume:

Virsta:

Sex:

☐ Masculin

☐ Feminin

Limbi cunoscute:

☐ L.Română

☐ L.Rusă

☐ L.Engleza

☐ L.Franceza

Universitatea absolvită:

Comentarii:

Scriptul (**form.php**)care preia datele din formular pentru afişare: